Seneca | SCHOOL OF INFORMATION AND COMMUNICATIONS TECHNOLOGY

A Technical Report Of CAA900NCE – Capstone Project On Secure Cloud-Based E-Commerce Platform: SOS (SEA of Style)

Submitted By:
Group 8
Date: 12 April 2026

Team Member and their Responsibilities

Shirish Anand Joshi	:	Infrastructure Engineer
Eldar Azimov	:	Lead DevOps Engineer
Amir Bhandari	:	Cloud Solutions Architect

Submitted To:
Professor: Mehdi Akbari
Cloud Architecture and Administration

Table of Contents

Table of Figures

Introduction

The SEA of Style (SOS) project is a secure, cloud-based e-commerce platform developed to simulate a real-world online shopping system. The application enables users to browse products, create accounts, place orders, and receive notifications, while maintaining a strong focus on security, scalability, and performance. The system is built using Amazon Web Services (AWS) and follows a serverless architecture approach, eliminating the need for traditional server management. By leveraging managed services such as Amazon S3, Amazon CloudFront, Amazon API Gateway, AWS Lambda, and Amazon DynamoDB, the platform can automatically scale based on user demand while reducing operational complexity.

The architecture is structured into multiple layers, each responsible for a specific function, including frontend delivery, authentication, API processing, data storage, notification services, and monitoring. This separation of concerns improves maintainability and allows each component to operate independently while contributing to the overall system. Security is integrated throughout the design using Amazon Cognito for user authentication and AWS Identity and Access Management (IAM) for controlled access between services. In addition, AWS Key Management Service (KMS) is used to encrypt sensitive data at rest, ensuring data protection across the platform.

The SOS platform also incorporates additional features to enhance both functionality and user experience. Amazon Lex is integrated to provide a conversational chatbot interface, allowing users to interact with the system using natural language. Furthermore, the project demonstrates modern DevOps practices using Terraform and GitHub Actions for infrastructure automation. This Infrastructure as Code (IaC) approach enables consistent, repeatable, and secure deployment of cloud resources without relying on manual configuration.

Overall, the project highlights how a modern cloud-native application can be designed using serverless and event-driven principles. It demonstrates the practical implementation of scalable architecture, secure authentication mechanisms, efficient data management, and automated infrastructure deployment, reflecting real-world industry practices.

System Overview

The SOS platform is designed as a multi-layered cloud application, where each component is responsible for handling specific aspects of user interaction and system operations. When a user accesses the website, the request is first routed through Amazon CloudFront, which delivers static frontend content stored in Amazon S3. By using a global content delivery network (CDN), the platform ensures low latency, faster load times, and consistent user experience across different geographic locations.

User authentication is managed using Amazon Cognito, which handles user registration, login, and identity verification. Once authenticated, users receive a secure JSON Web Token (JWT) that is used to authorize all subsequent API requests. All communication between the frontend and backend services is routed through Amazon API Gateway, which acts as the central entry point for the application. The API Gateway validates incoming requests and forwards them to AWS Lambda functions, where the core business logic of the application is executed.

In addition to standard API interactions, the system also integrates Amazon Lex to provide a chatbot-based interface. This allows users to interact with the application using conversational inputs, which are processed and routed to AWS Lambda functions for further handling. This feature enhances usability and introduces an alternative interaction method beyond traditional web navigation.

The application data, including product details and customer orders, is stored in Amazon DynamoDB, which provides a highly scalable and low-latency NoSQL database solution. For event-driven communication and notifications, the platform uses Amazon SNS together with a Telegram bot integration to deliver real-time purchase alerts to administrators. Monitoring and logging are handled by Amazon CloudWatch, which collects system logs and performance metrics, while AWS CloudTrail records control plane activities for auditing and security purposes.

In addition to the application runtime components, the platform also incorporates an infrastructure automation layer using Terraform. This is integrated with a CI/CD pipeline powered by GitHub Actions, which uses OpenID Connect (OIDC) to securely assume AWS IAM roles without storing credentials. Terraform manages infrastructure provisioning, with its state stored in Amazon S3 and state locking handled by DynamoDB. This setup enables automated, consistent, and secure deployment of resources, supporting modern DevOps practices. Together, these components form a cohesive and scalable system that supports the complete lifecycle of the application, from user interaction and data processing to monitoring, security, and infrastructure management.

SOS-Sea of Style Infrastructure Diagram

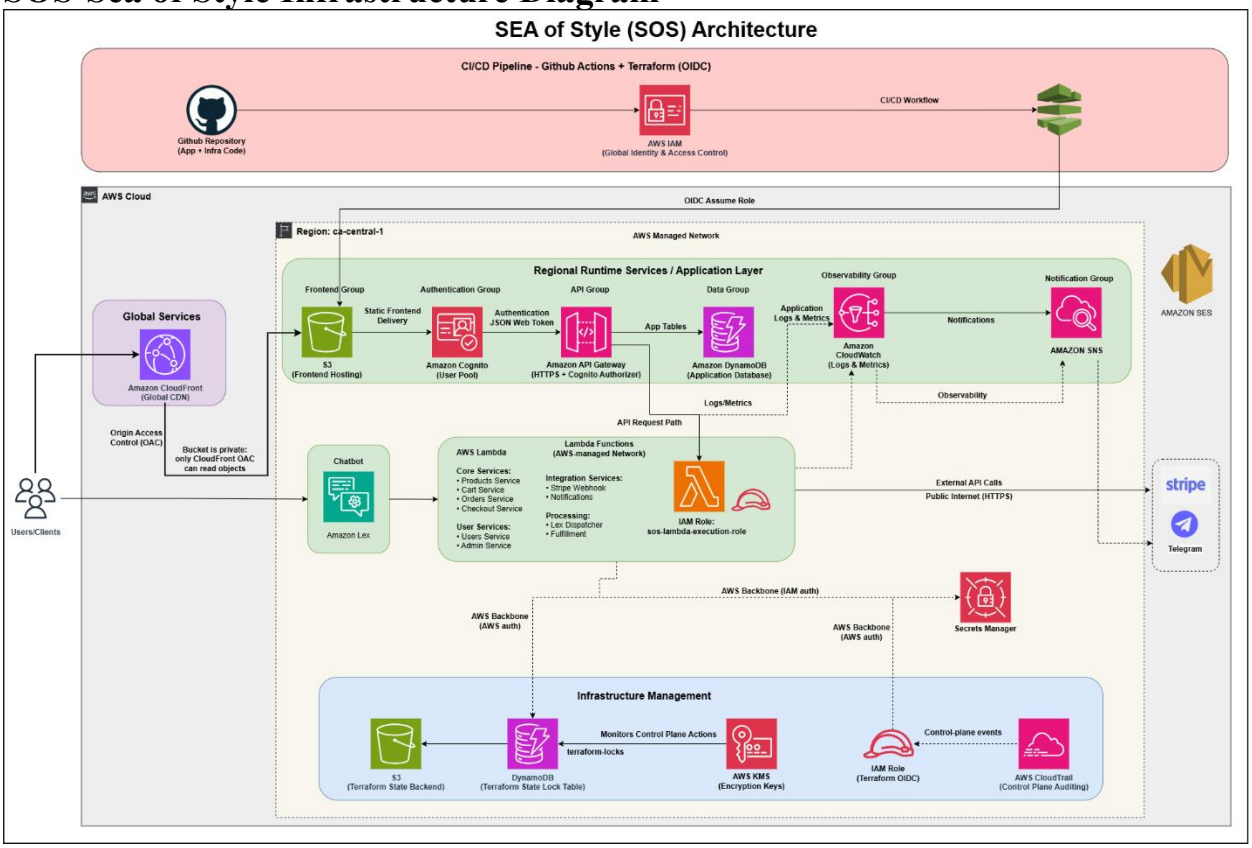


Figure 1: Architecture Diagram.

Figure 1 presents the overall architecture of the SOS platform and illustrates how various AWS services interact to deliver a secure, scalable, and serverless e-commerce solution. The system is organized into multiple logical layers, including frontend delivery, authentication, API processing, data storage, observability, notification services, and infrastructure management.

User requests are first handled by Amazon CloudFront, which delivers static frontend content from a private Amazon S3 bucket. Authentication is managed through Amazon Cognito, while backend communication is routed through Amazon API Gateway to AWS Lambda functions that execute the core application logic. The system also integrates Amazon Lex to provide a chatbot interface, enabling users to interact with the platform using natural language.

Application data is stored in Amazon DynamoDB, and notifications are handled through Amazon SNS and Telegram bot integration for real-time admin alerting. Monitoring and logging are managed by Amazon CloudWatch, with AWS CloudTrail providing auditing capabilities. In addition, the architecture includes a CI/CD pipeline using GitHub Actions and Terraform, which automates infrastructure provisioning through secure IAM role assumptions using OIDC. Overall, the diagram highlights a fully serverless, event-driven design that emphasizes scalability, security, and operational efficiency.

Explanation of All Services

Frontend Layer

The frontend of the SOS application is hosted on Amazon S3, where all static assets such as HTML, CSS, and JavaScript files are stored. To improve performance and user experience, Amazon CloudFront is used as a global content delivery network (CDN). CloudFront caches content at edge locations, reducing latency and ensuring faster access for users across different regions. For security purposes, the S3 bucket is configured as private, and access is restricted using Origin Access Control (OAC). This ensures that users cannot directly access the storage layer, and all requests must pass through CloudFront, adding an additional layer of protection.

Authentication Layer

User authentication and identity management are handled using Amazon Cognito. It provides secure user registration, login, and session management without requiring custom authentication logic. Once a user successfully authenticates, Cognito generates a JSON Web Token (JWT), which is used to authorize API requests. Amazon API Gateway integrates with Cognito through a built-in authorizer, ensuring that only authenticated users can access protected backend resources. This approach enhances security by validating user identity before allowing access to application services.

API & Compute Layer

Amazon API Gateway serves as the central entry point for all backend requests. It receives incoming API calls from the frontend and validates them using Cognito authentication. After validation, API Gateway routes the requests to AWS Lambda functions, where the business logic of the application is executed. The application follows a microservices-based approach using multiple Lambda functions. These functions handle core operations such as product management, cart processing, order handling, user services, and administrative tasks. In addition, integration services such as Stripe webhooks and notification processing are also managed through Lambda

functions. Amazon Lex is also integrated within this layer to provide a conversational chatbot interface. User inputs from the chatbot are processed by Lex and forwarded to Lambda functions, enabling dynamic and interactive communication with the system

Data Layer
Amazon DynamoDB is used as the primary database for the SOS platform. It stores application data such as product information, customer orders, and user-related data. DynamoDB is a fully managed NoSQL database that offers high availability, low latency, and automatic scaling. This makes it well-suited for handling dynamic workloads and real-time application requirements, ensuring that the system can efficiently manage increasing user traffic without performance degradation.

Notification Layer
The SOS platform uses Amazon SNS to handle event-driven notifications within the system. When a user completes a purchase, an event is generated and published through SNS, which is then processed by the notification workflow and forwarded to a Telegram bot. This allows administrators to receive real-time purchase alerts in the admin Telegram group. This event-driven approach ensures reliable and scalable communication.

Observability & Monitoring
Amazon CloudWatch is used to monitor system performance and collect logs from various services such as Lambda and API Gateway. It provides insights into application behavior, helping identify issues and track performance metrics. AWS CloudTrail complements this by recording all control plane activities, such as resource creation and configuration changes. This provides an audit trail that supports security, compliance, and troubleshooting

Security & IAM
Security within the SOS platform is implemented using AWS Identity and Access Management (IAM), which enforces role-based access control across services. Each component is assigned only to the permissions it requires, following the principle of least privilege. AWS Key Management Service (KMS) is used to encrypt sensitive data at rest, ensuring that stored information remains protected. Additionally, AWS Secrets Manager is used to securely store and manage sensitive credentials such as API keys, eliminating the need for hardcoded secrets in the application.

DevOps & Infrastructure Management (CI/CD + Terraform)
The SOS platform incorporates modern DevOps practices using Terraform and GitHub Actions. Infrastructure is defined and managed using Terraform, enabling consistent and repeatable deployment of cloud resources. The CI/CD pipeline is implemented using GitHub Actions, which uses OpenID Connect (OIDC) to securely assume AWS IAM roles without storing access credentials. Terraform state is stored in Amazon S3, while DynamoDB is used for state locking to prevent conflicts during deployment. This setup ensures automated, secure, and efficient infrastructure management, aligning with industry best practices for Infrastructure as Code (IaC).

Workflow of SOS Web Application

The SOS platform follows a complete end-to-end workflow that simulates a real-world e-commerce system. The system supports two main operational flows: the customer workflow, which covers the user journey from accessing the website to completing a purchase, and the admin workflow, which handles product and user management operations.

Customer Workflow (End-User Journey)

User Access and Website Loading

When a user enters the website URL, the request is first routed to Amazon CloudFront. CloudFront serves the frontend content from edge locations, ensuring fast and reliable access. The static files, including HTML, CSS, and JavaScript, are stored in a private Amazon S3 bucket and are accessed securely using Origin Access Control (OAC). This prevents direct access to the storage layer and ensures that all content delivery is handled through CloudFront.

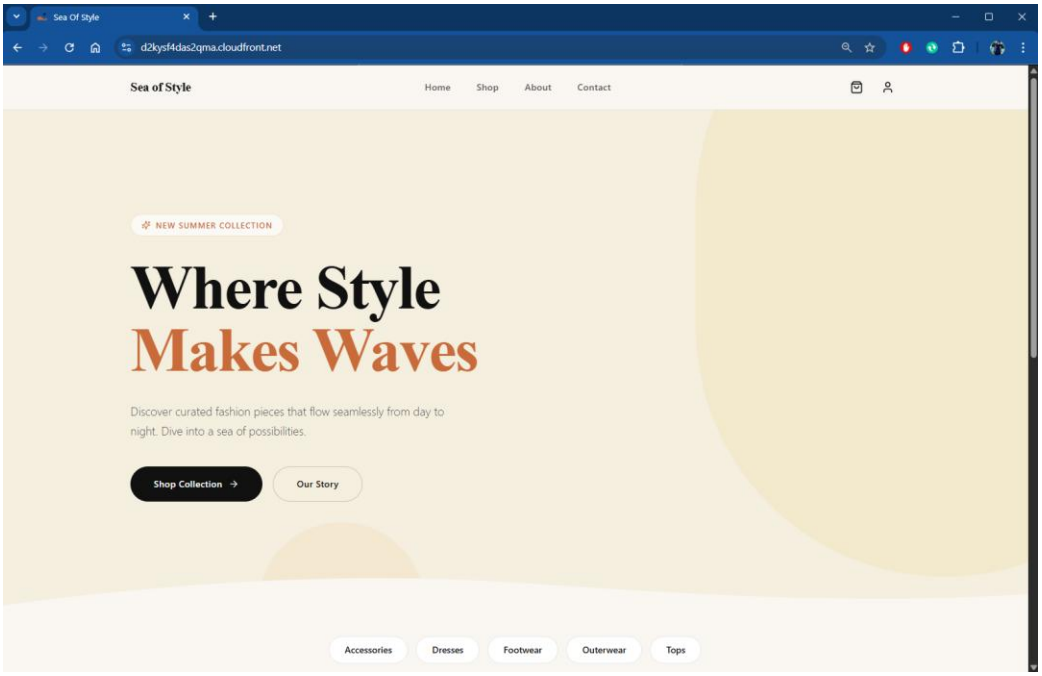


Figure 2: Home Page

Product Browsing

Once the website is loaded, users can browse available products. When product data is required, the frontend sends a request to Amazon API Gateway. The request is forwarded to AWS Lambda functions, which retrieve product information from Amazon DynamoDB and return it to the frontend.

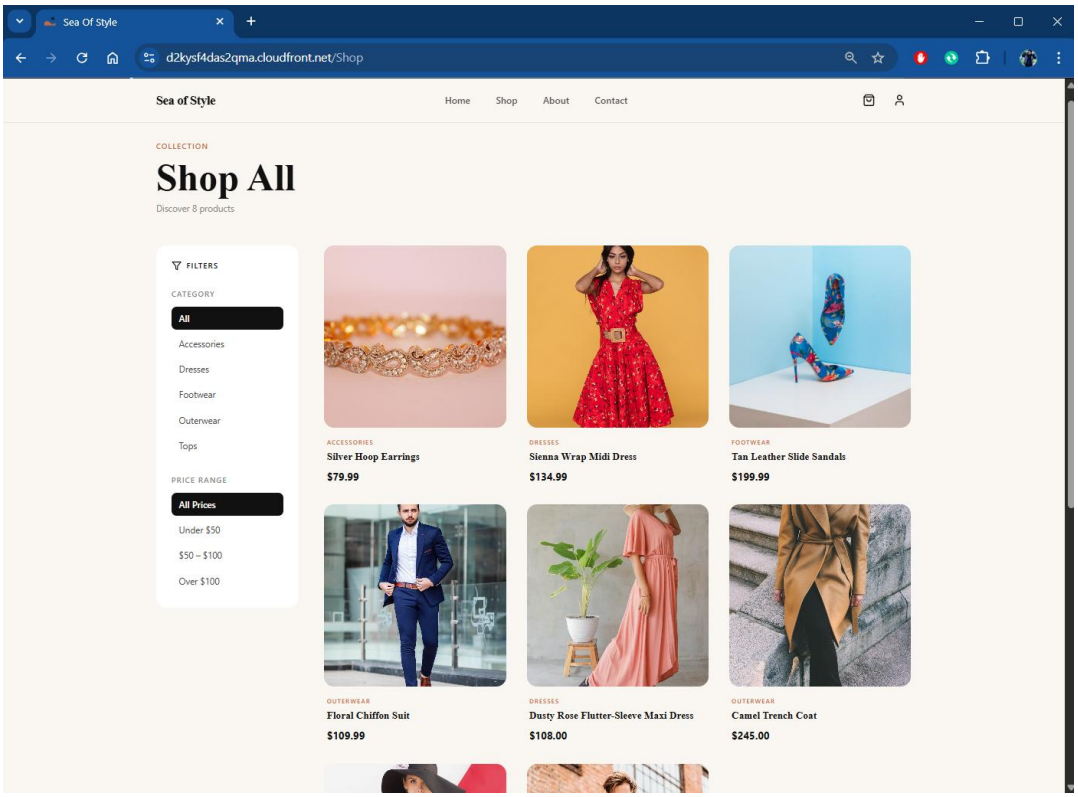


Figure 3: Shop Page

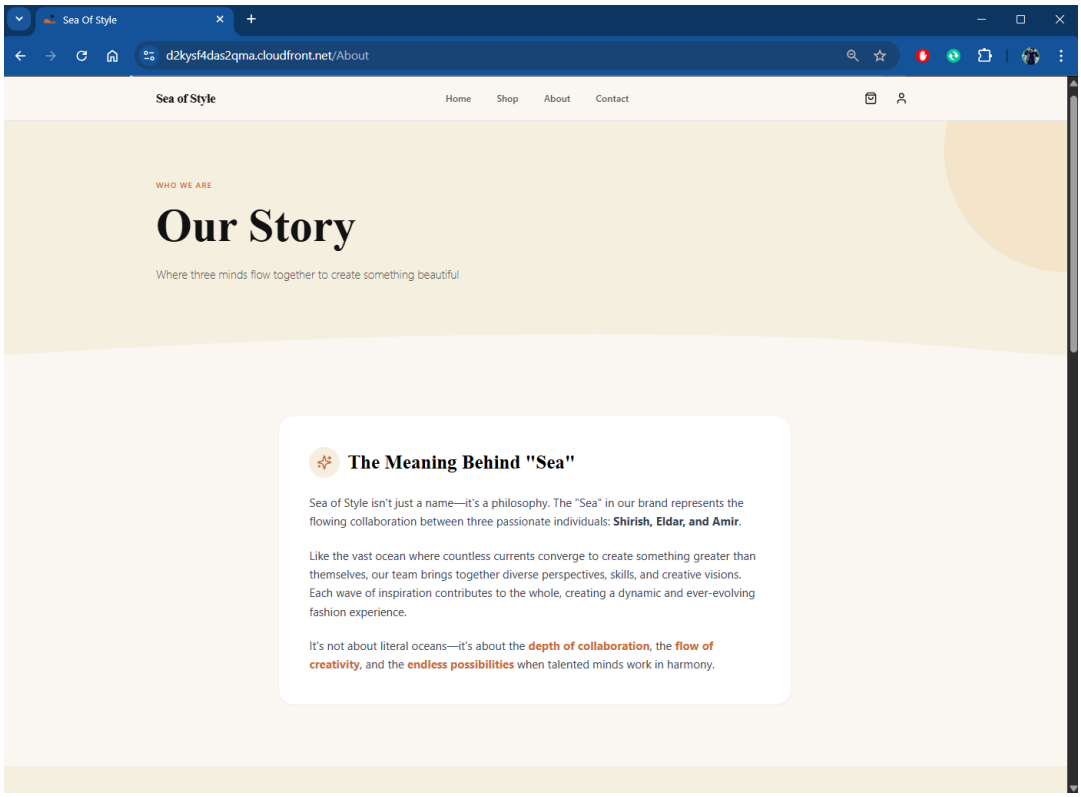


Figure 4: About Page

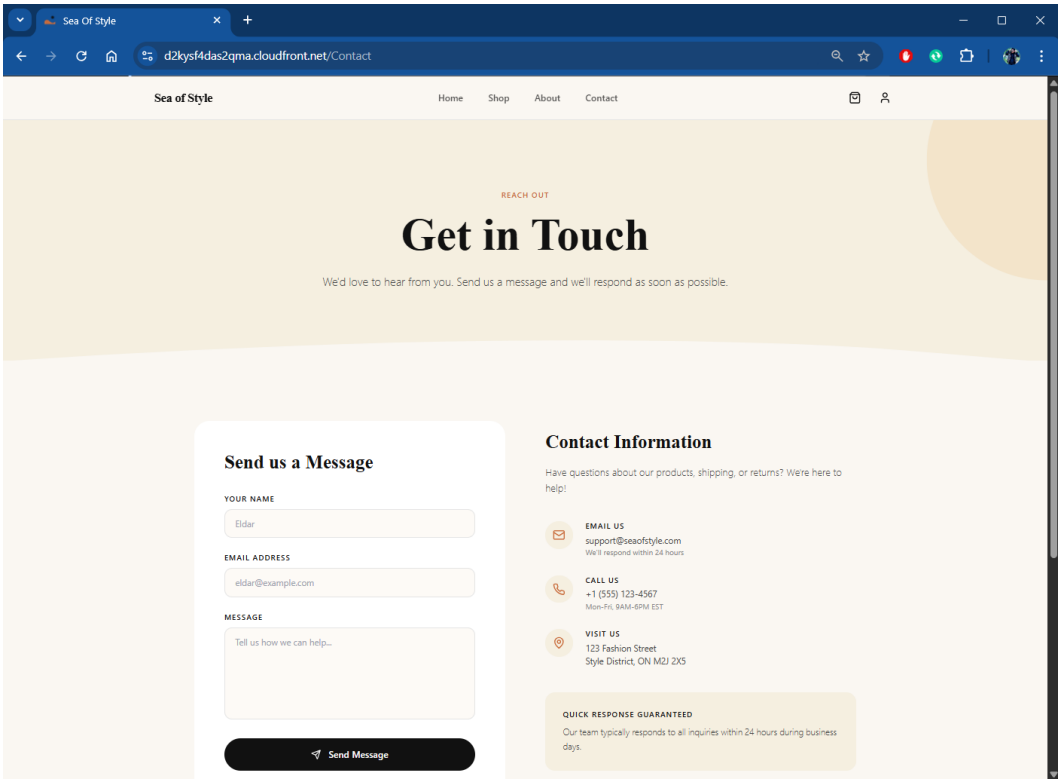


Figure 5: Contact Page

User Registration and Authentication

To access protected features such as placing an order, users must create an account or log in. Amazon Cognito manages the authentication process, including user registration, login, and verification. After successful authentication, a JSON Web Token (JWT) is generated and returned to the frontend. This token is included in subsequent API requests and is validated by API Gateway to ensure secure access to backend services.

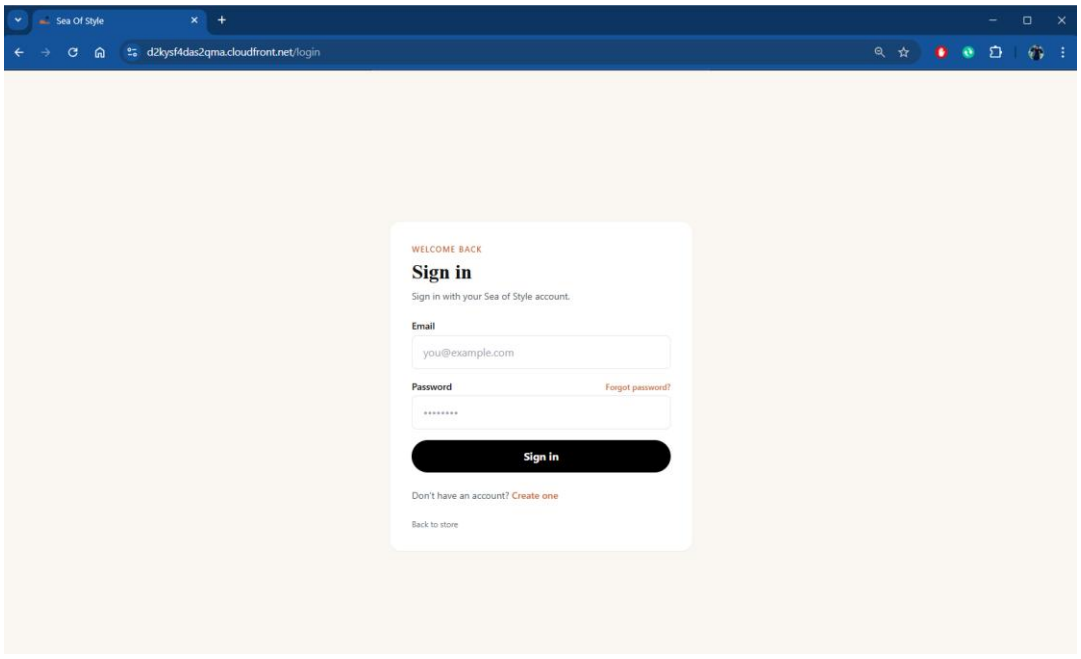


Figure 6: Login Page

Create an Account

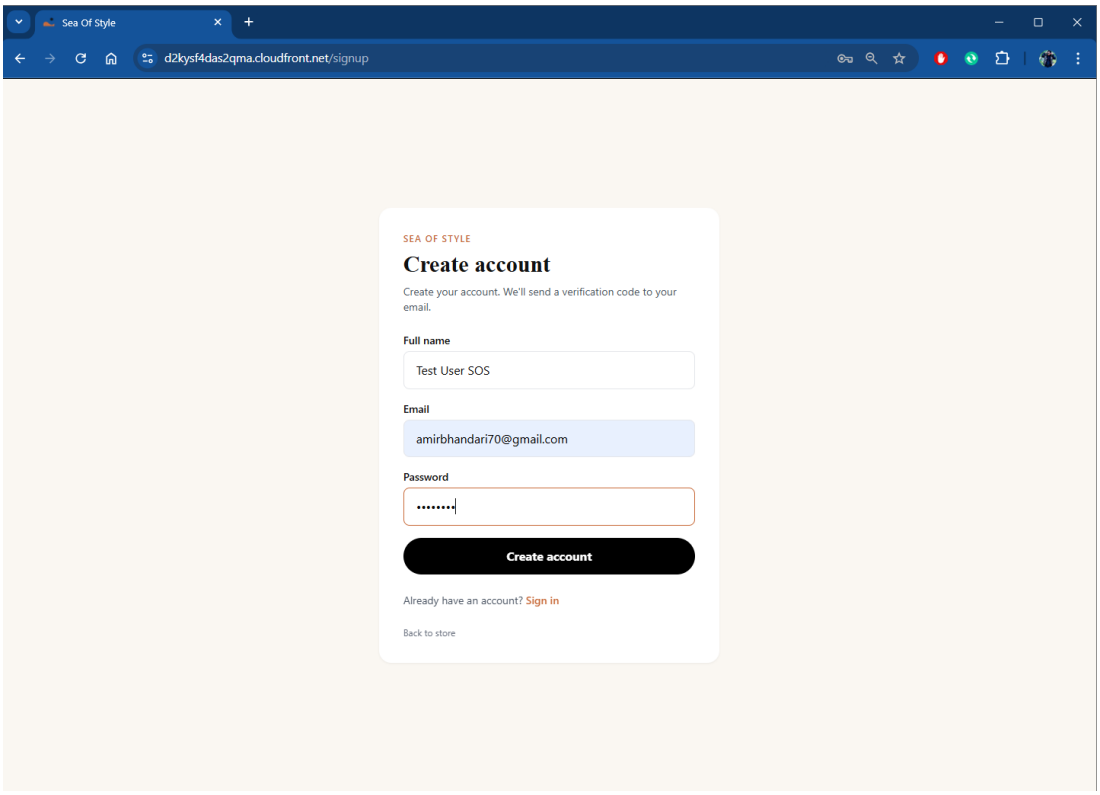


Figure 7: Sign up Page

Verification Code

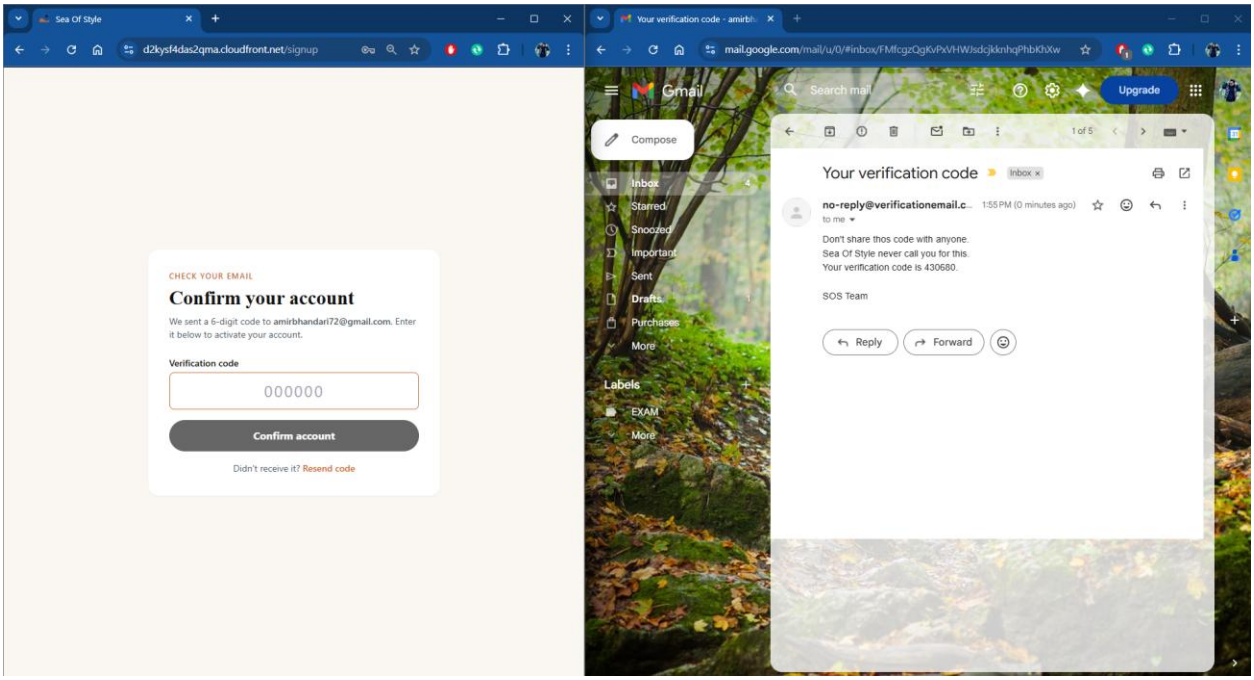


Figure 8: Verification Code

Account Confirmed

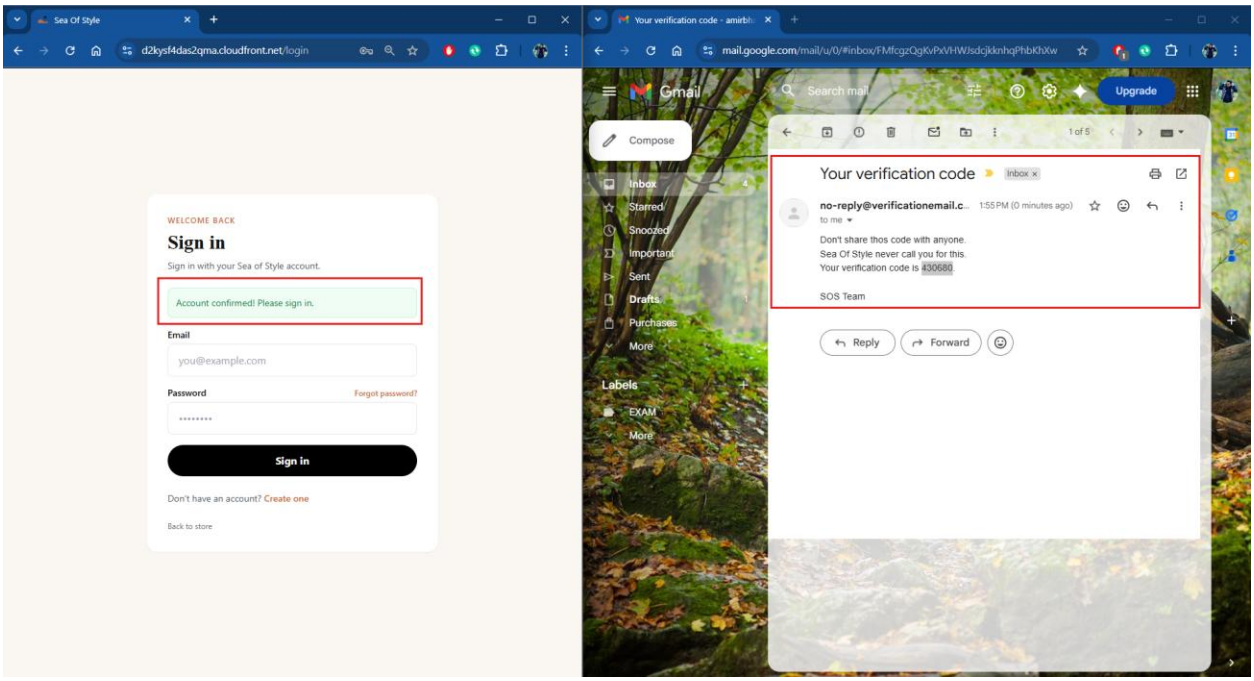


Figure 9: Account Confirmed

User Profile

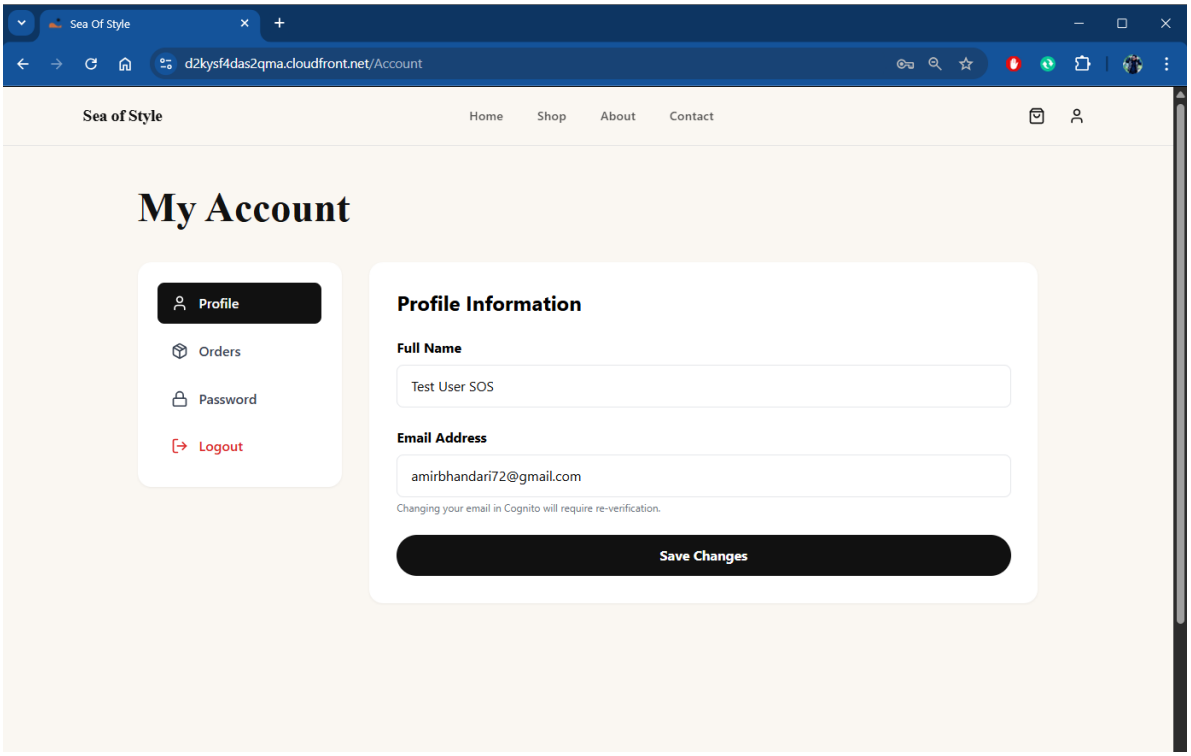


Figure 10: User Profile Dashboard

User Actions and Cart

Authenticated users can perform actions such as adding items to the cart, viewing their profile, and managing account settings. Each request includes the JWT token, which is validated before being processed by Lambda functions. These functions handle business logic such as updating cart data and retrieving user-specific information from DynamoDB.

Order History

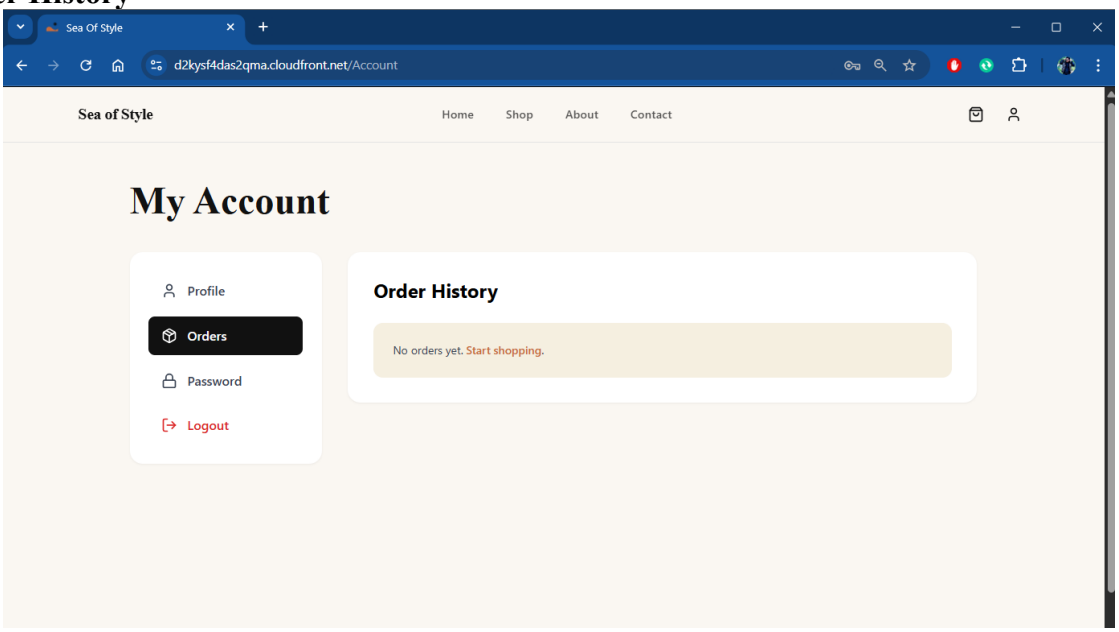


Figure 11: Order History

Change Password

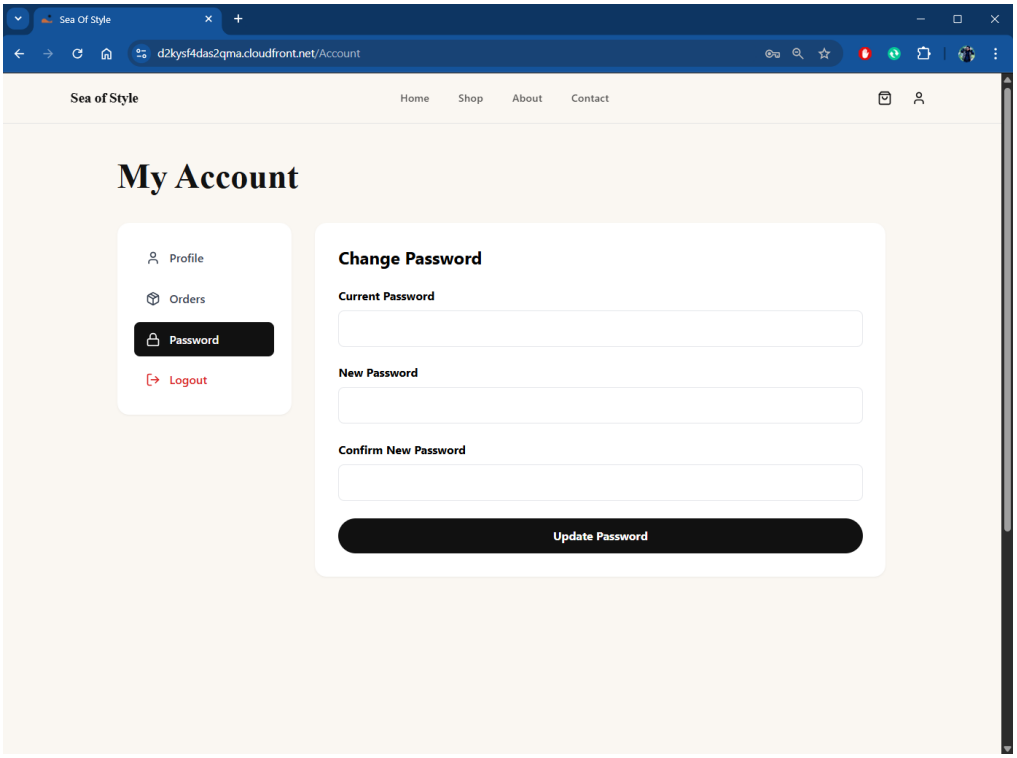


Figure 12: Change Password

Product Details from user side

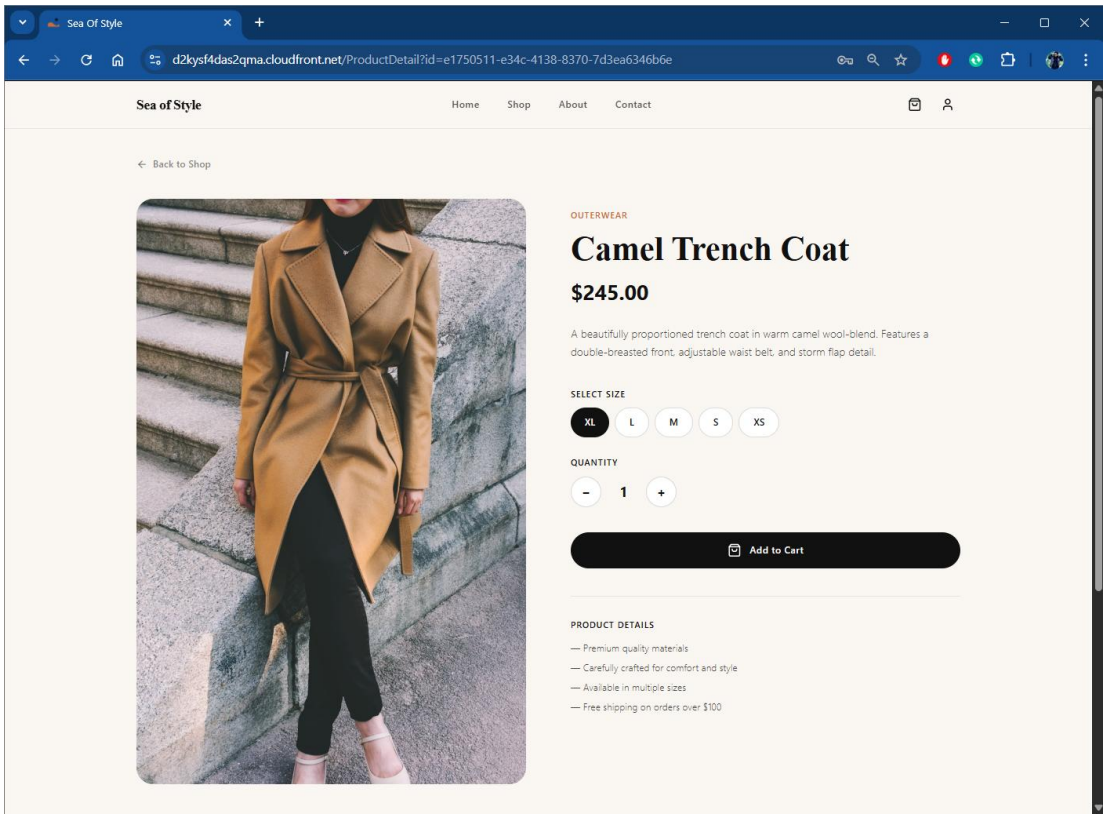


Figure 13: Product Information

Add to Cart

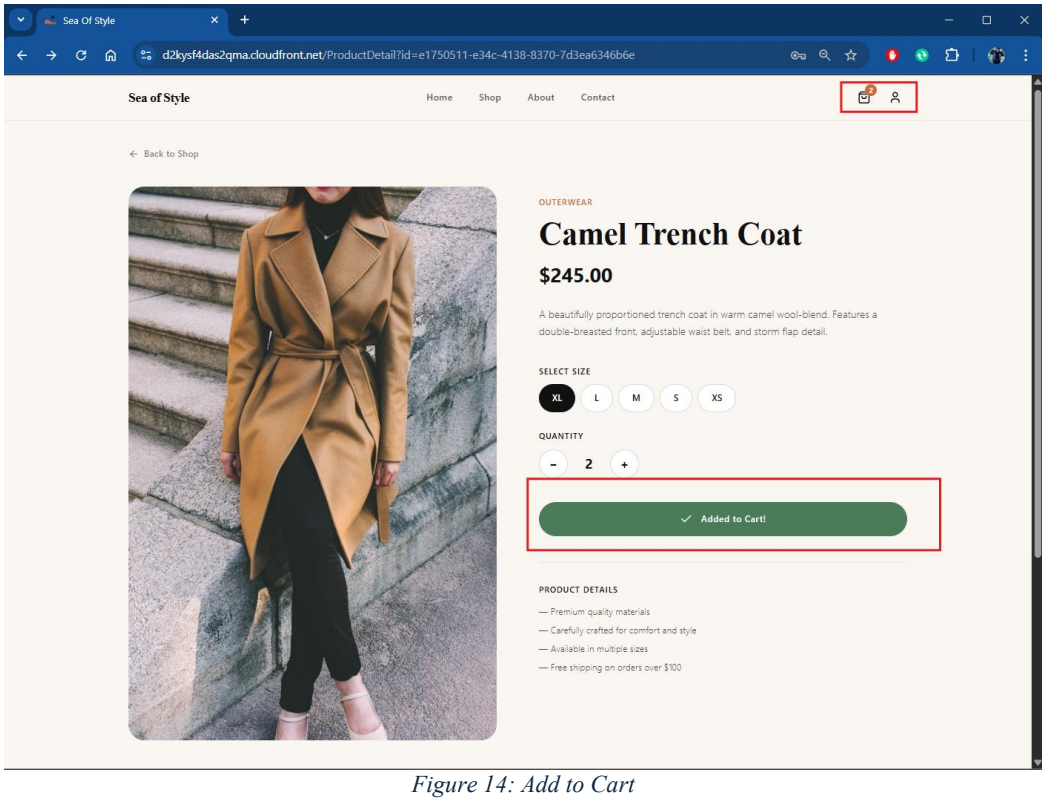


Figure 14: Add to Cart

Shopping Cart Page

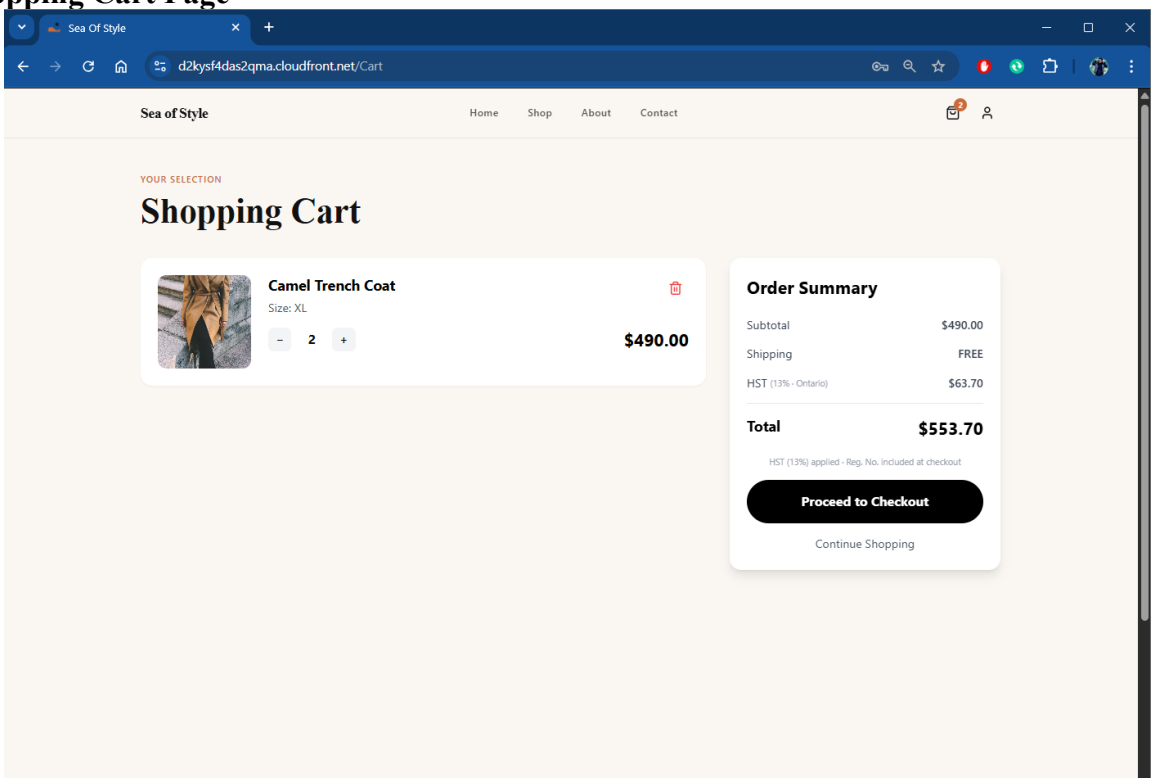


Figure 15: Shopping Cart

Finalizing the order with details

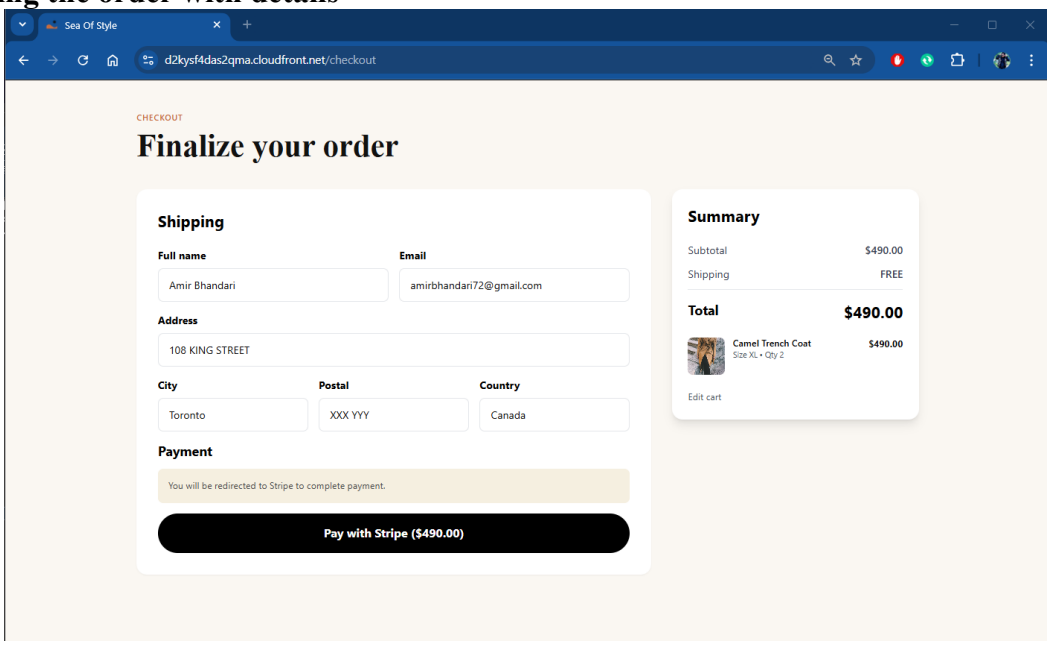


Figure 16: Shipping Page

Checkout and Payment Processing (Stripe Integration)

When the user proceeds to checkout, the application integrates with Stripe to handle payment processing. The user is redirected to Stripe's secure payment page, ensuring that sensitive financial information is not handled or stored within the SOS system. After the payment is completed, Stripe generates a webhook event and sends it back to the application.

Payment with STRIPE

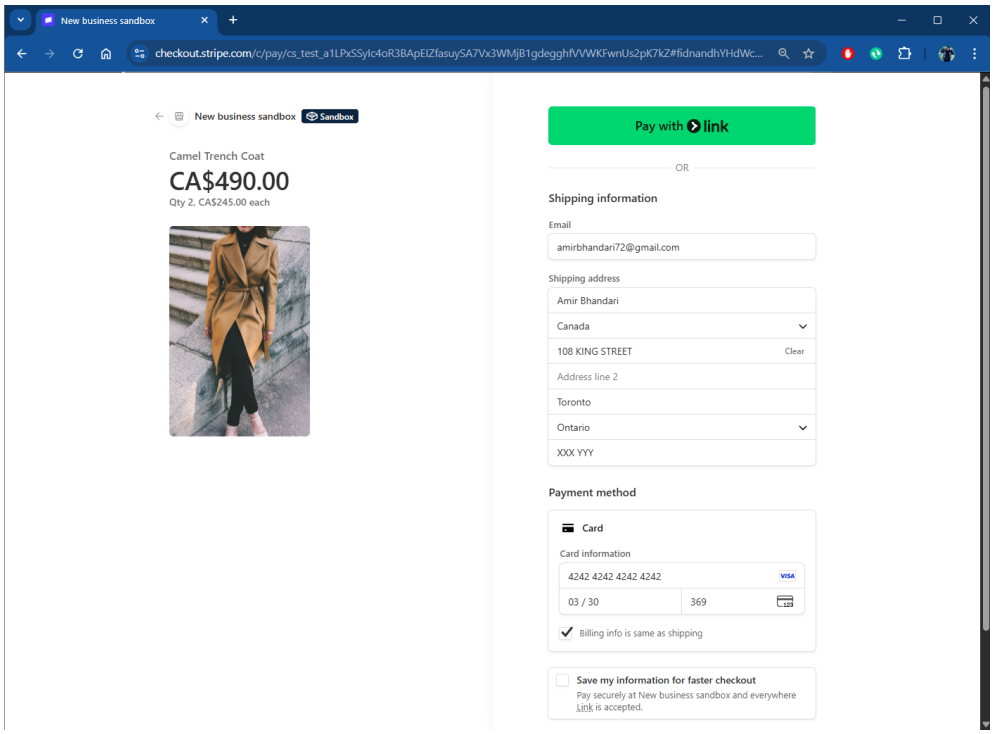


Figure 17: Payment with Stripe

Payment Successful

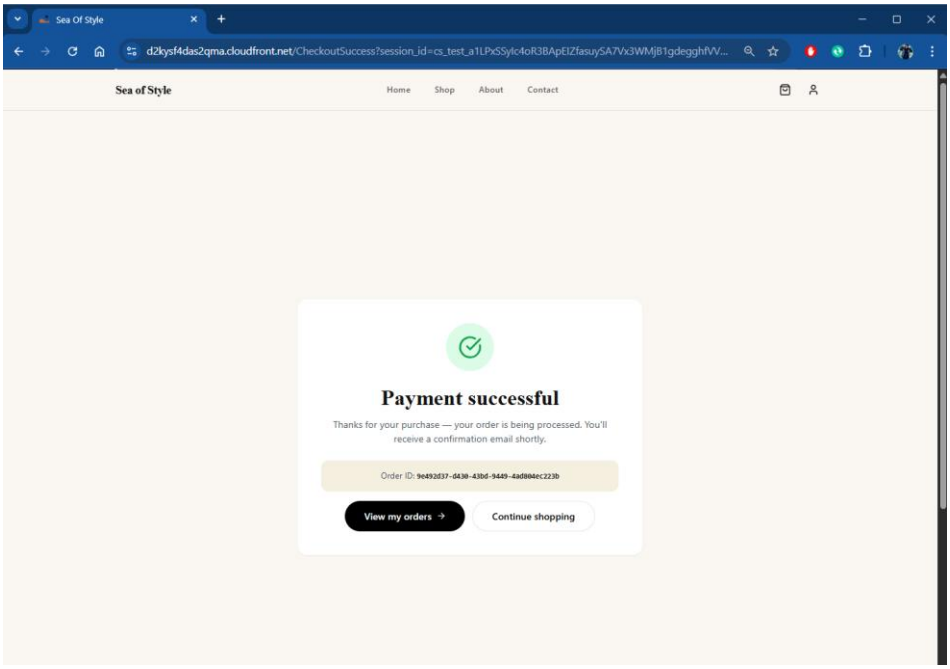


Figure 18: Payment Successful

Order Processing via Webhook

The Stripe webhook is received through Amazon API Gateway, which triggers a Lambda function. This function verifies the payment status and processes the order. Once confirmed, order details such as user information, purchased products, and transaction status are stored in DynamoDB. This ensures that only successful transactions are recorded in the system.

Order History Details

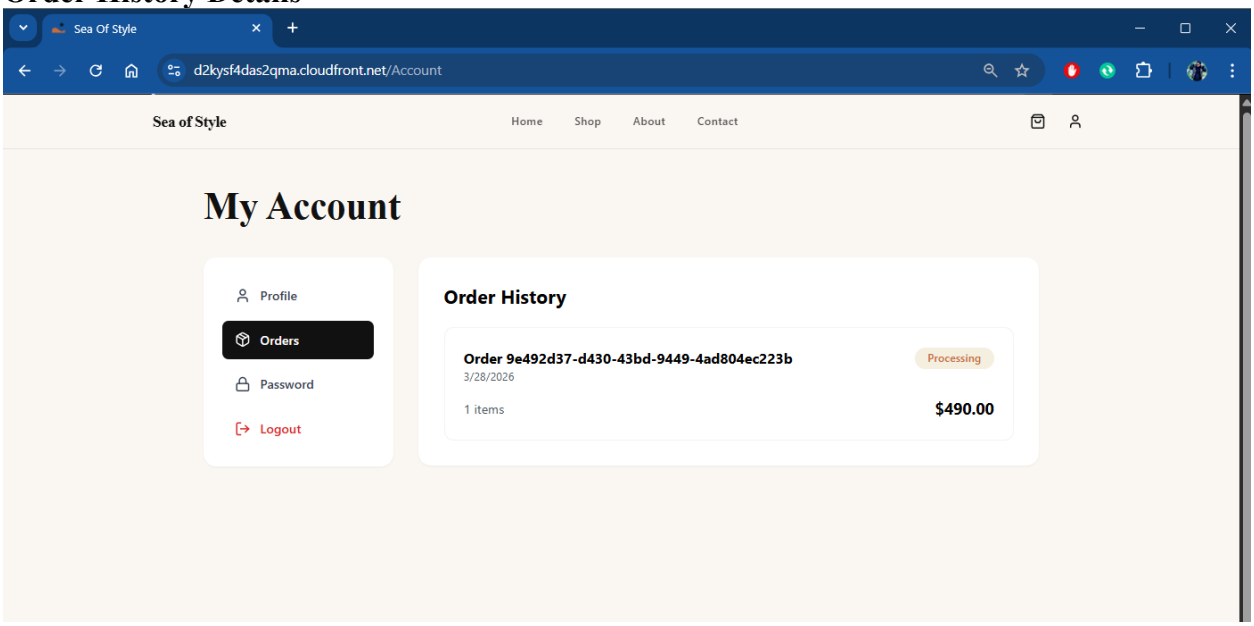


Figure 19: Order History

Purchase Notification (Admin Alerting)

After a user successfully completes a purchase, the system triggers a real-time notification workflow to inform administrators. Instead of using email-based notifications, the SOS platform sends alerts to a Telegram group for faster and more direct communication. Once the payment is completed through Stripe, a webhook event is sent to Amazon API Gateway, which triggers an AWS Lambda function to process the order details. The event is then published via Amazon SNS and forwarded to a Telegram bot using webhook integration, delivering the notification to the admin group. The message includes key information such as order ID, payment status, total amount, and customer details, allowing administrators to monitor transactions instantly.

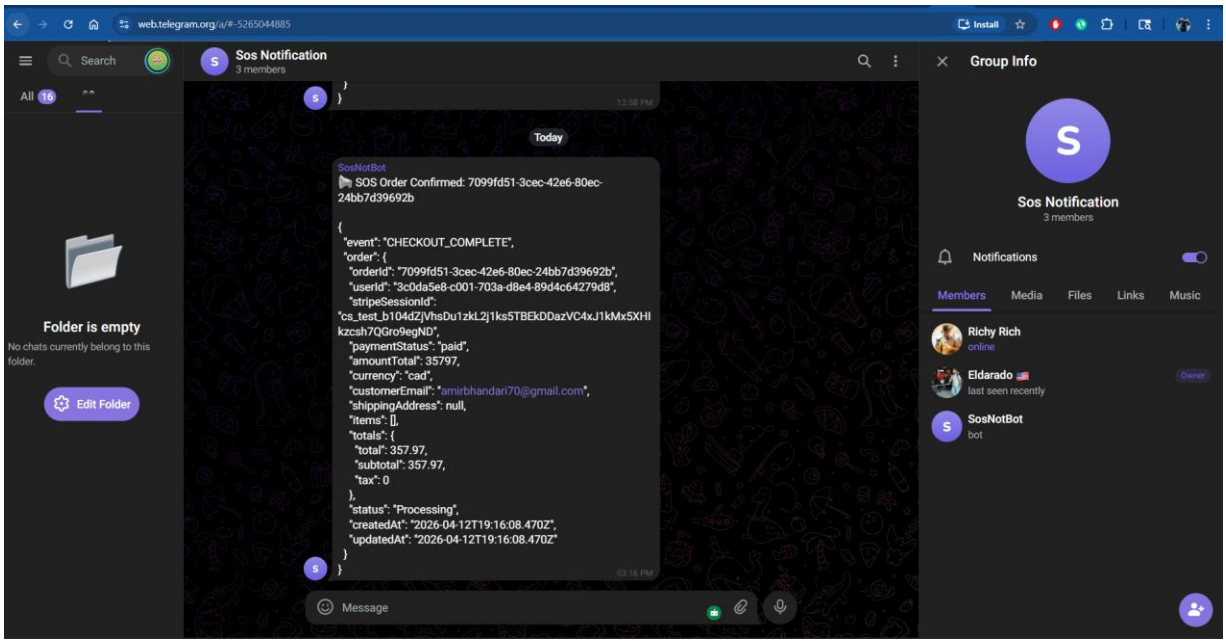


Figure 20: Telegram Notification

Chatbot Interaction (Amazon Lex)

In addition to traditional navigation, users can interact with the system using a chatbot powered by Amazon Lex. User queries are processed through Lex and forwarded to AWS Lambda functions, which generate appropriate responses. This provides an alternative and interactive way for users to access information and perform actions within the application.

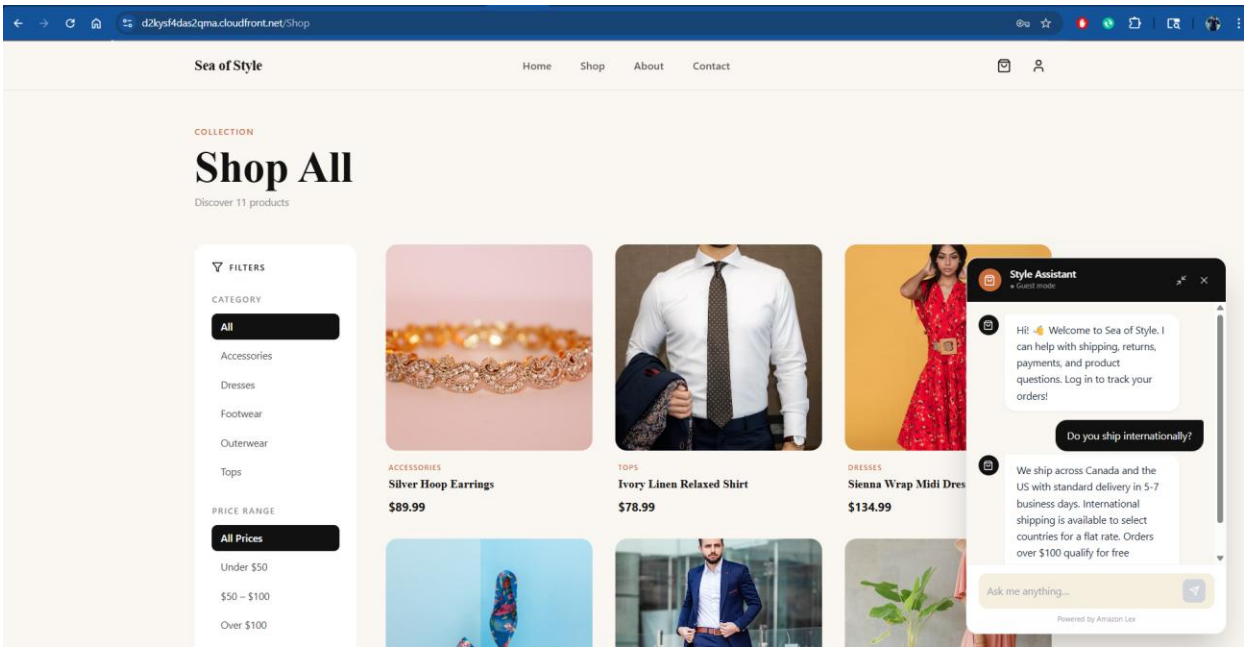


Figure 21: Amazon Lex - User Interaction

Logging, Monitoring and Auditing

Throughout the entire workflow, system activities are continuously monitored. Amazon CloudWatch collects logs and performance metrics from services such as Lambda and API Gateway, enabling real-time monitoring and troubleshooting. Additionally, AWS CloudTrail records all control plane operations, providing an audit trail for security and compliance purposes.

User's Password Recovery

If a user forgets their password, the system provides a secure password recovery mechanism through Amazon Cognito. From the login page, the user can select the "Forgot Password" option, which initiates the password reset process. The user is prompted to enter their registered email address, after which Cognito sends a verification code via email. The user then enters the received code along with a new password that meets the defined password policy. Once verified, the password is successfully updated, and the user can log in using the new credentials.

This process ensures secure account recovery without exposing sensitive information and maintains the integrity of user authentication.

Forget Password

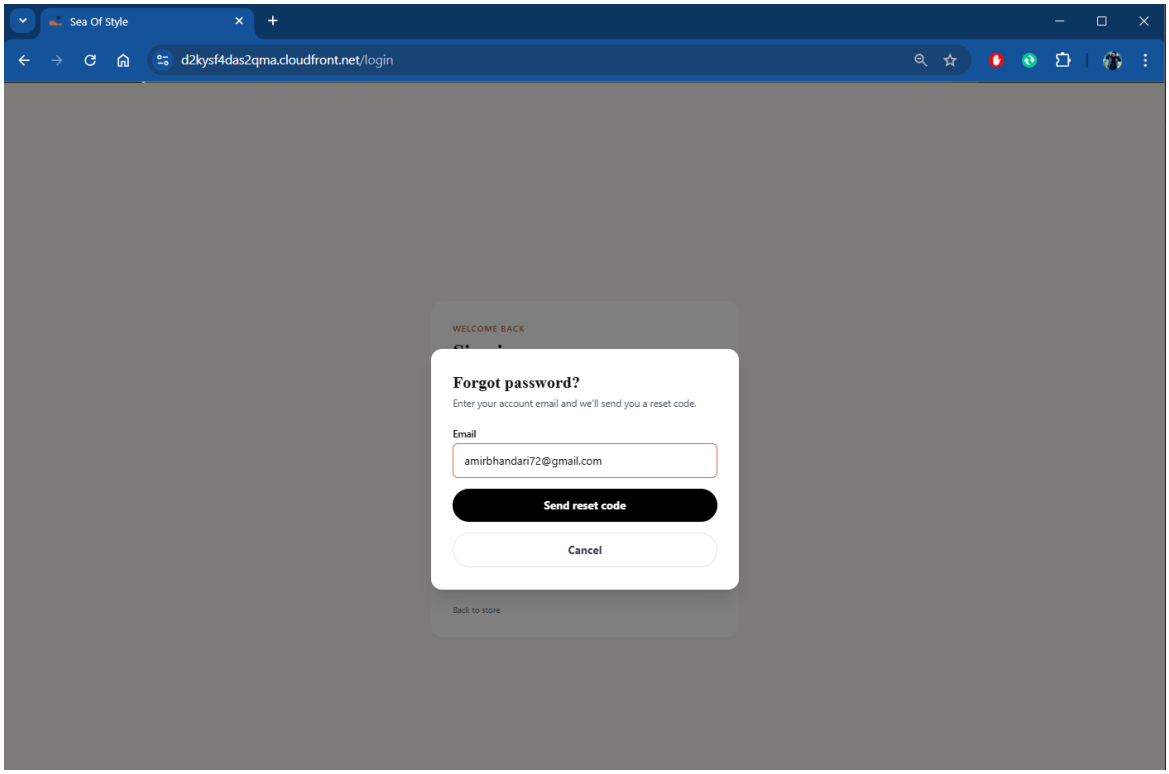


Figure 22: Forget Password

Received an email while forgetting the password

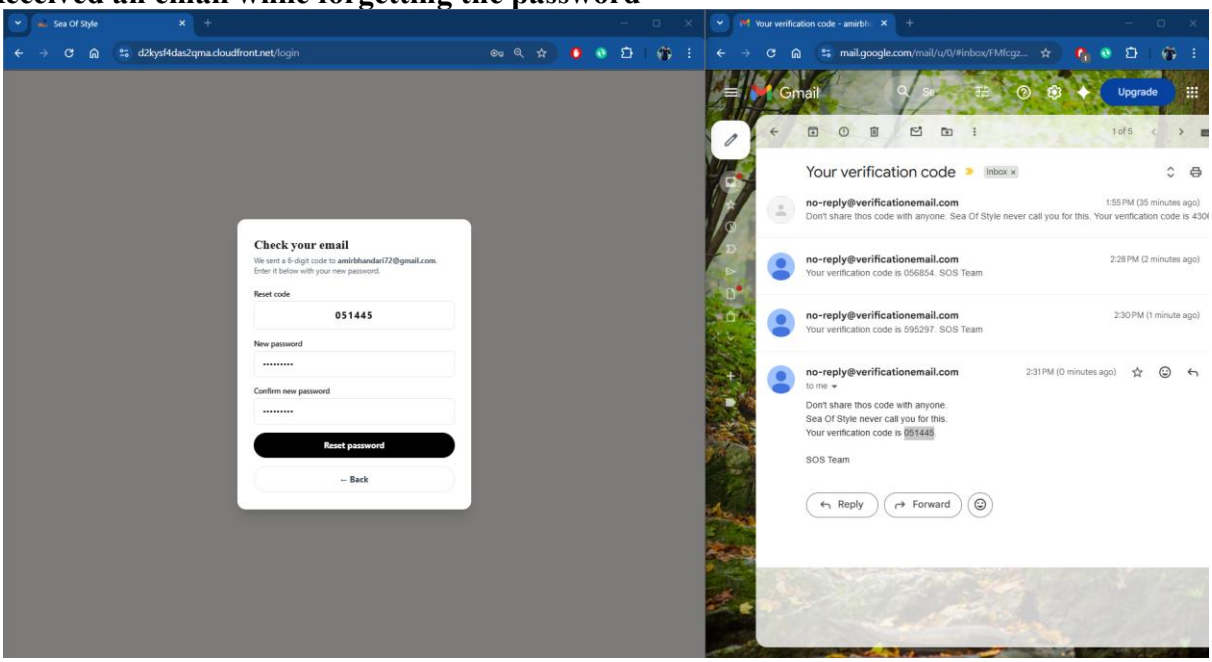


Figure 23: Verification Code for forgetting the password

Password Reset

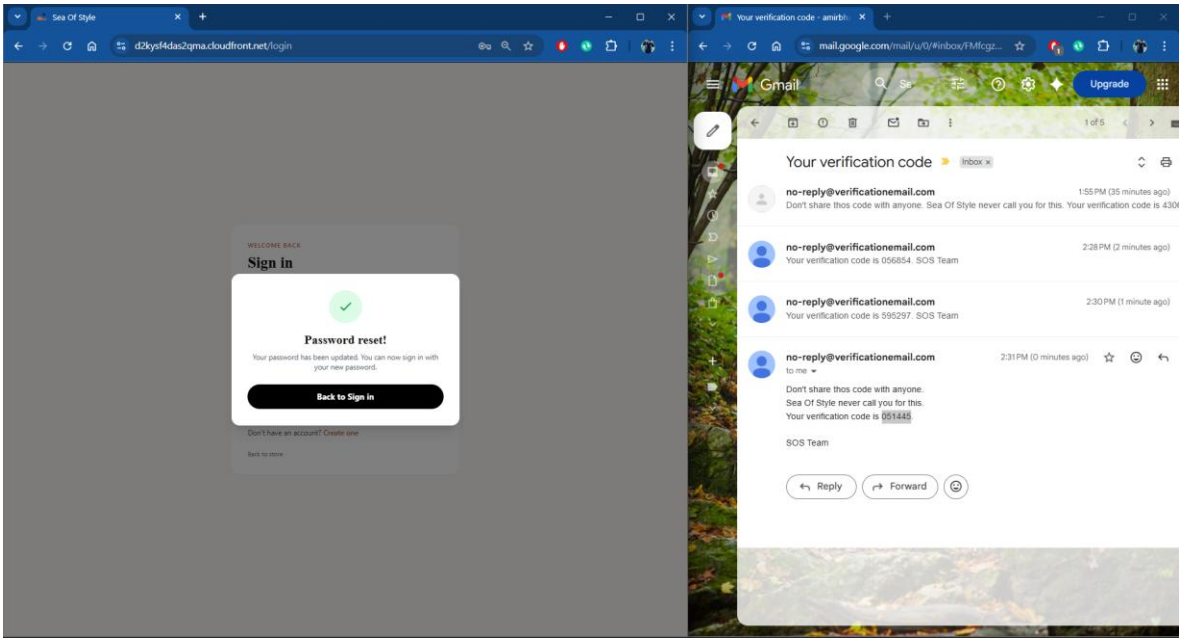


Figure 24: Password Reset

Tried with previous password

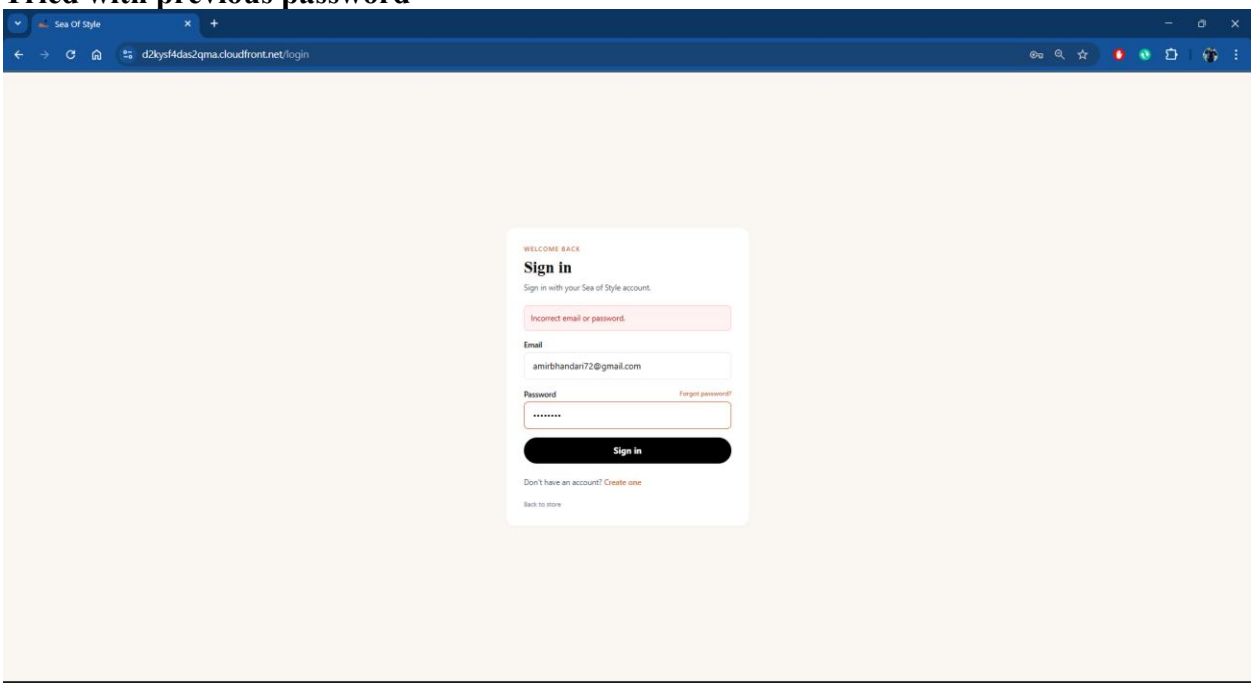


Figure 25: Invalid Username or Password

Password Policy should match

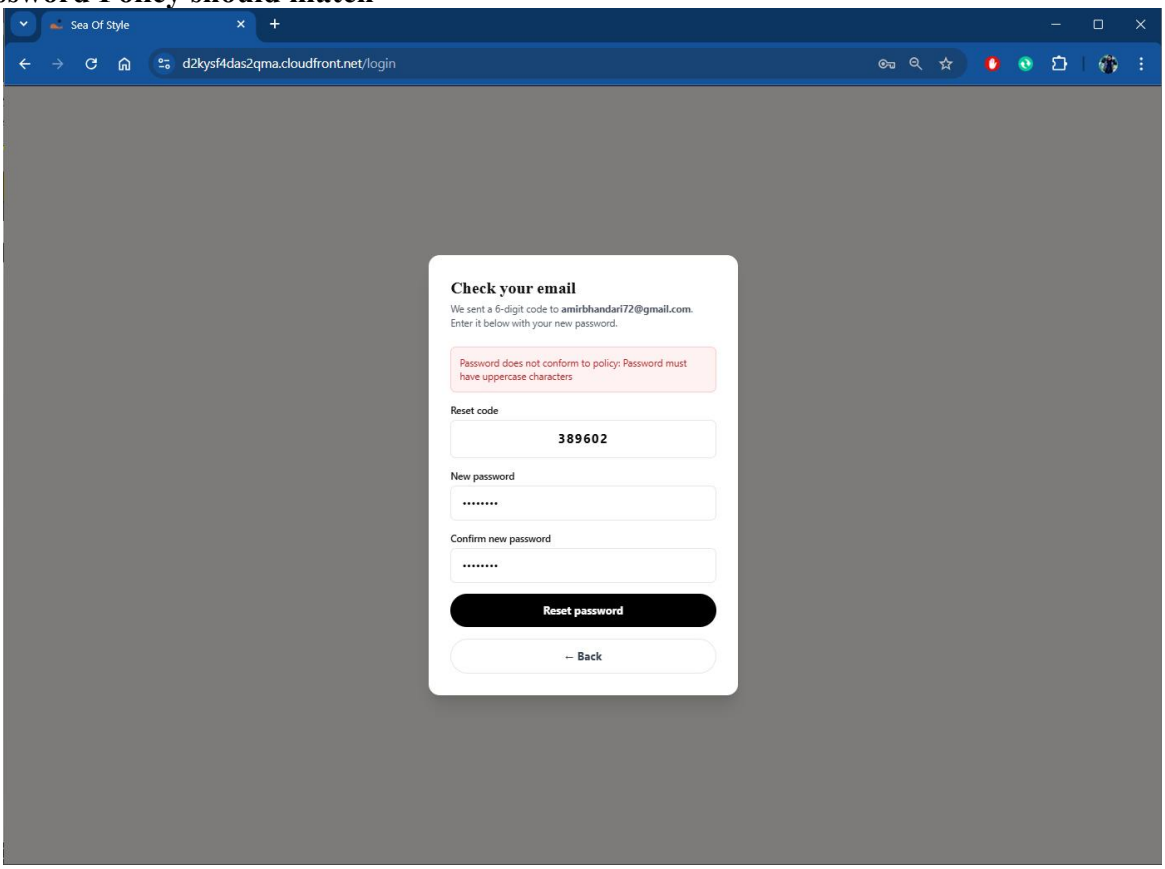


Figure 26: Password Policy Unmatched

Admin Workflow (Management Operations)

Admin Access and Authentication

Administrators log into the system using Amazon Cognito. Based on assigned roles, admins are granted elevated permissions to access management features.

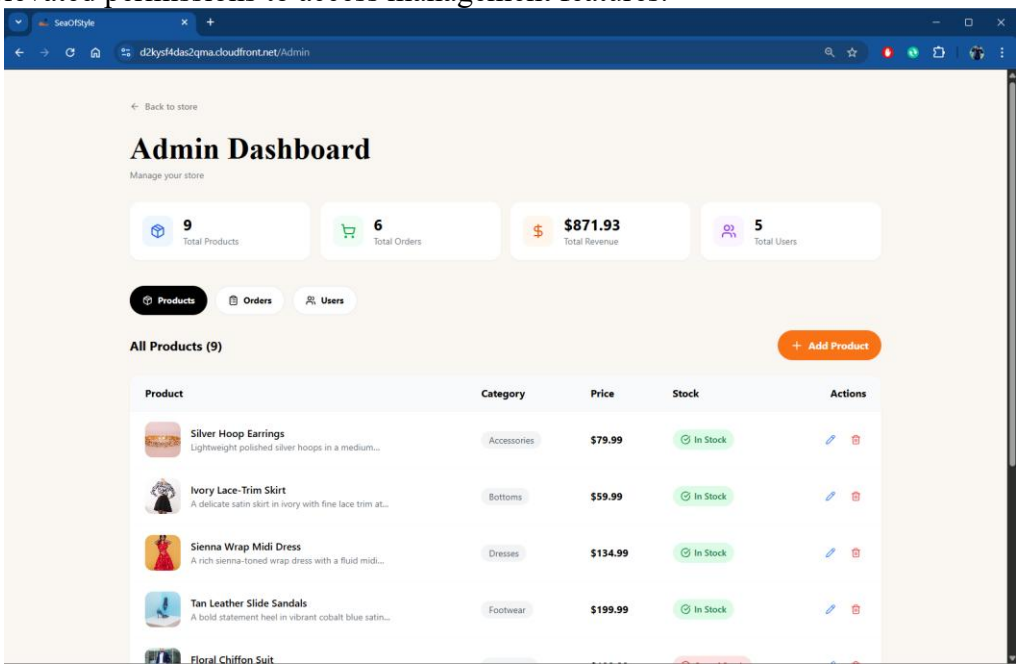


Figure 27: Admin Dashboard

Product Management (Add, Edit, Delete)

Admins can add new products, update existing product details, or delete products from the system. These operations are performed through API Gateway and Lambda functions, which update the corresponding data in DynamoDB. This allows real-time management of the product catalog

Add Product

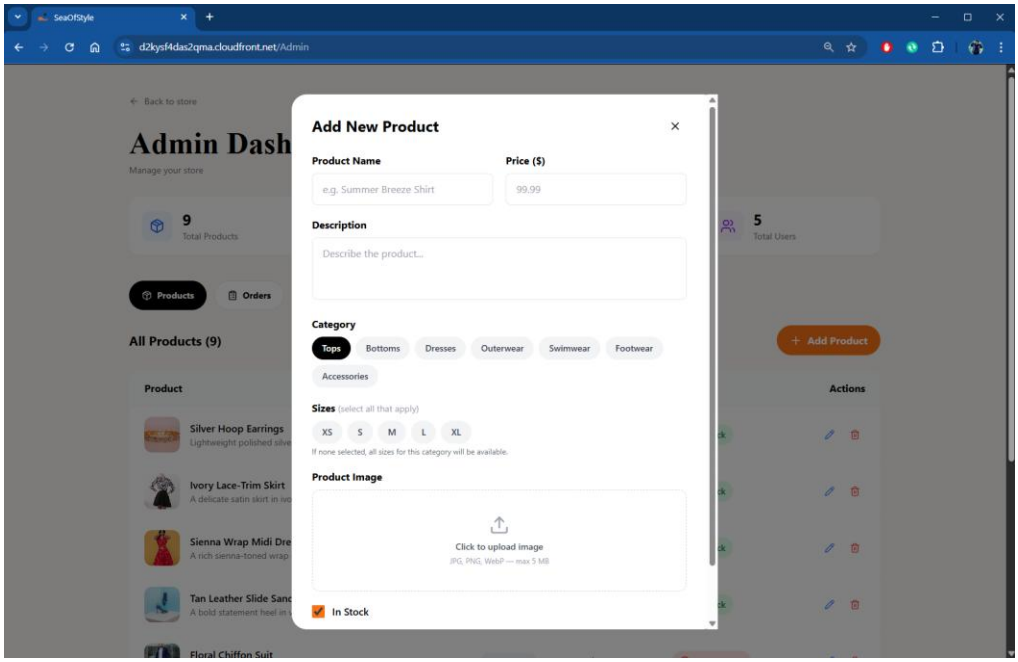


Figure 28: Add Product

Edit Product

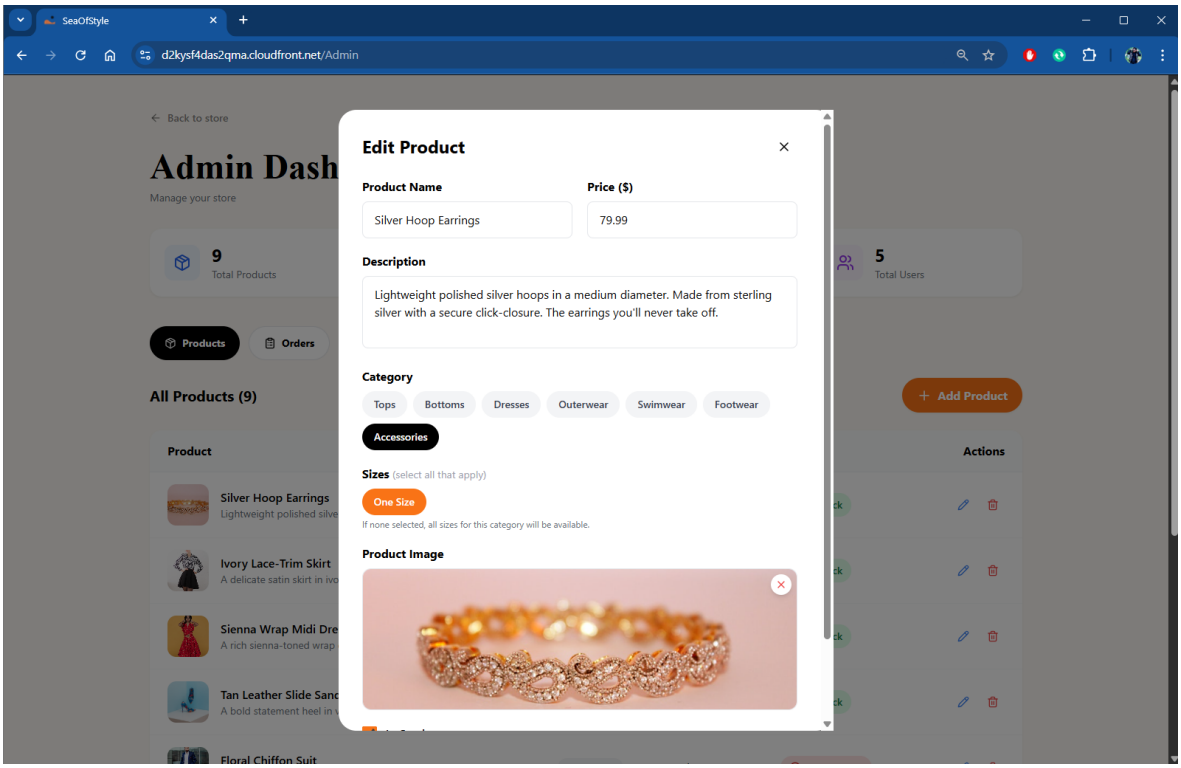


Figure 29: Edit Product

Delete Product

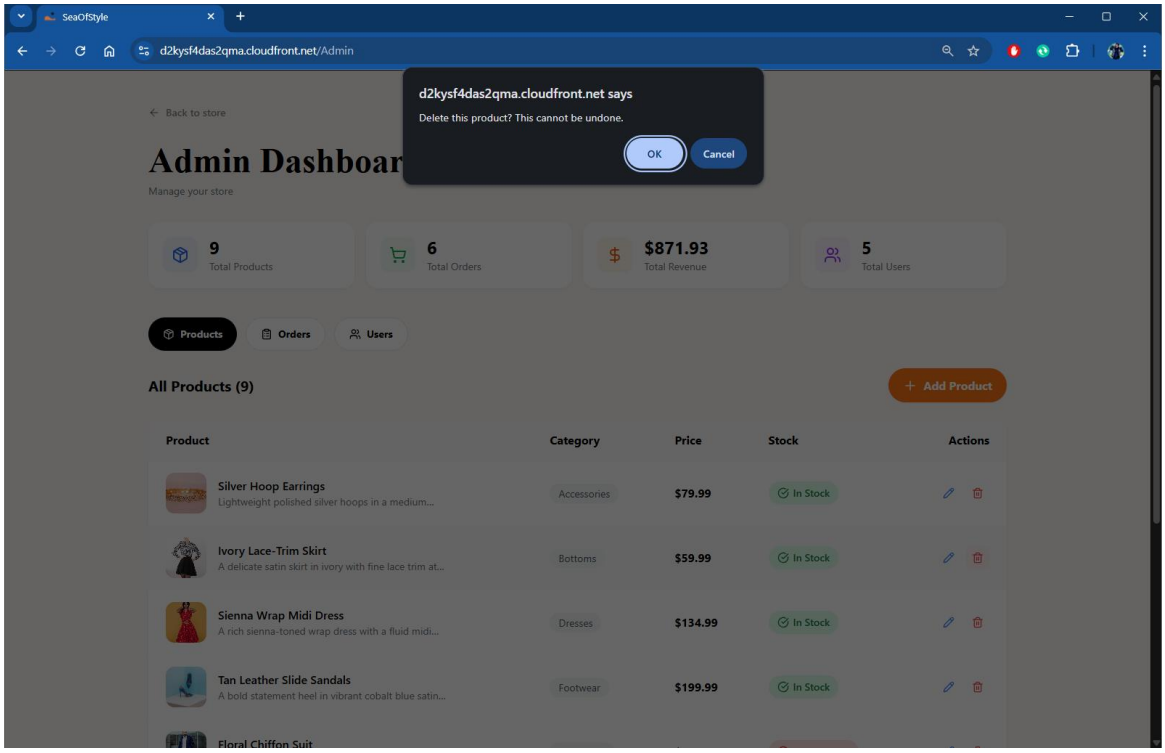


Figure 30: Delete the Product

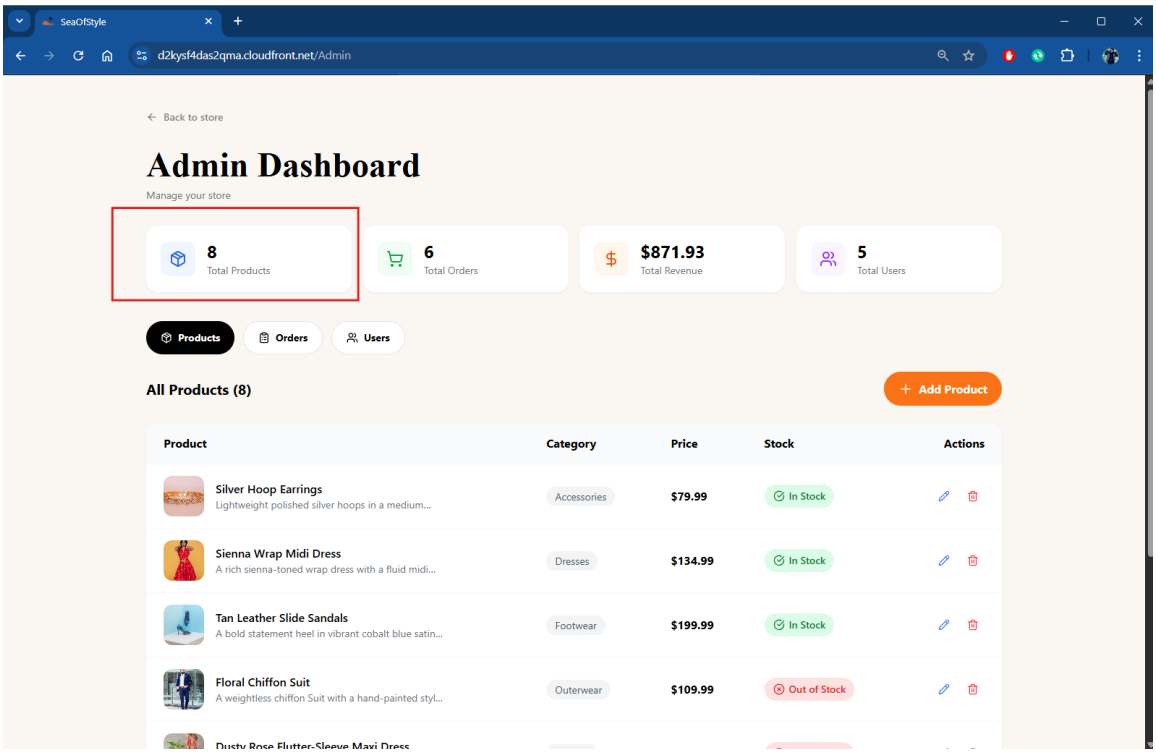


Figure 31: Delete Successful

Figure 29 shows, after deleting, products become from 9 to 8.

User Role Management

Admins can modify user roles, such as promoting a regular user to an admin role. This is handled through secure API calls and controlled using IAM and Cognito-based authorization.

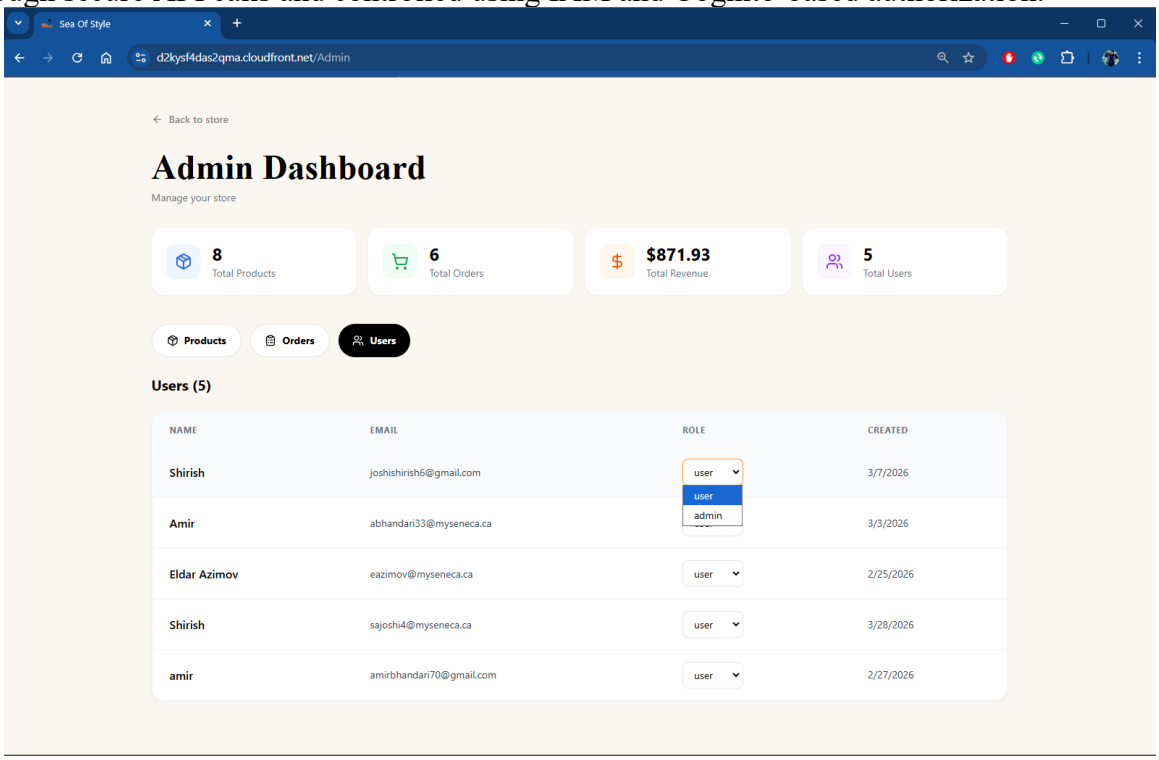


Figure 32: Modify User's Role

Admin User's Page

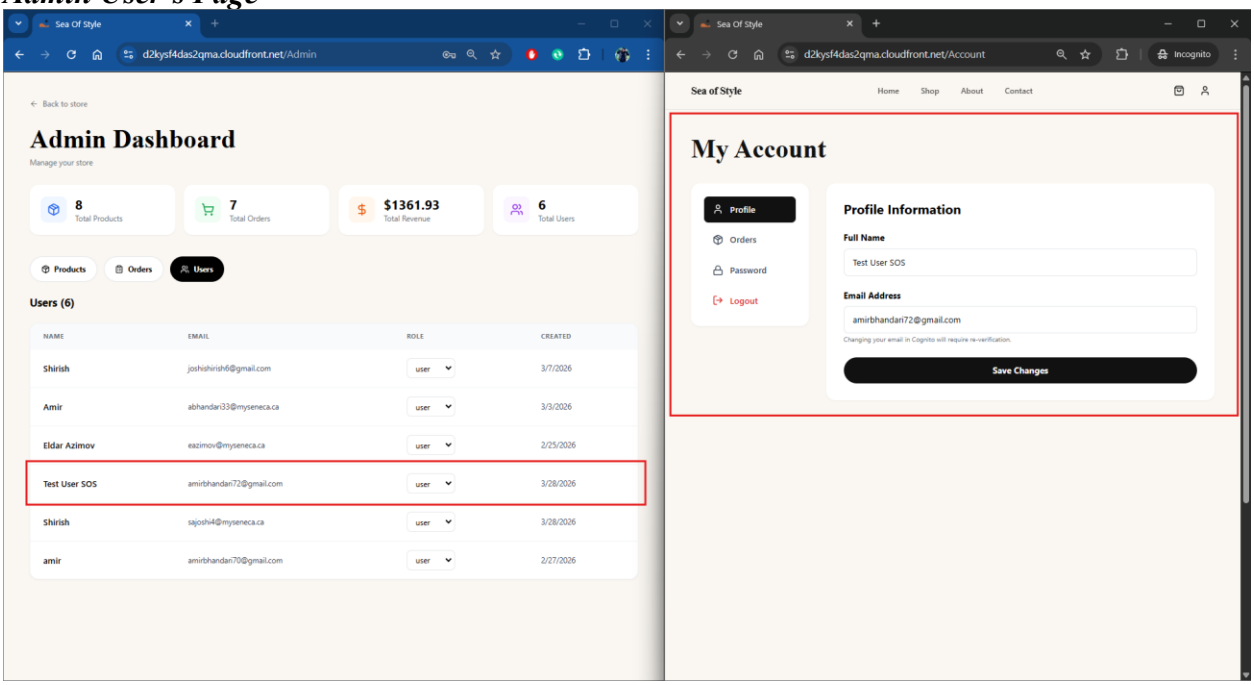


Figure 33: User status on Admin Panel

Order Status Management

Admins can update order statuses, such as processing, confirmed, shipped, Delivered, Cancelled or Refunded orders. These updates are processed through Lambda functions and stored in DynamoDB, ensuring consistency across both admin and user views.

Admin's Order Page

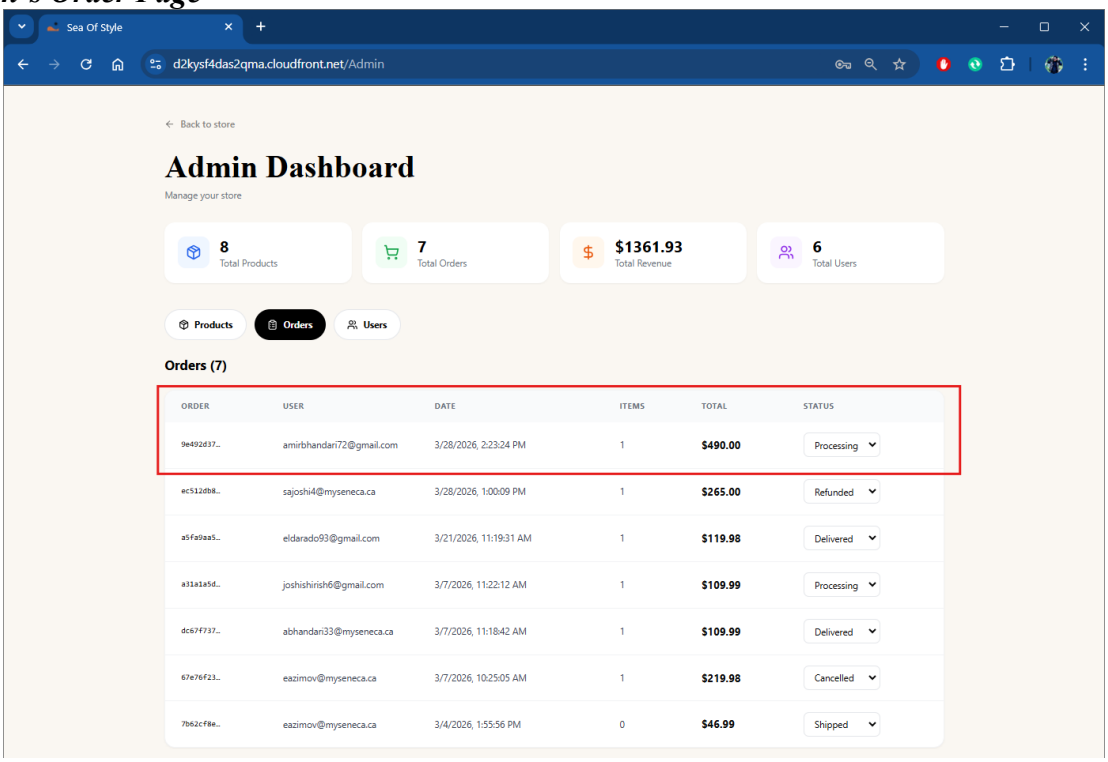


Figure 34: Admin's Order Page

Order Status

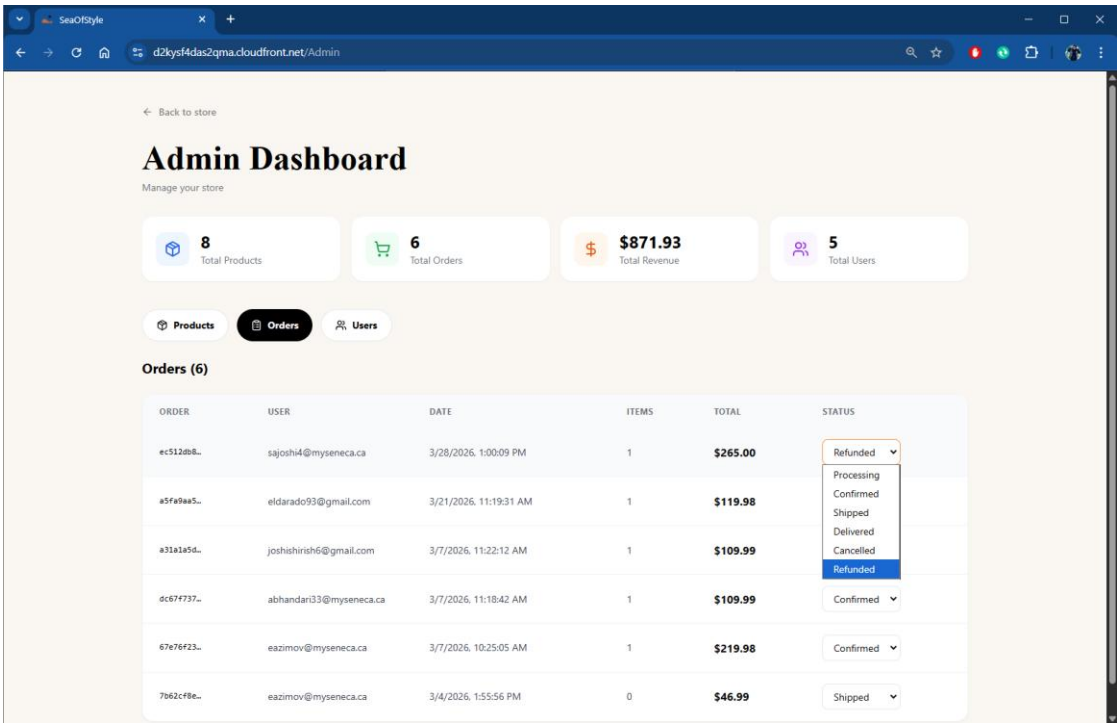


Figure 35: Order Status

Order Confirmed

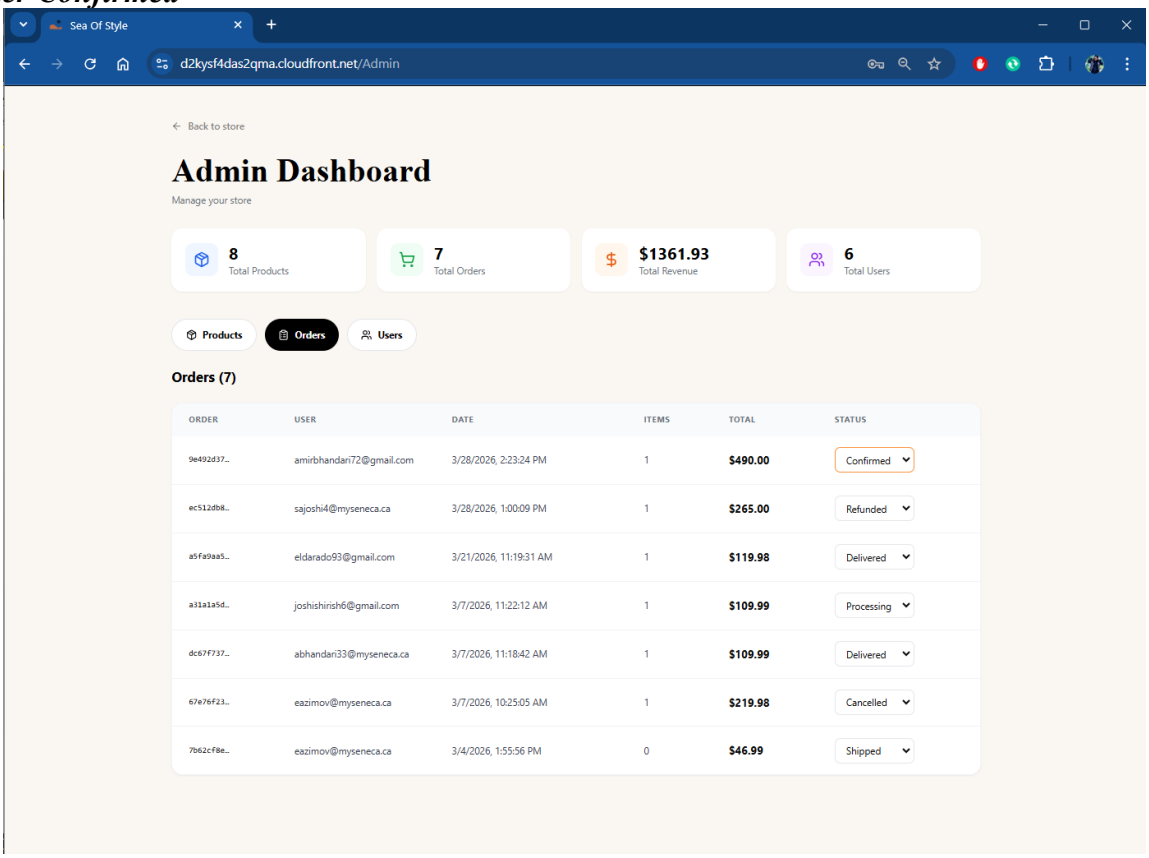


Figure 36: Order Confirmed

Modify Order status from confirmed to delivered

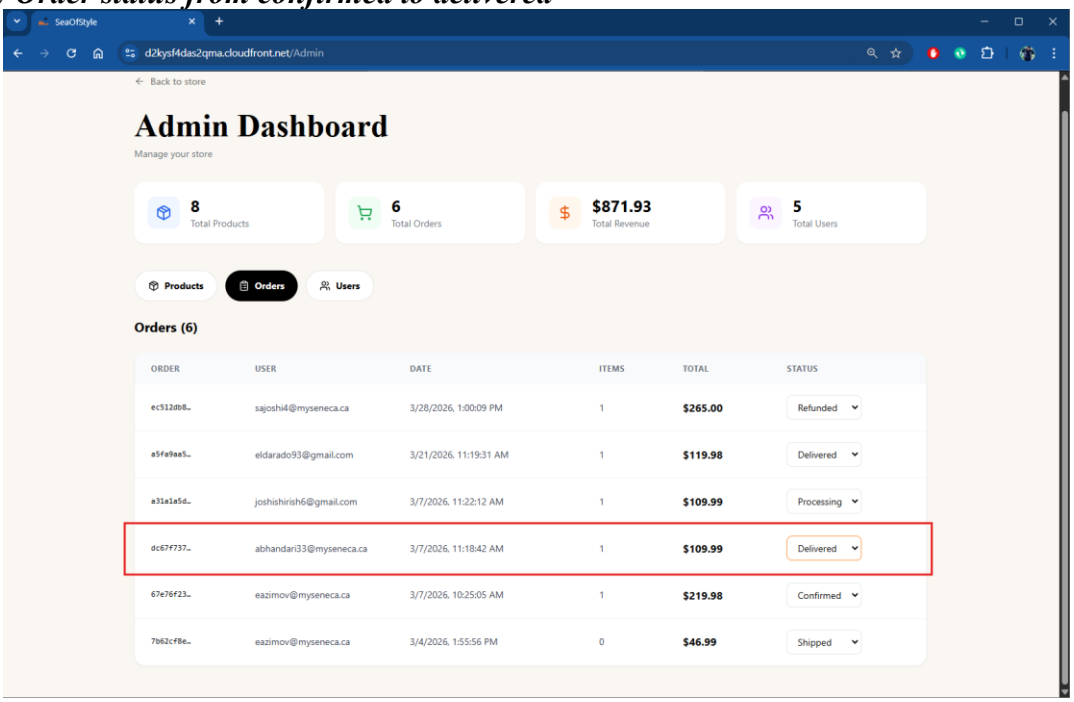


Figure 37: Order Delivered

Modify Order status from confirmed to cancellation

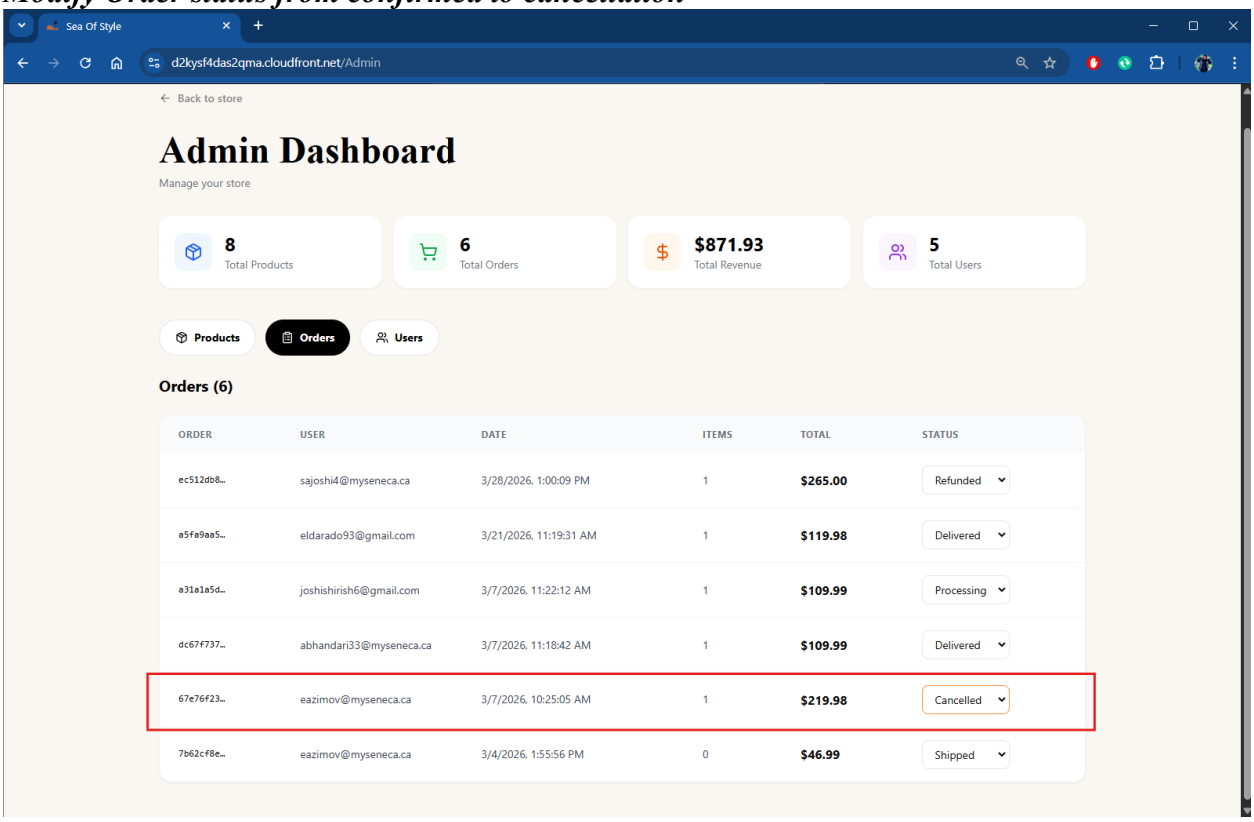


Figure 38: Order Cancelled

Order Delivered on admin side and user side

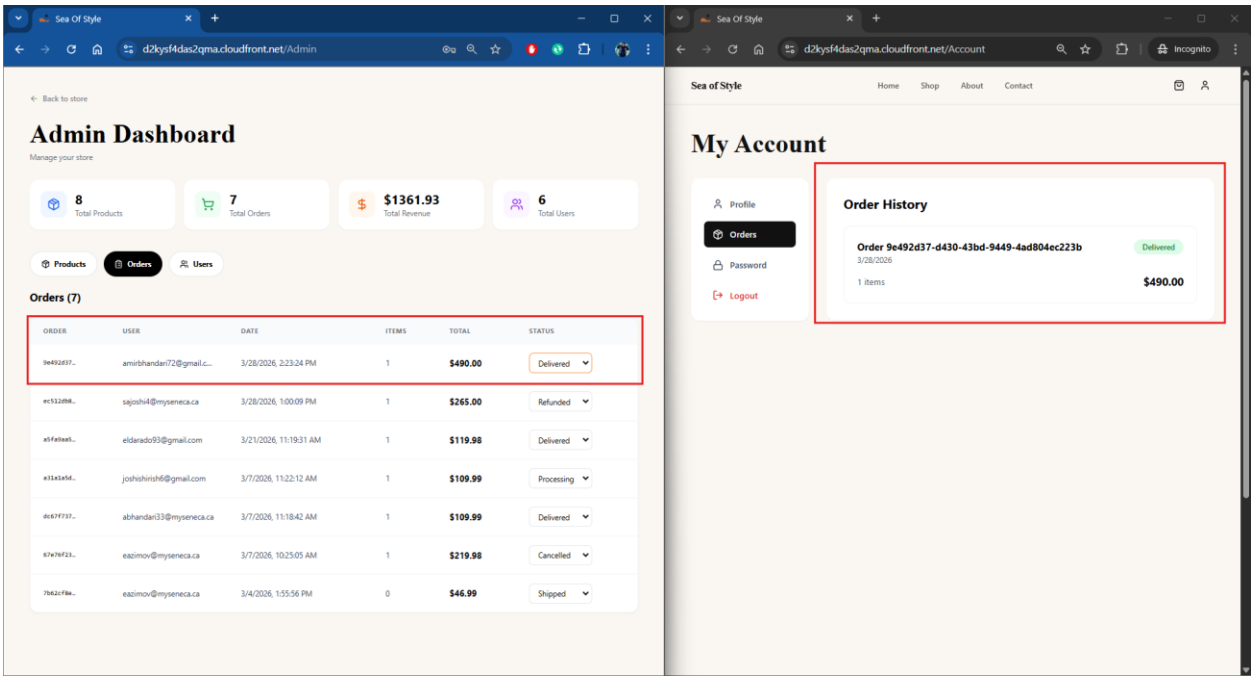


Figure 39: Order status confirmed on both Admin and User side

Admin Password Update

Password Length

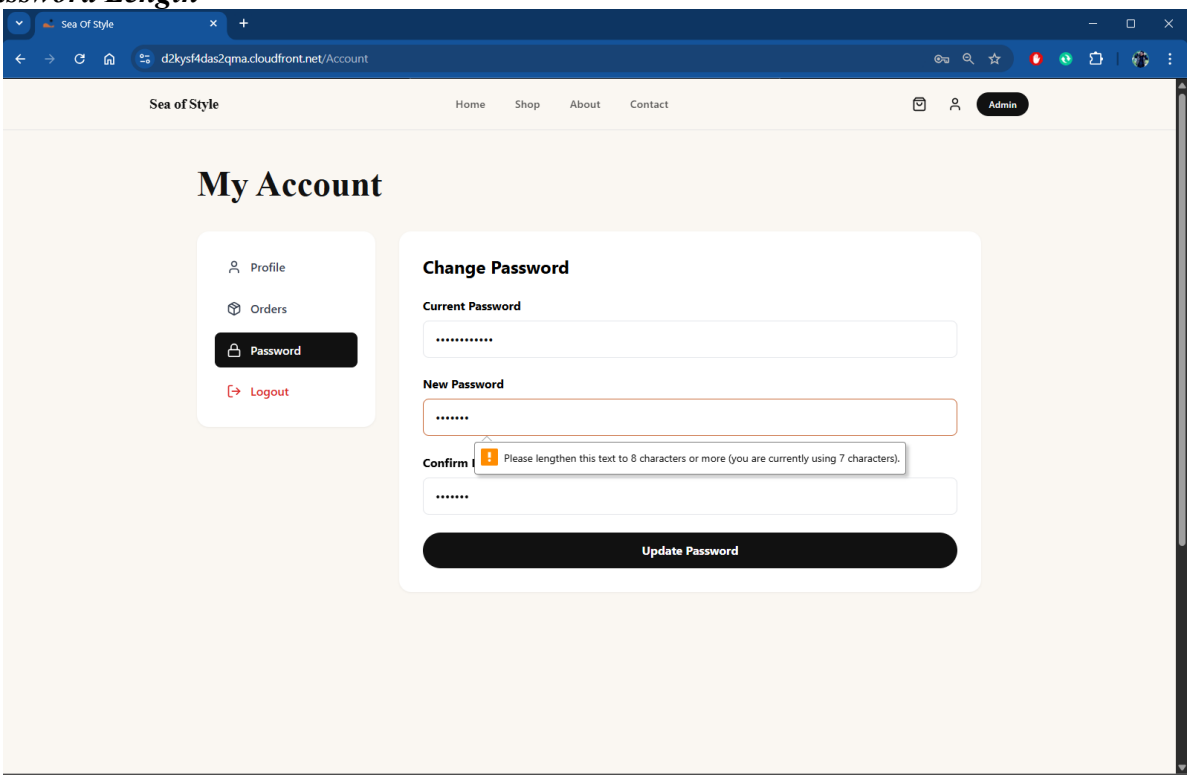


Figure 40: Password Length Error

Update password matching password character length

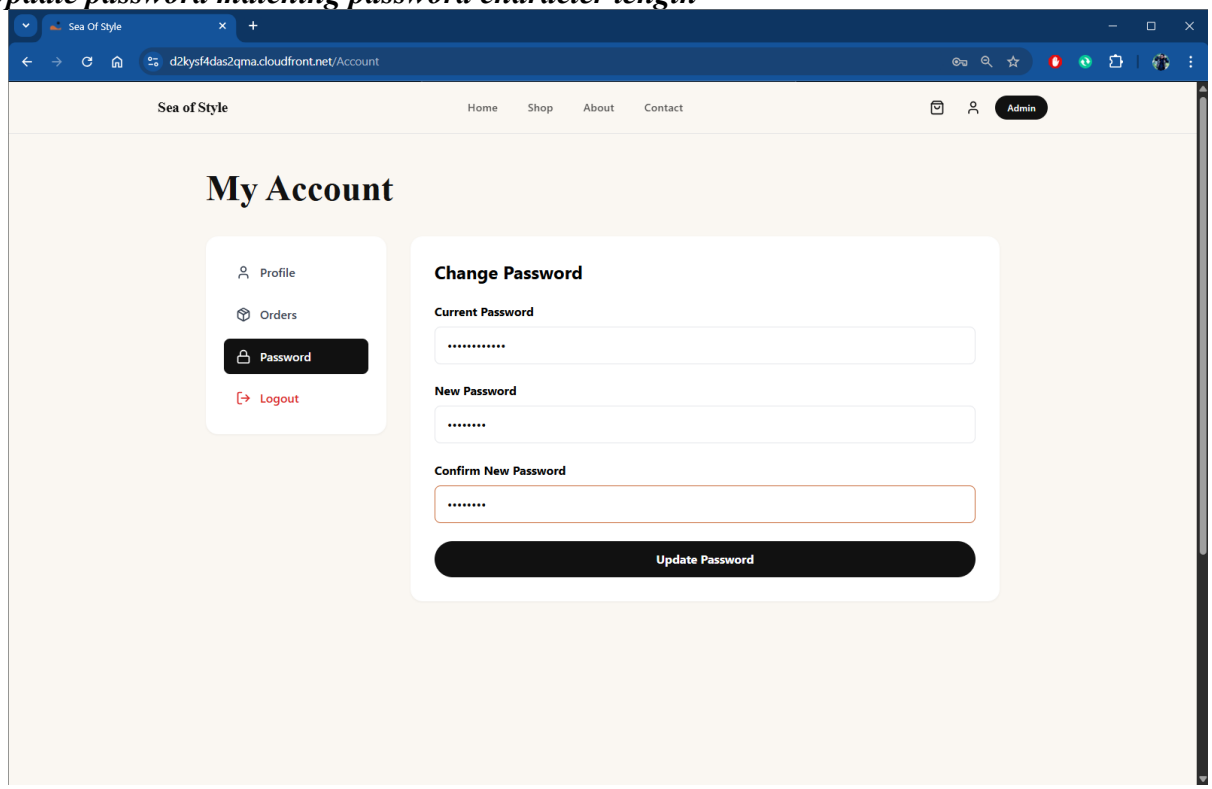


Figure 41: Update Password with valid

Incorrect Password while login

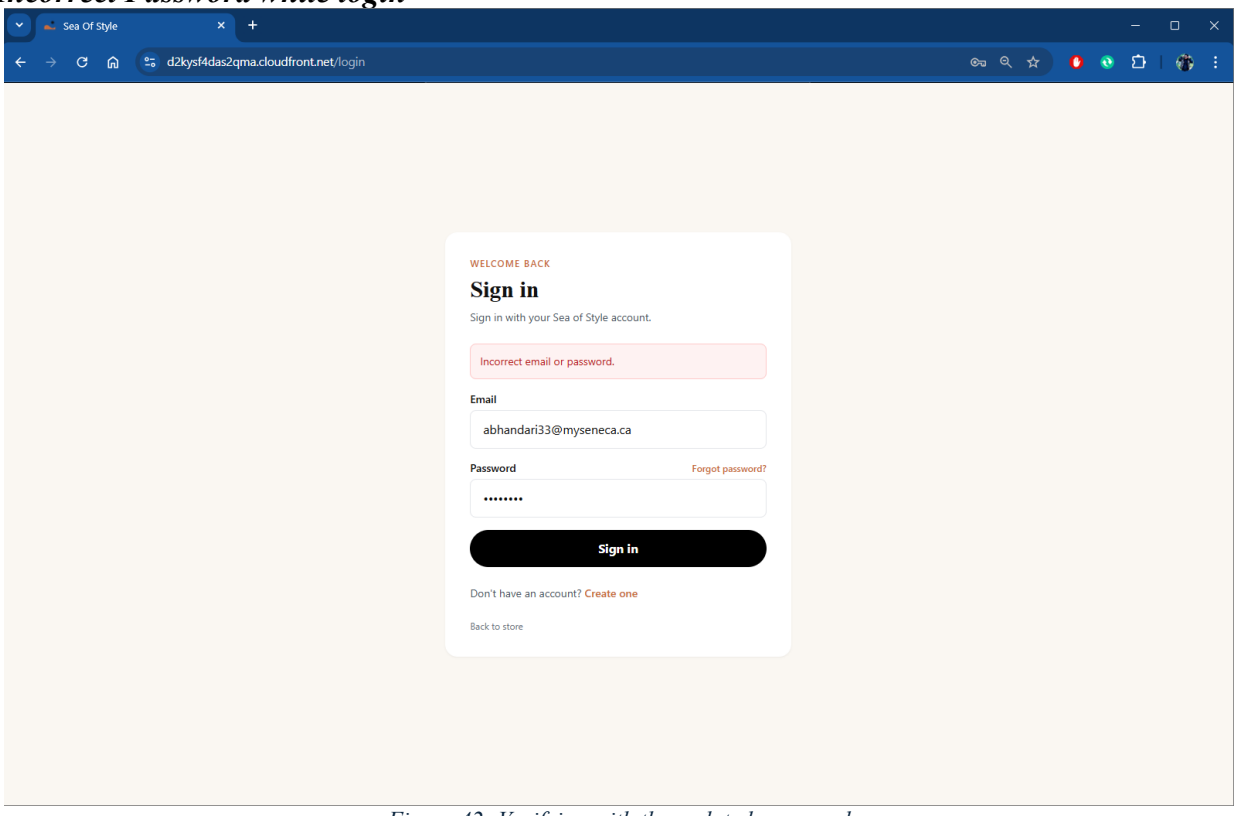


Figure 42: Verifying with the updated password

AWS Services Used

The SOS platform is built using a collection of AWS managed services that together provide a scalable, secure, and serverless architecture. Each service is responsible for a specific function, including frontend delivery, authentication, backend processing, data storage, communication, monitoring, and infrastructure automation.

Table: AWS Services and Their Purpose

Service	Purpose for the Project
Amazon S3	Hosts static frontend files and stores Terraform state
Amazon CloudFront	Delivers content globally with low latency
Amazon Cognito	Manages user authentication and authorization
Amazon API Gateway	Acts as entry point for backend APIs
AWS Lambda	Executes backend business logic
Amazon DynamoDB	Stores application data (products, orders, users)
Amazon SNS	Enables event-driven notifications
Amazon SES	Provisioned in infrastructure (password reset handled by Cognito)
Amazon Lex	Provides chatbot-based interaction
Amazon CloudWatch	Monitors logs and system performance
AWS CloudTrail	Tracks API and account activity
AWS IAM	Manages roles and permissions
AWS KMS	Encrypts sensitive data
AWS Secrets Manager	Stores sensitive credentials securely
Terraform	Automates infrastructure provisioning
GitHub Actions	Implements CI/CD pipeline

Core Application Services

These services form the foundation of the SOS platform and handle the main application functionality.

Amazon S3

Amazon S3 is used to host the static frontend of the application and store Terraform state files for infrastructure management.

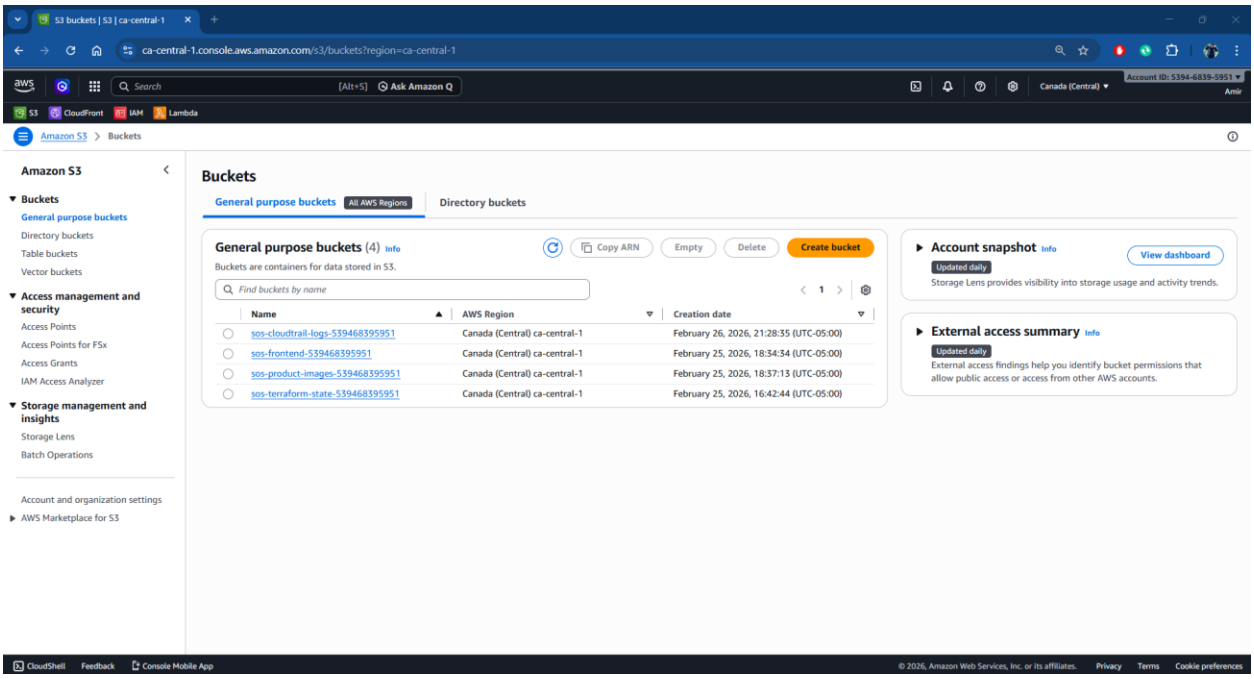


Figure 43: S3 Bucket List

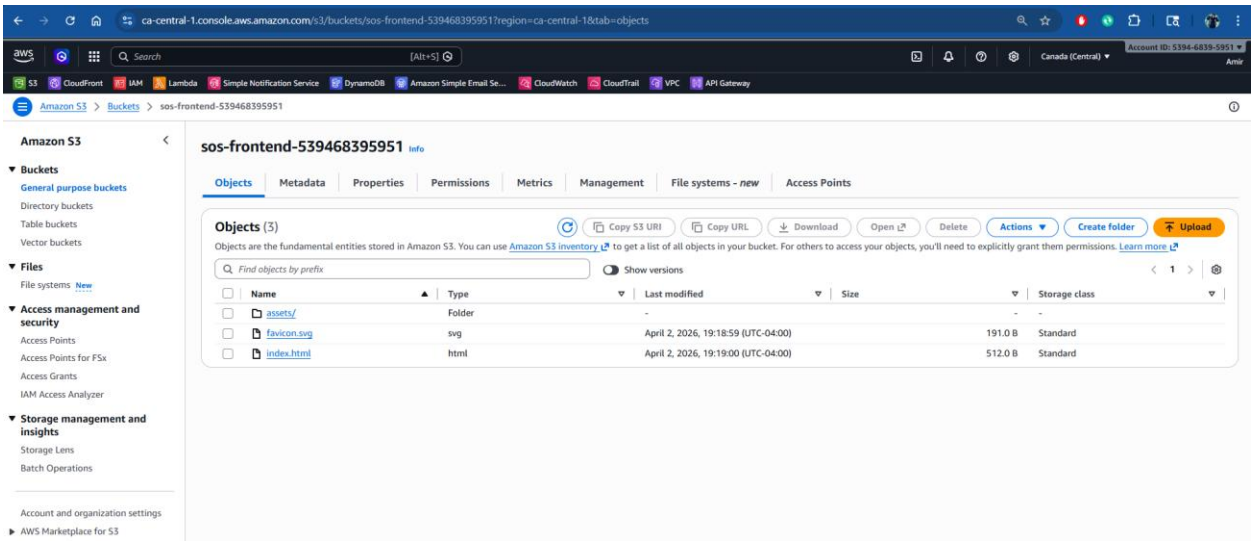


Figure 44: S3 Bucket (Frontend Hosting)

Amazon CloudFront

CloudFront improves performance by caching frontend content at edge locations and delivering it securely over HTTPS.

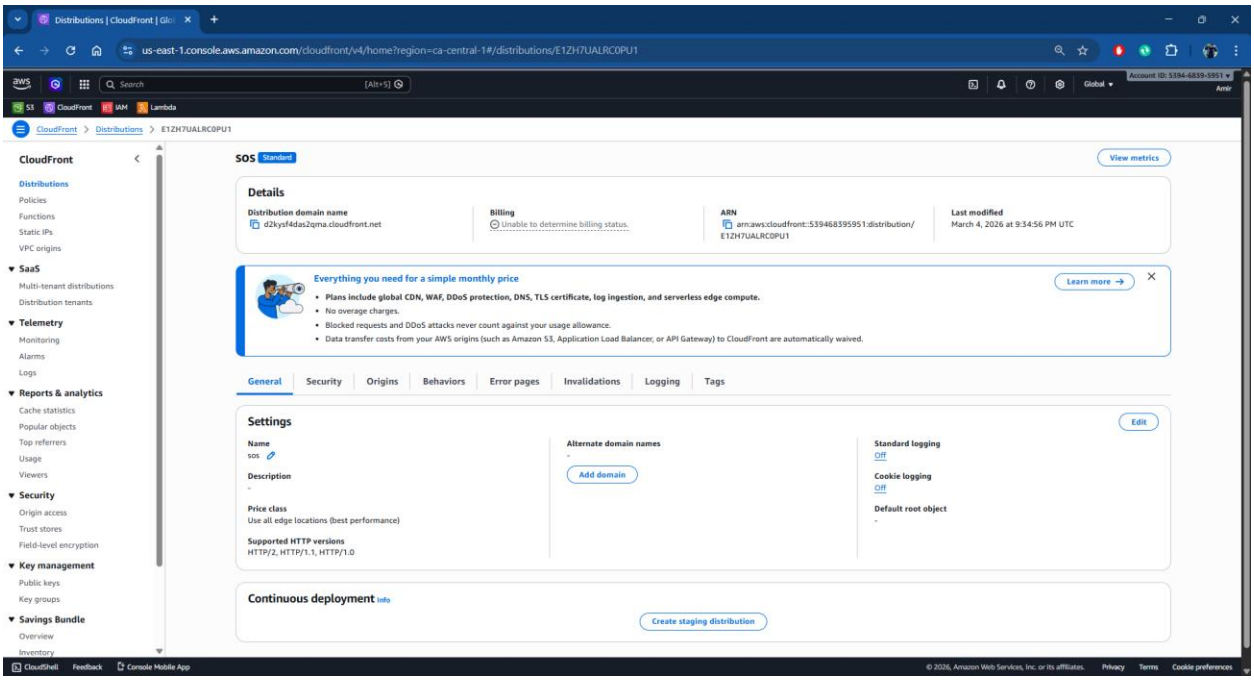


Figure 45: CloudFront Distribution

Amazon Cognito

Cognito manages user authentication, including signup, login, and password recovery (Forgot Password).

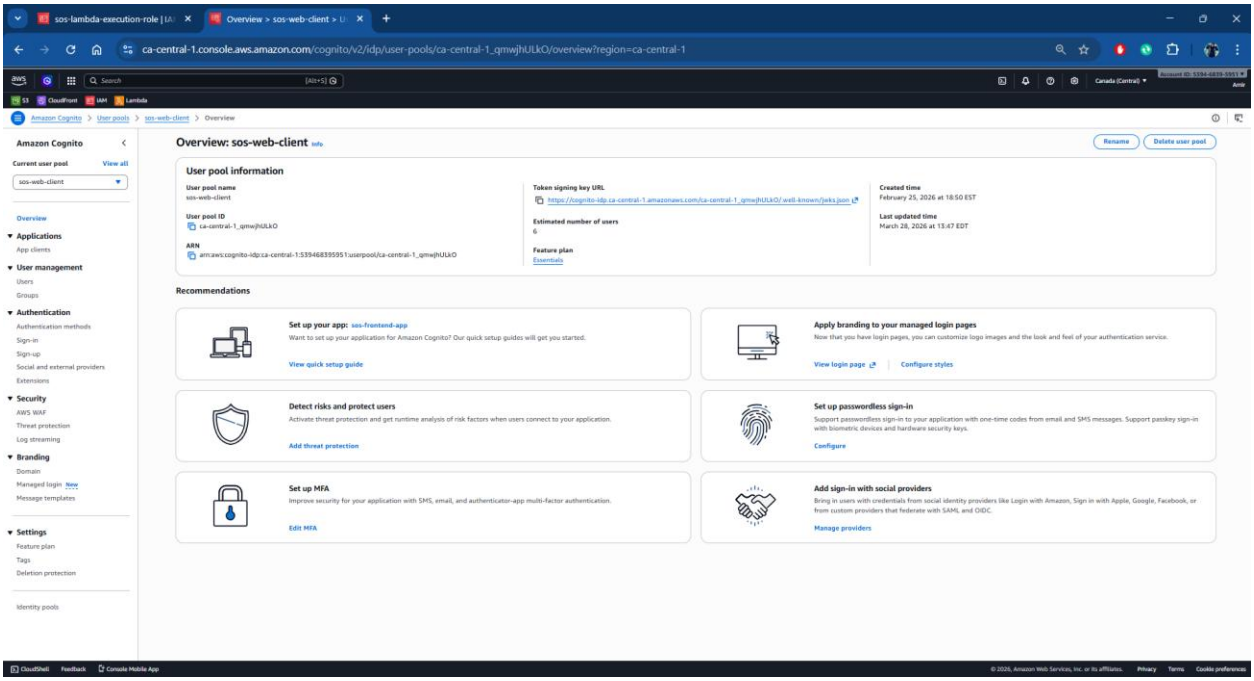


Figure 46: Amazon Cognito Overview

Amazon Cognito – App Client

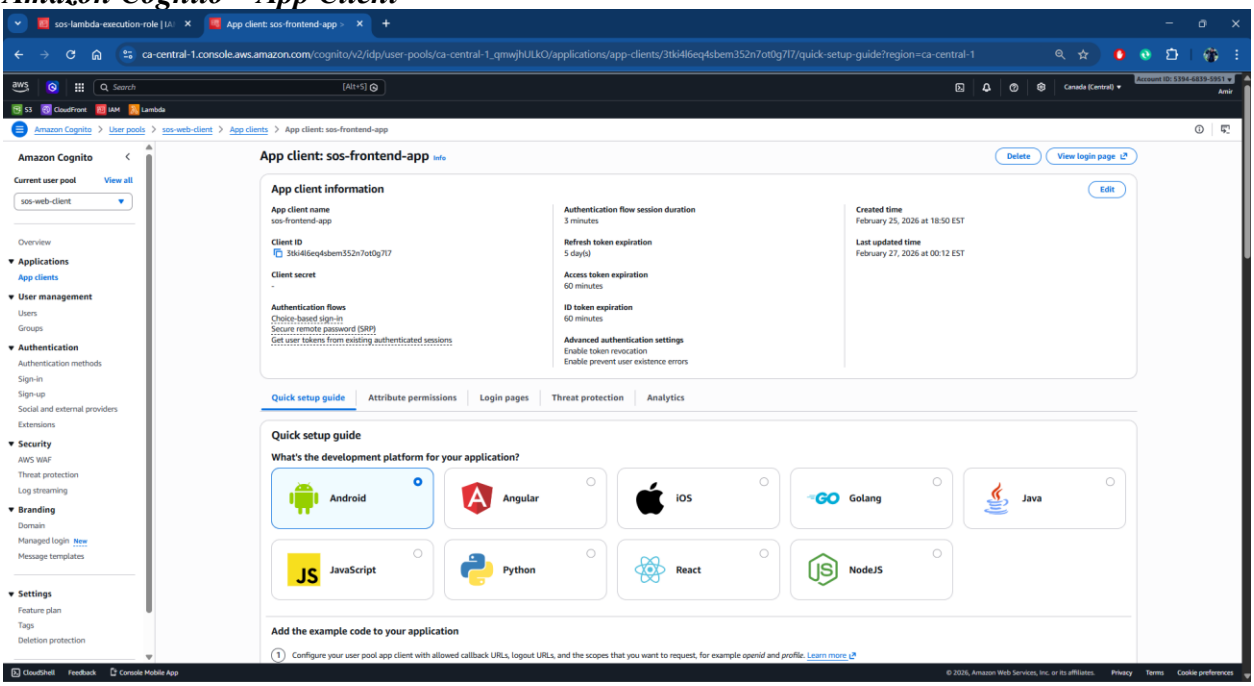


Figure 47: Amazon Cognito - App Client

Amazon Cognito – Users

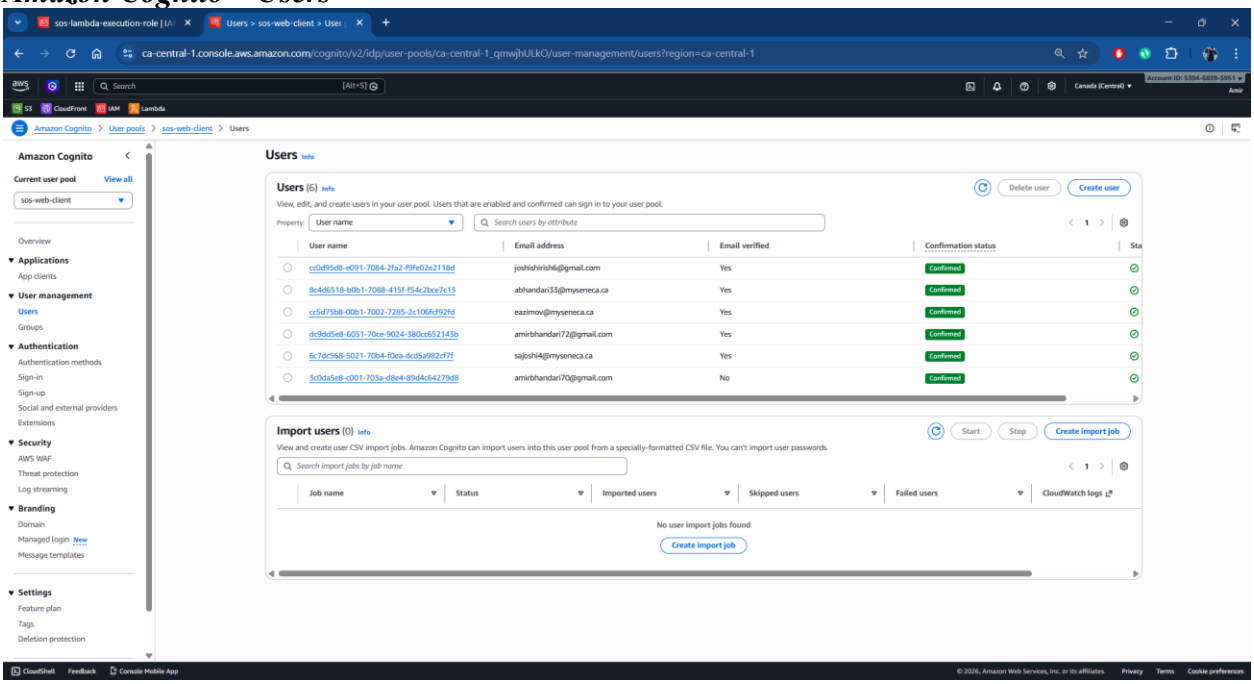


Figure 48: Amazon Cognito – Users

Amazon Cognito – Groups

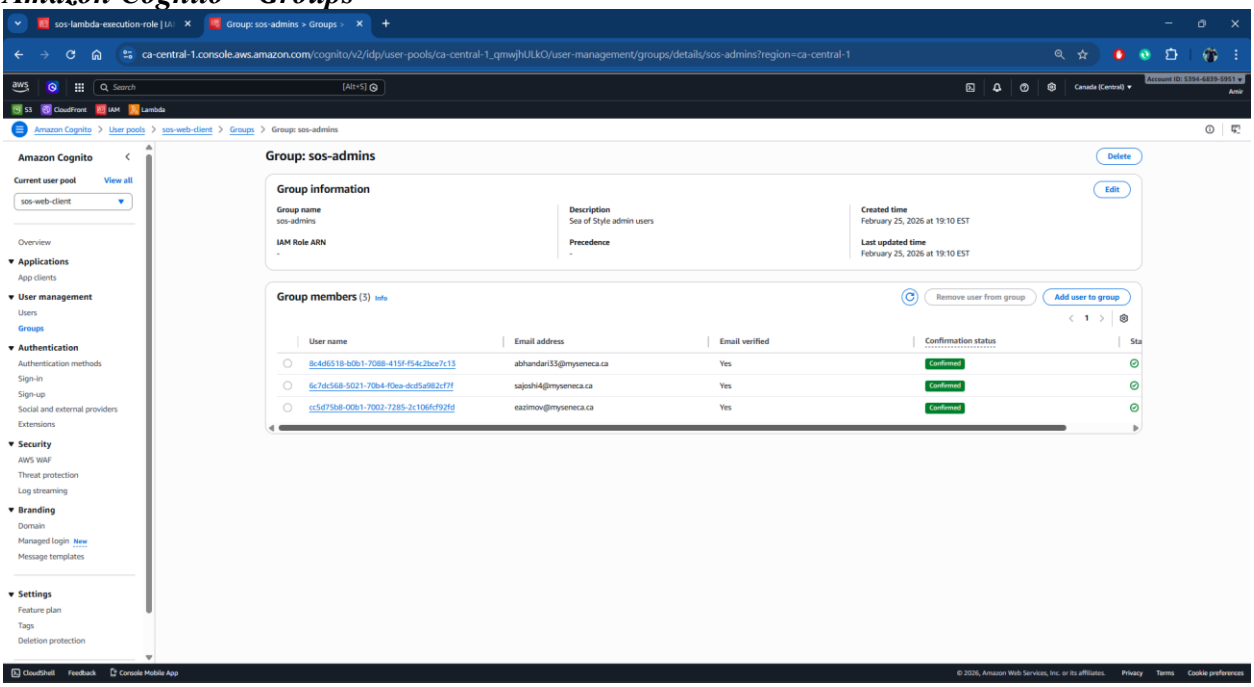


Figure 49: Amazon Cognito - Groups

Amazon API Gateway

API Gateway acts as the entry point for all backend requests and routes them to Lambda functions.

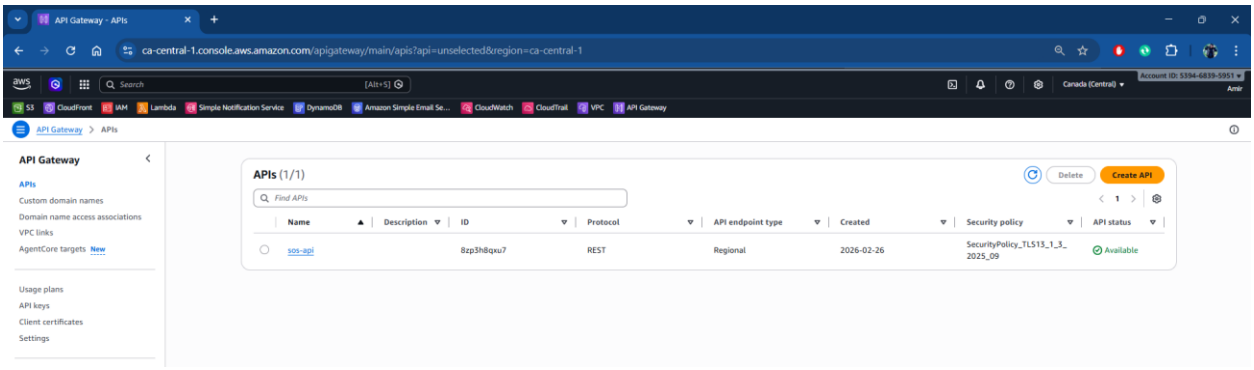


Figure 50: Amazon API Gateway

Resources

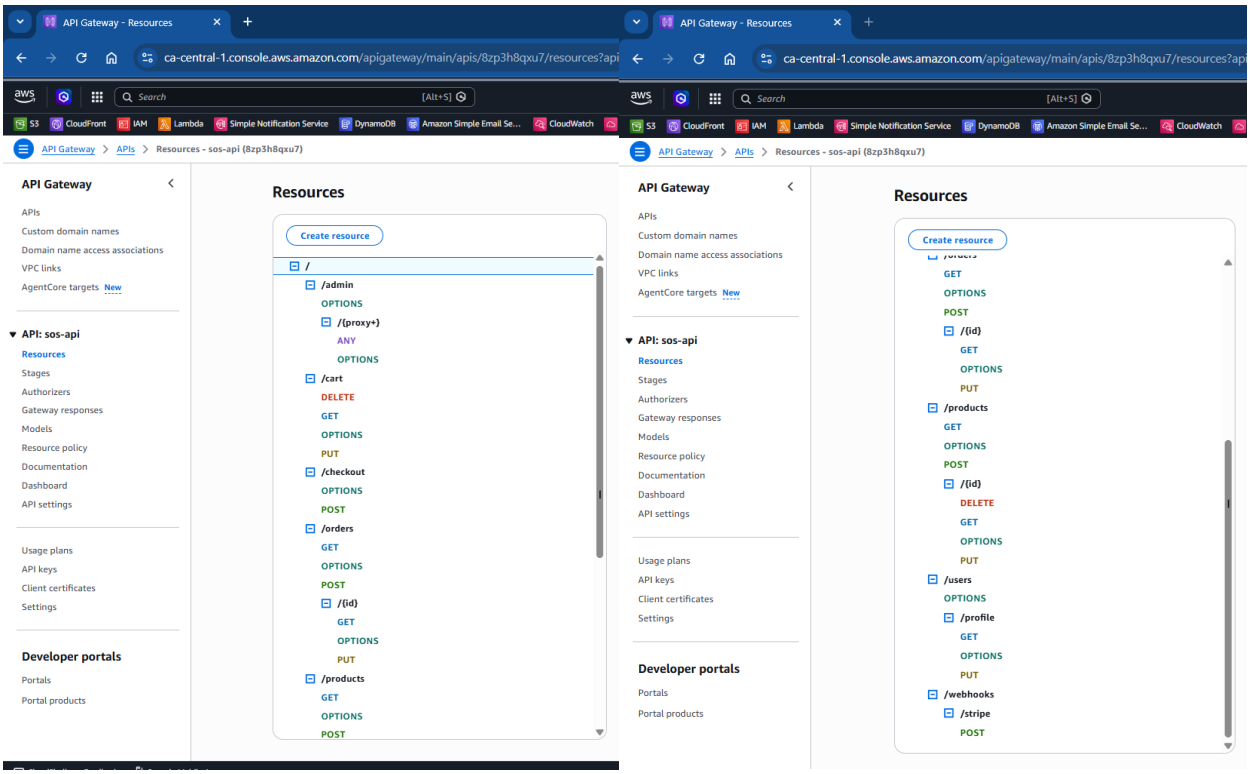


Figure 51: API Gateway - Resources

Stages

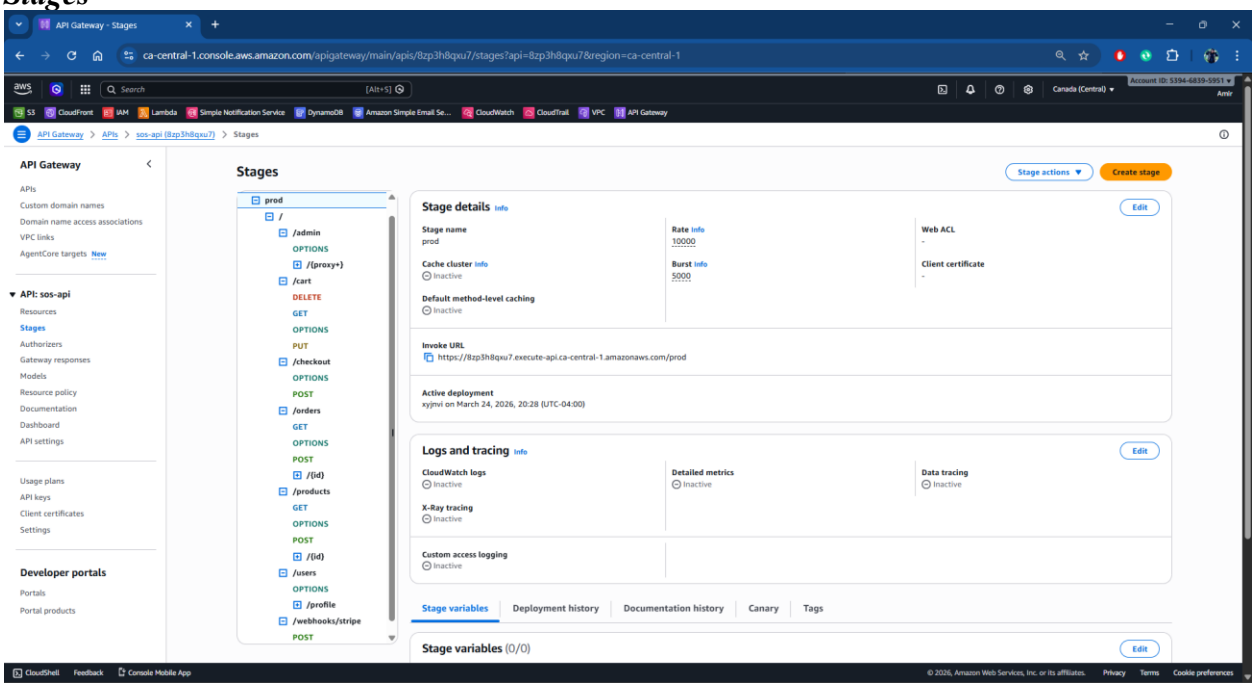


Figure 52: API Gateway – Stages

API Dashboard

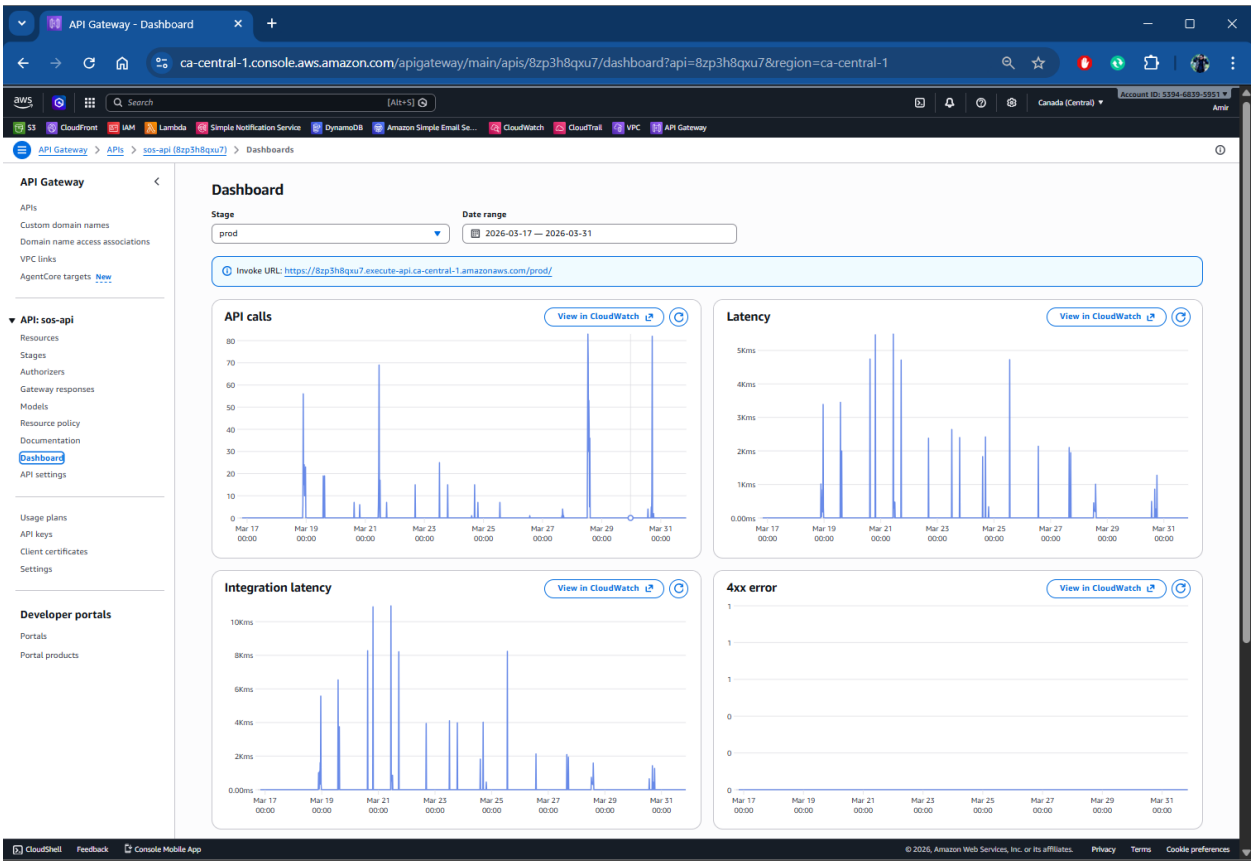


Figure 53: API Gateway - Dashboard

AWS Lambda

Lambda executes backend logic such as product retrieval, cart operations, and order processing.

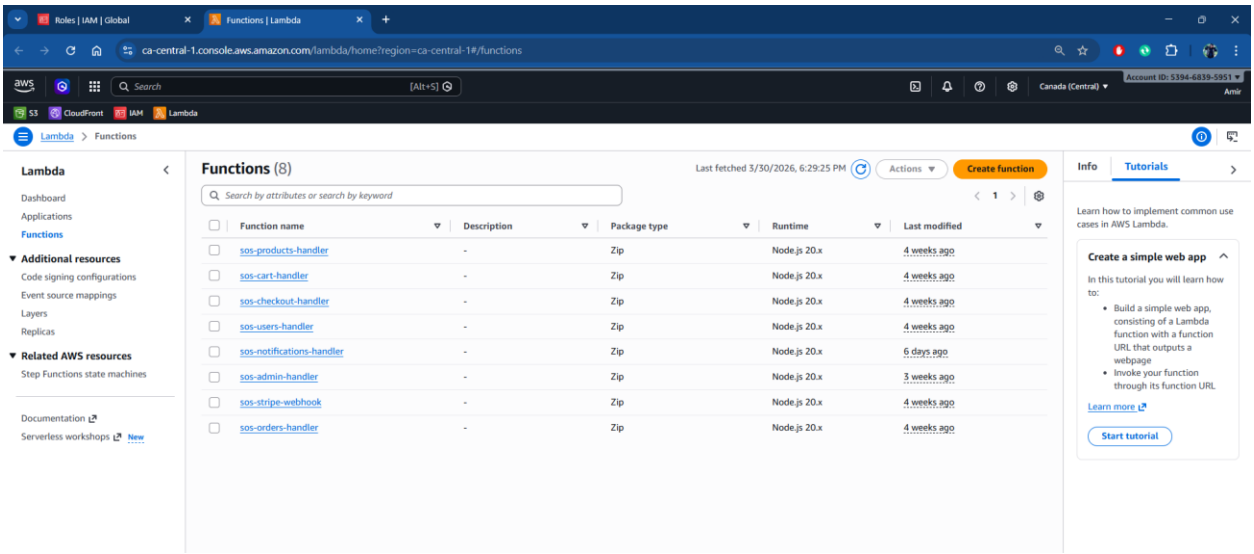


Figure 54: AWS Lambda Functions List

SOS-Products-handler

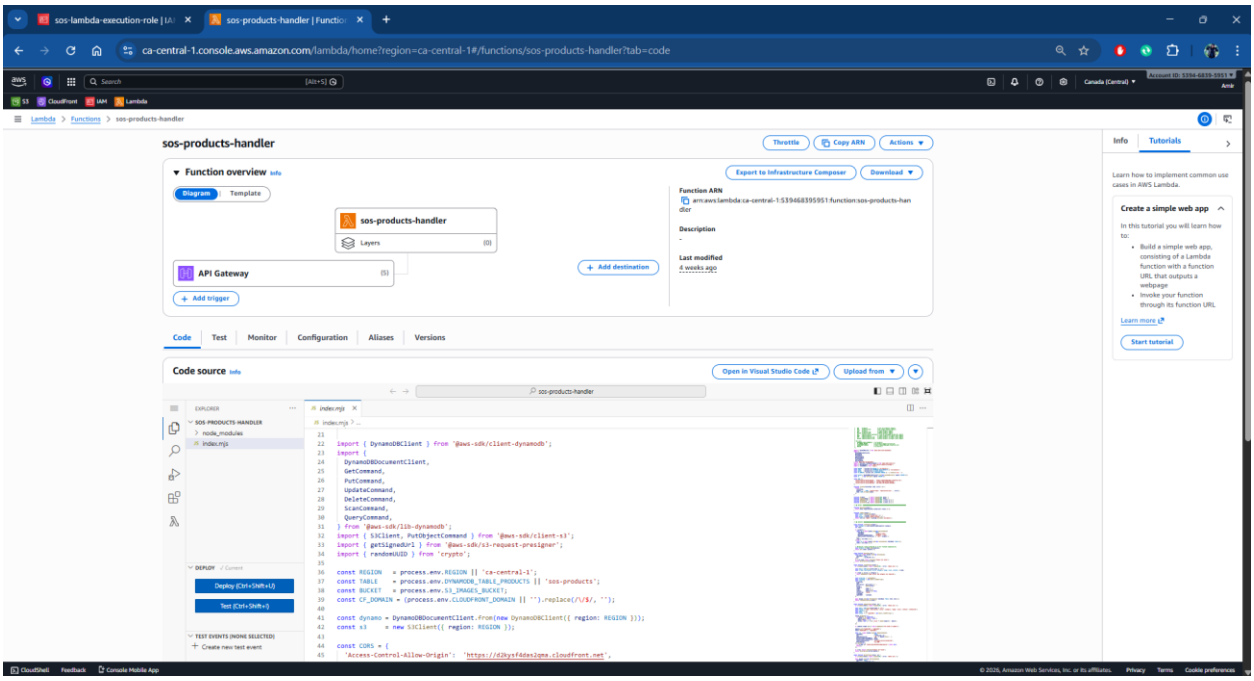


Figure 55: AWS Lambda Functions (sos-products-handler)

SOS-Cart-Handler

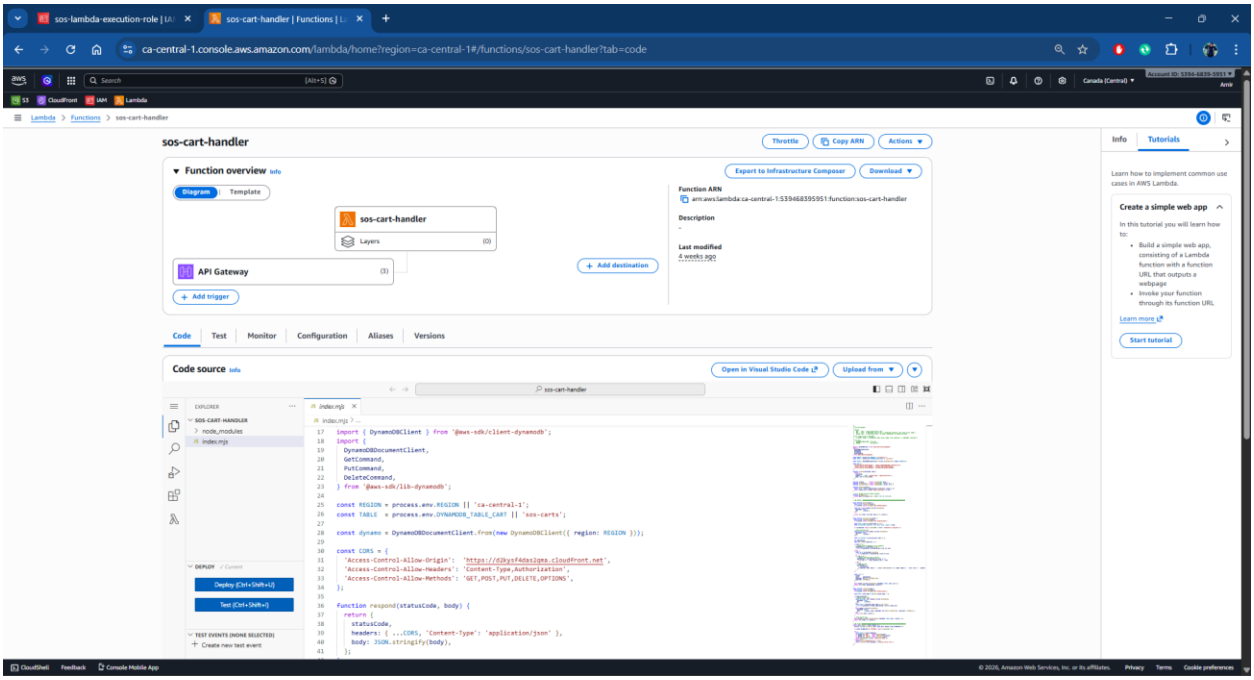


Figure 56: AWS Lambda Functions (sos-cart-handler)

sos-checkout-handler

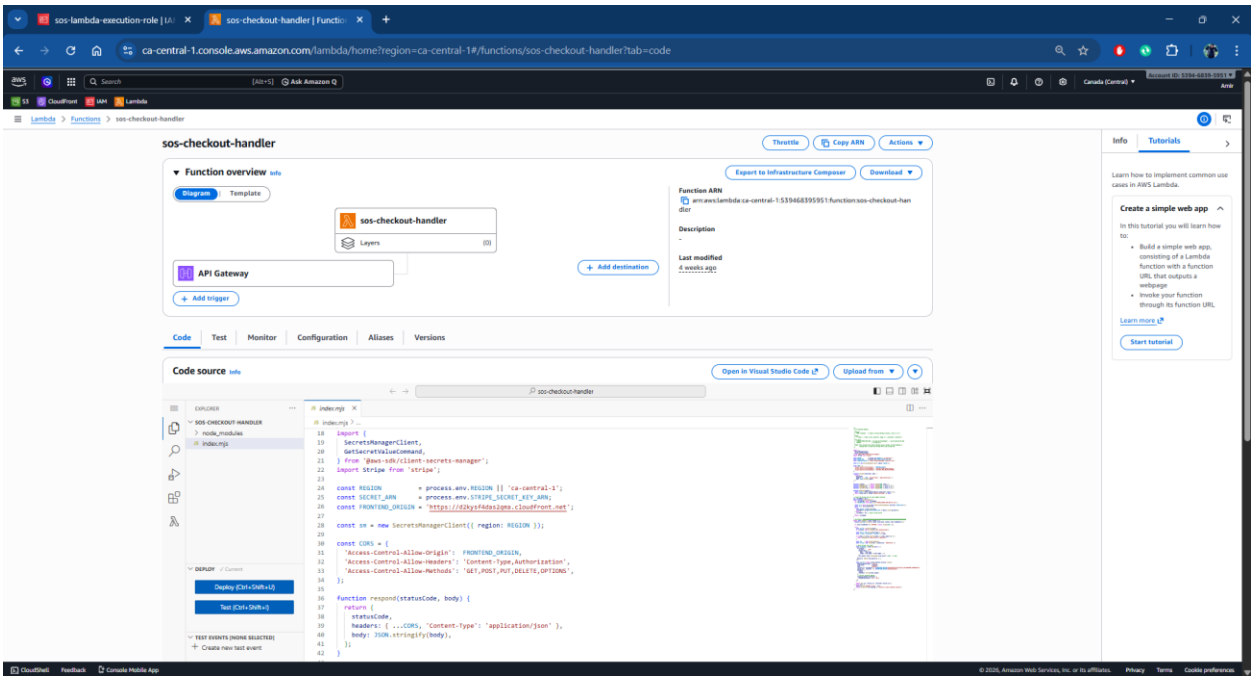


Figure 57: AWS Lambda Functions (sos-checkout-handler)

sos-users-handler

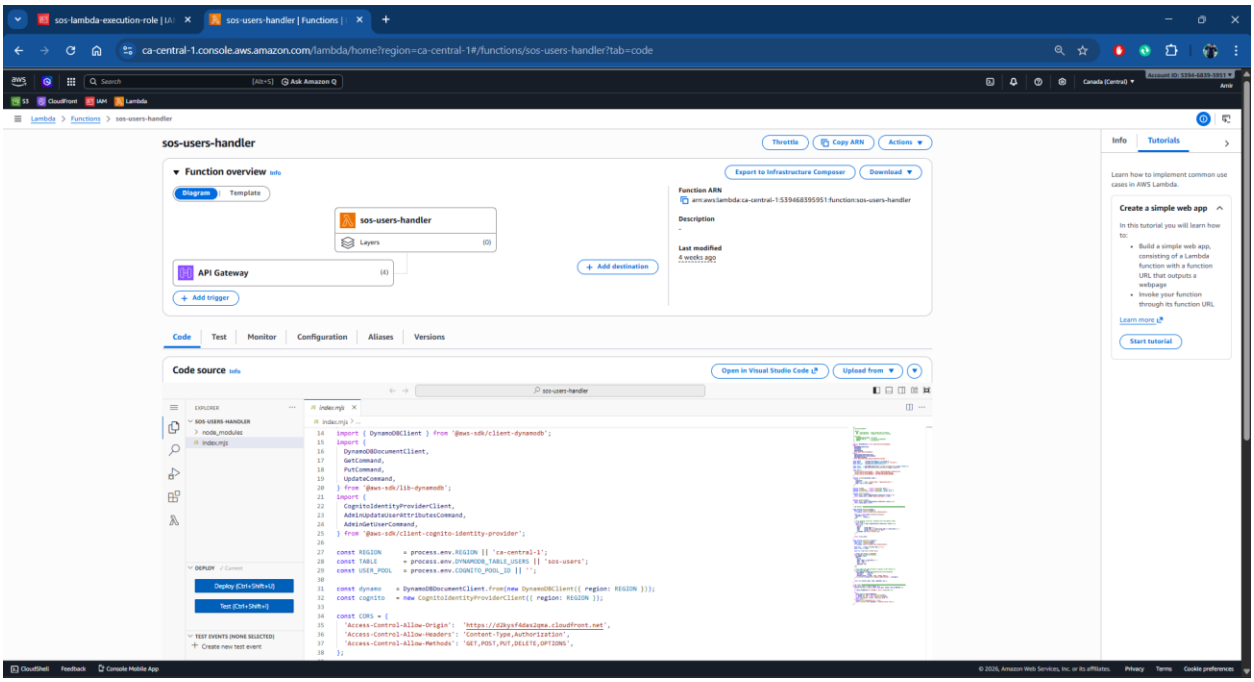


Figure 58: AWS Lambda Functions (sos-users-handler)

sos-notifications-handler

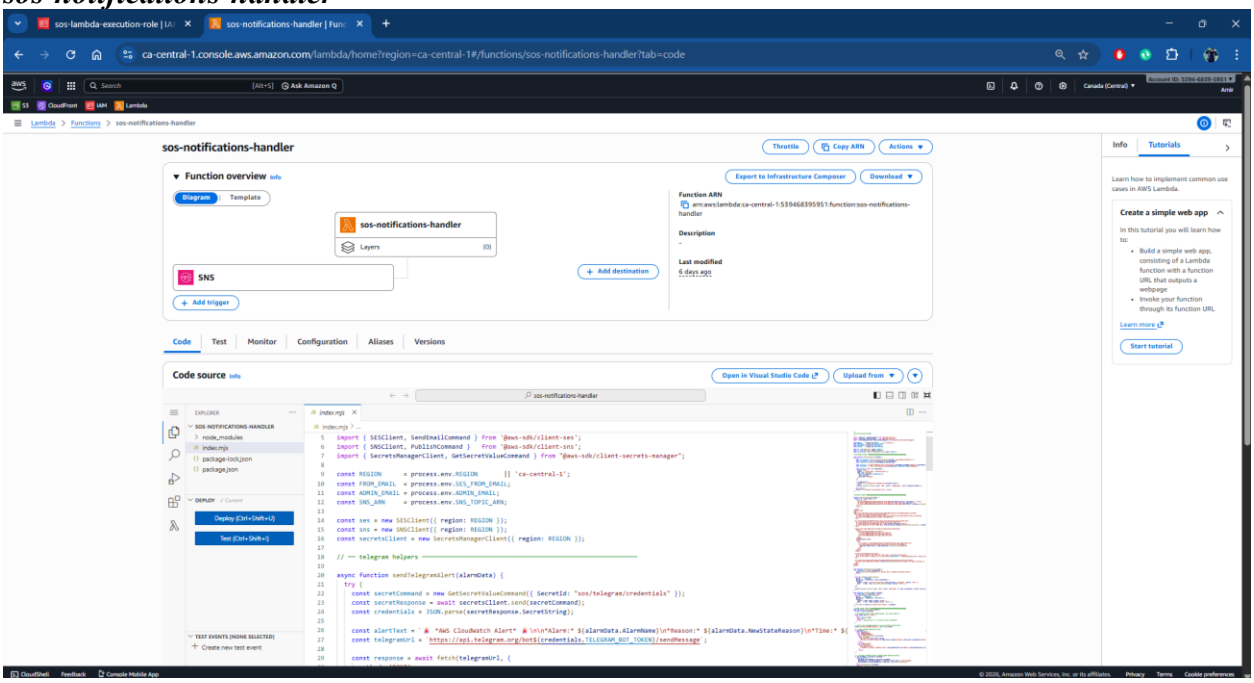


Figure 59: AWS Lambda Functions (sos-notifications-handler)

sos-admin-handler

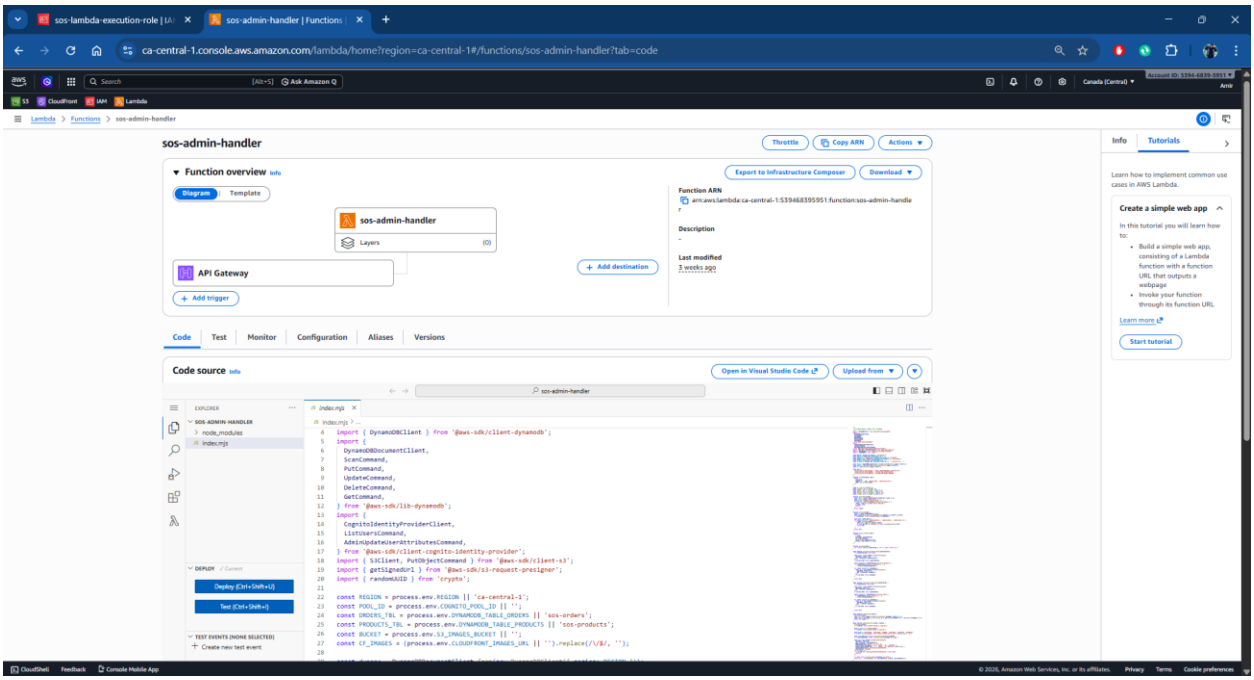


Figure 60: AWS Lambda Functions (sos-admin-handler)

sos-stripe-webhook

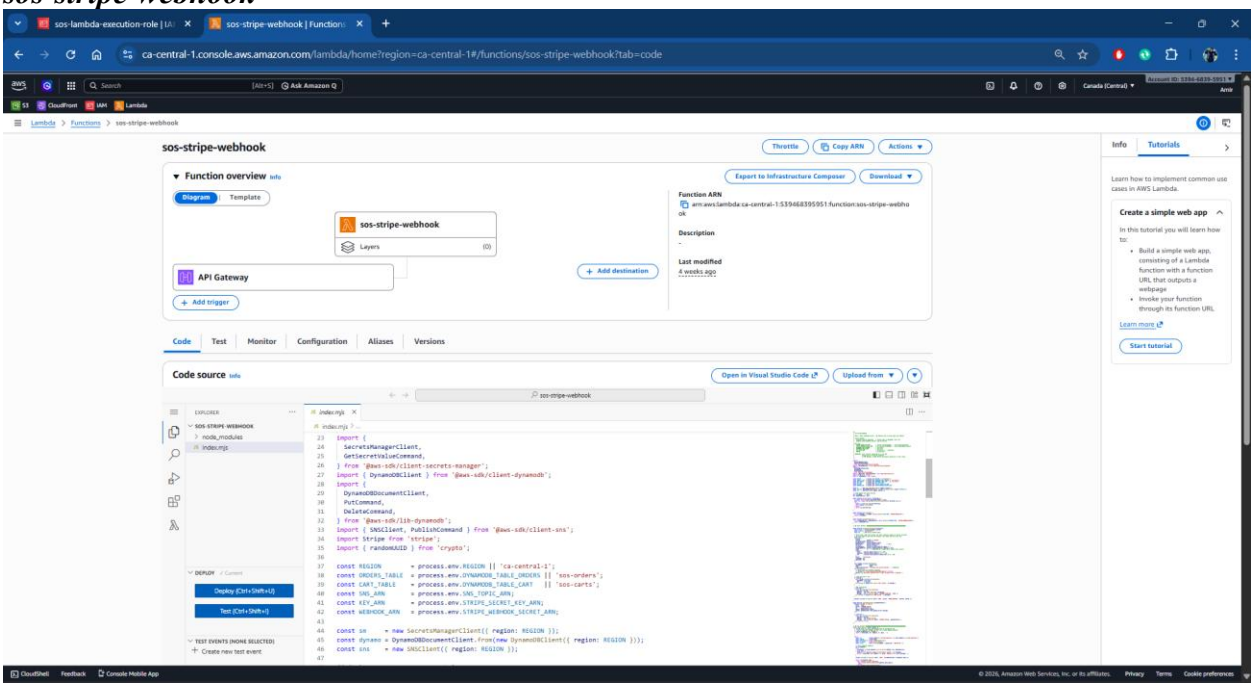


Figure 61: AWS Lambda Functions (sos-stripe-webhook)

sos-orders-handler

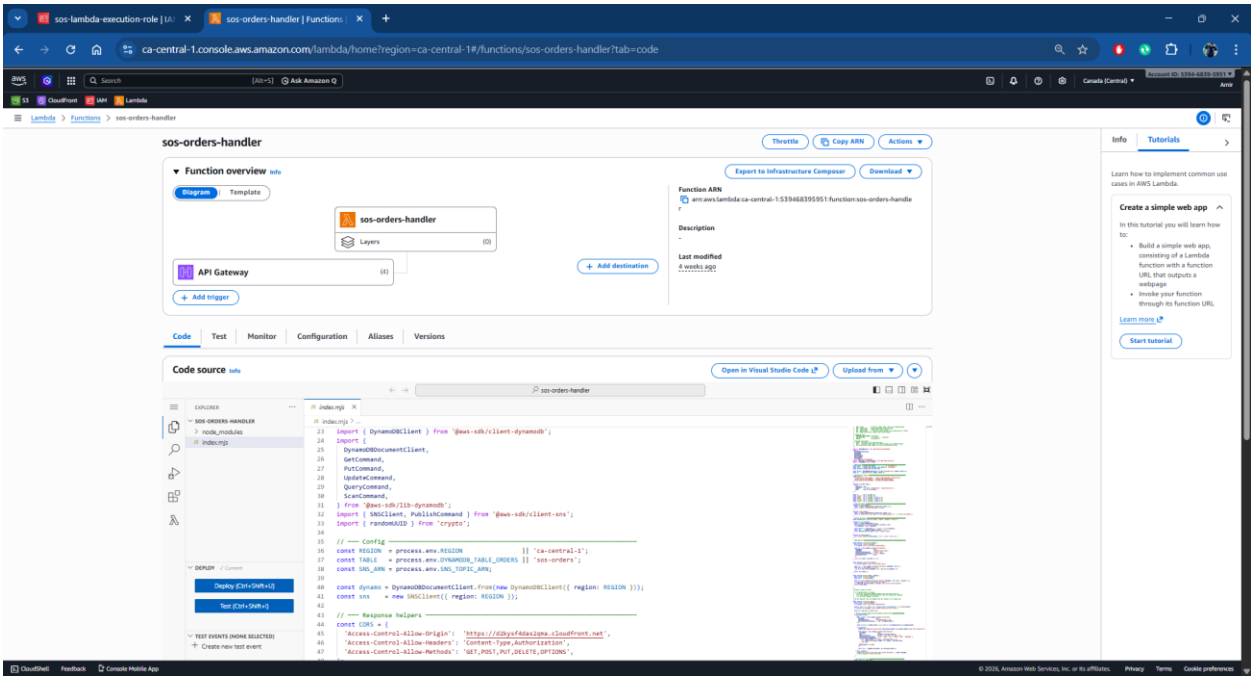


Figure 62: AWS Lambda Functions (sos-orders-handler)

Amazon DynamoDB

DynamoDB stores application data with high scalability and low latency.

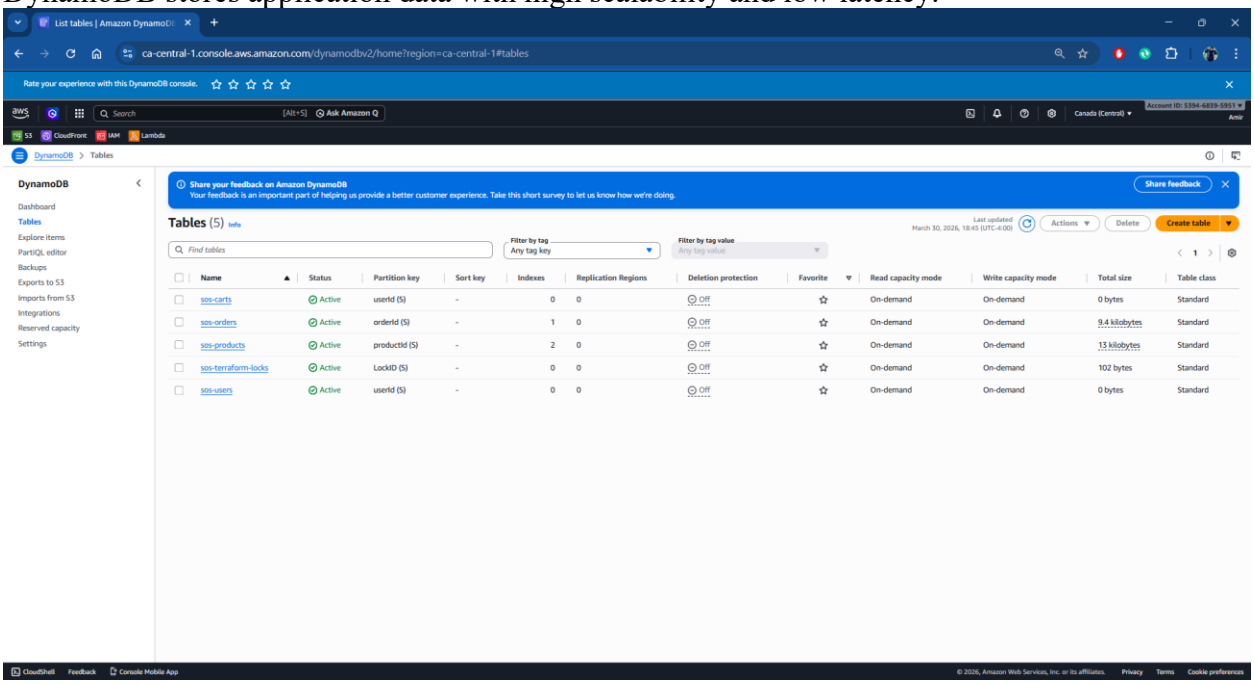


Figure 63: Amazon DynamoDB Tables

DynamoDB – sos-orders

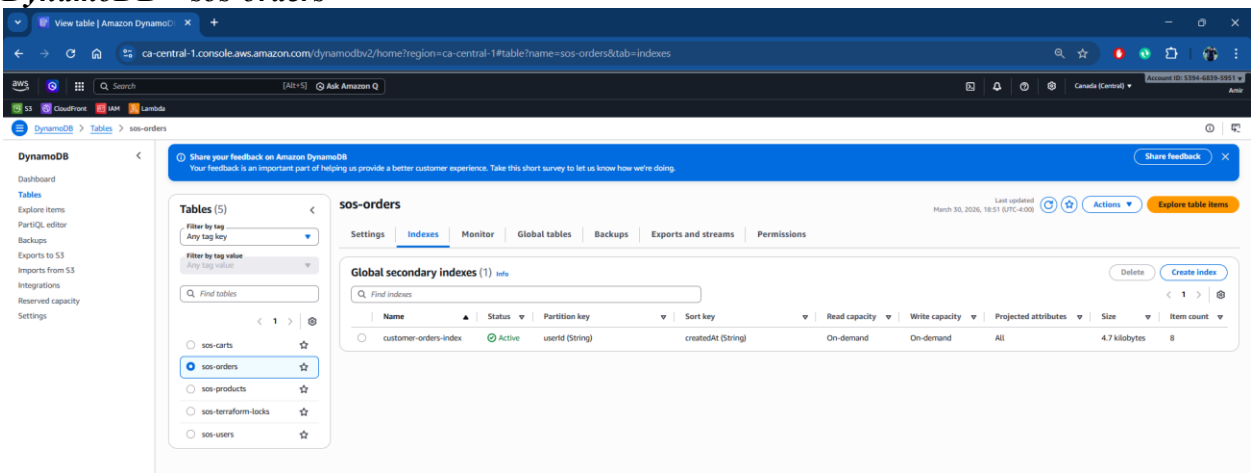


Figure 64: Amazon DynamoDB (sos-orders)

DynamoDB – sos-products

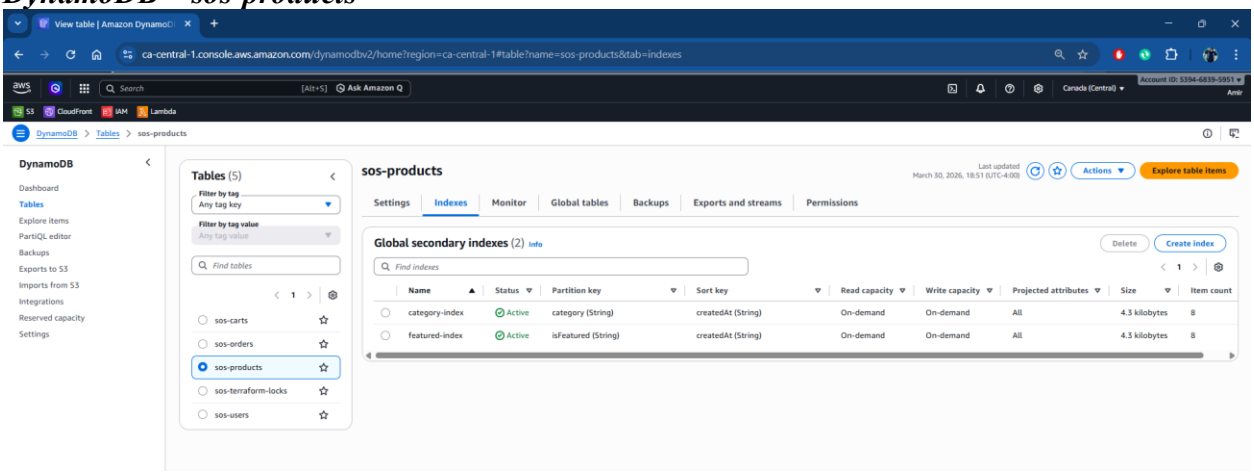


Figure 65: Amazon DynamoDB (sos-products)

DynamoDB – sos-orders – explore items

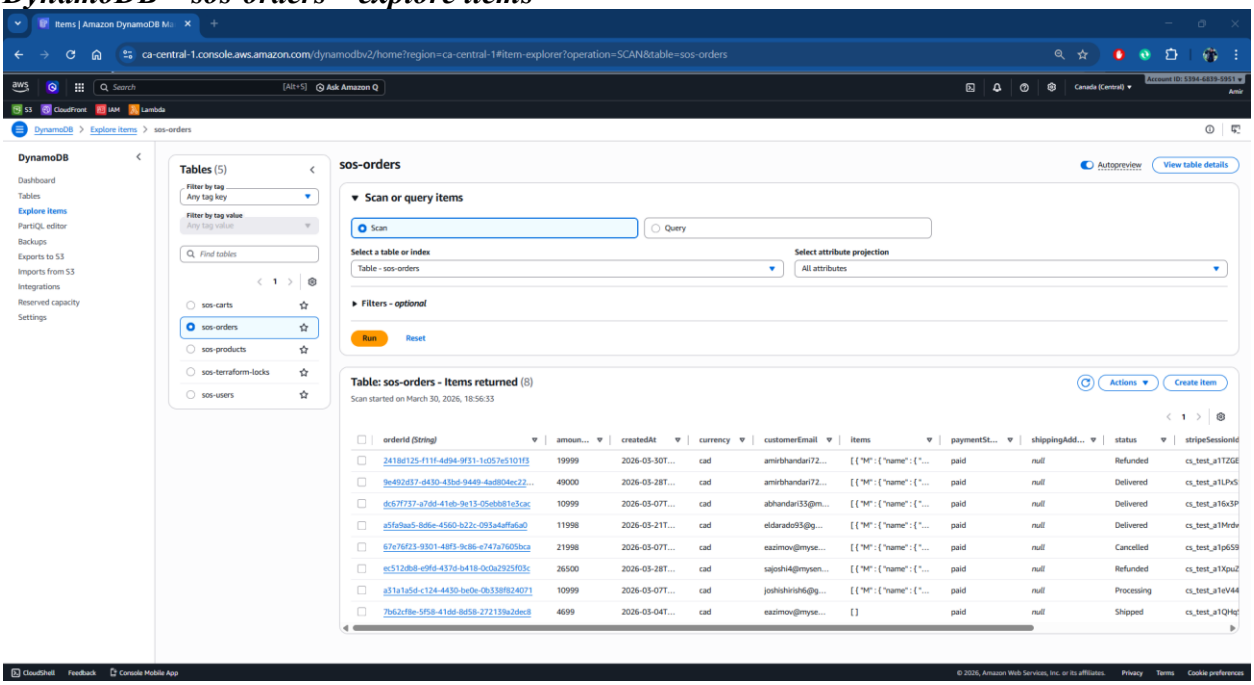


Figure 66: Amazon DynamoDB (sos-orders-explore-item)

DynamoDB – sos-products – explore items

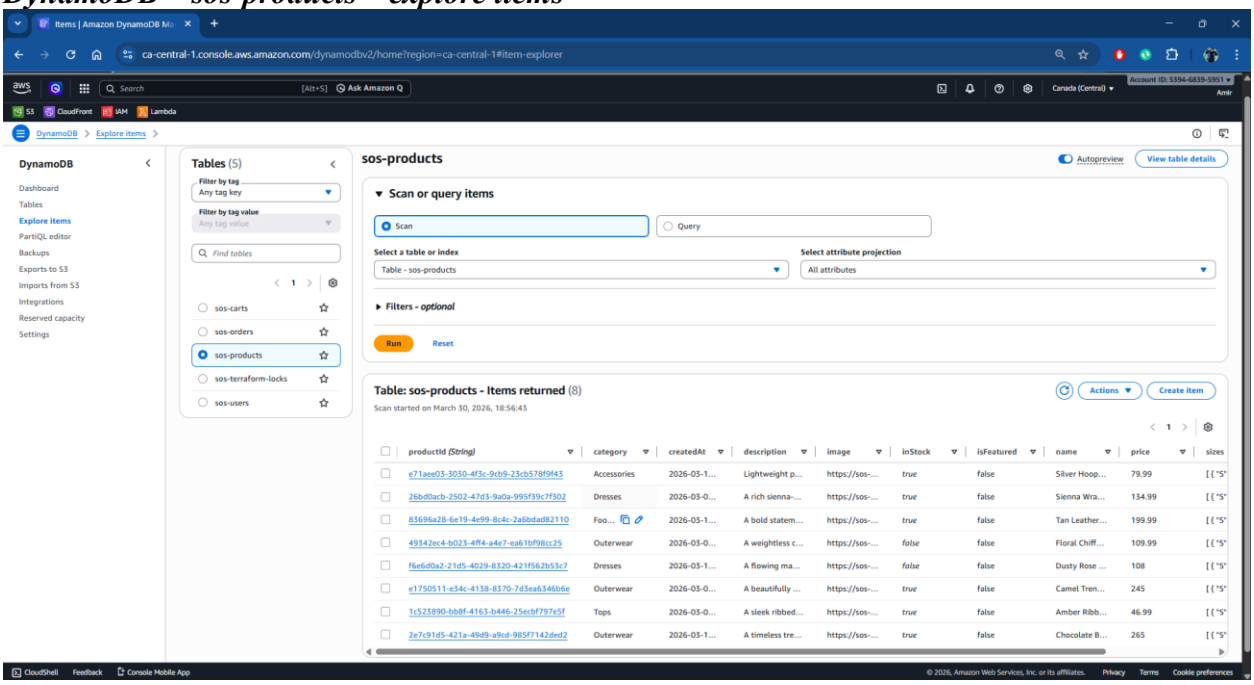


Figure 67: Amazon DynamoDB (sos-products-explore-items)

Integration and Communication Services

These services enable interaction and communication within the system.

Amazon SNS

SNS is used for event-driven communication, triggering workflows like notifications.

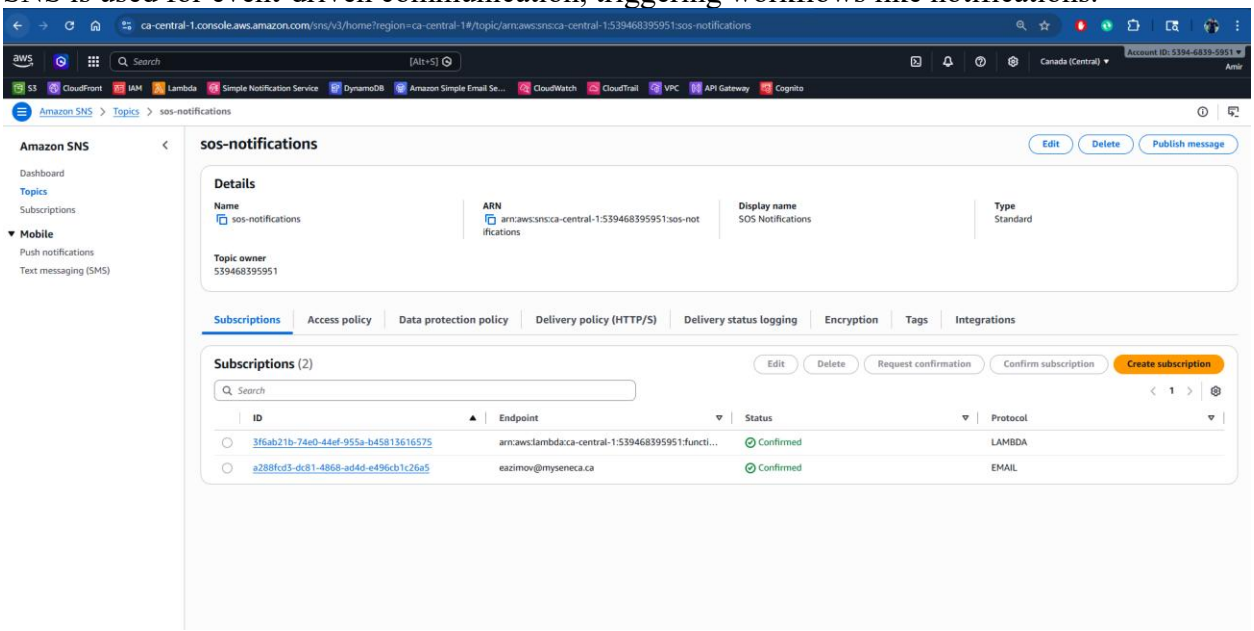


Figure 68: Amazon SNS Topic

Amazon SES

Email-based communication for password recovery is handled by Amazon Cognito. Order-related notifications are delivered through Telegram integration instead of email.

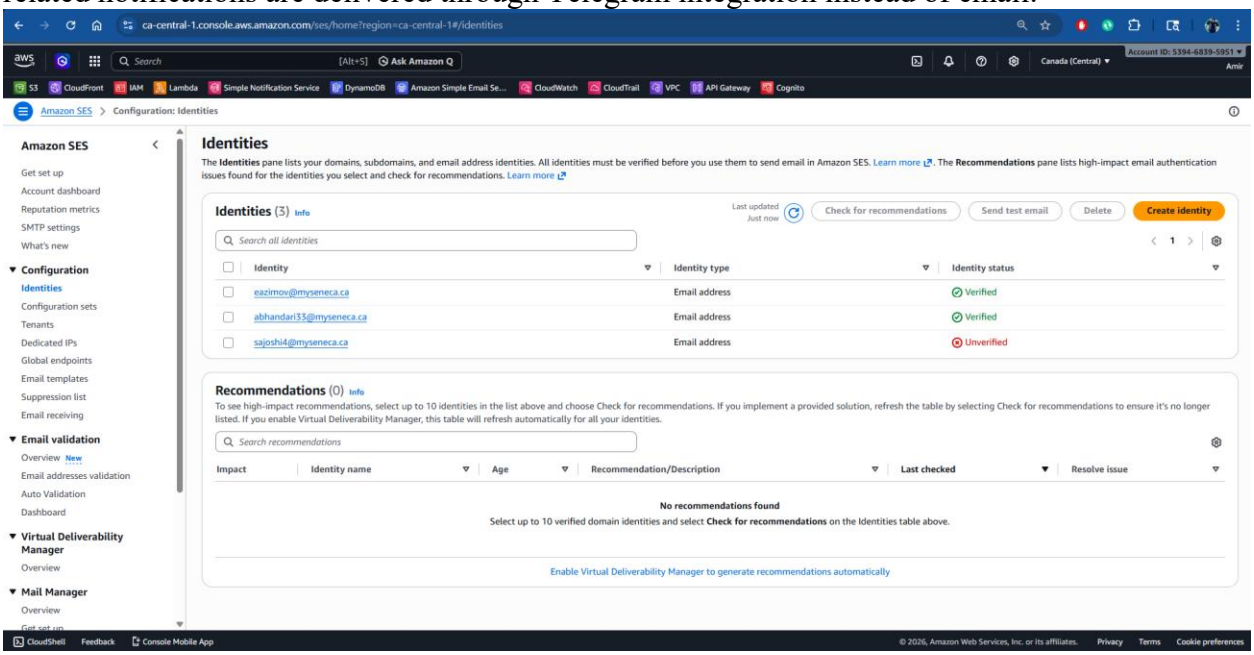


Figure 69: Amazon SES Identities

Amazon Lex

Lex provides a chatbot interface that allows users to interact with the application using natural language.

Amazon LEX - Overview

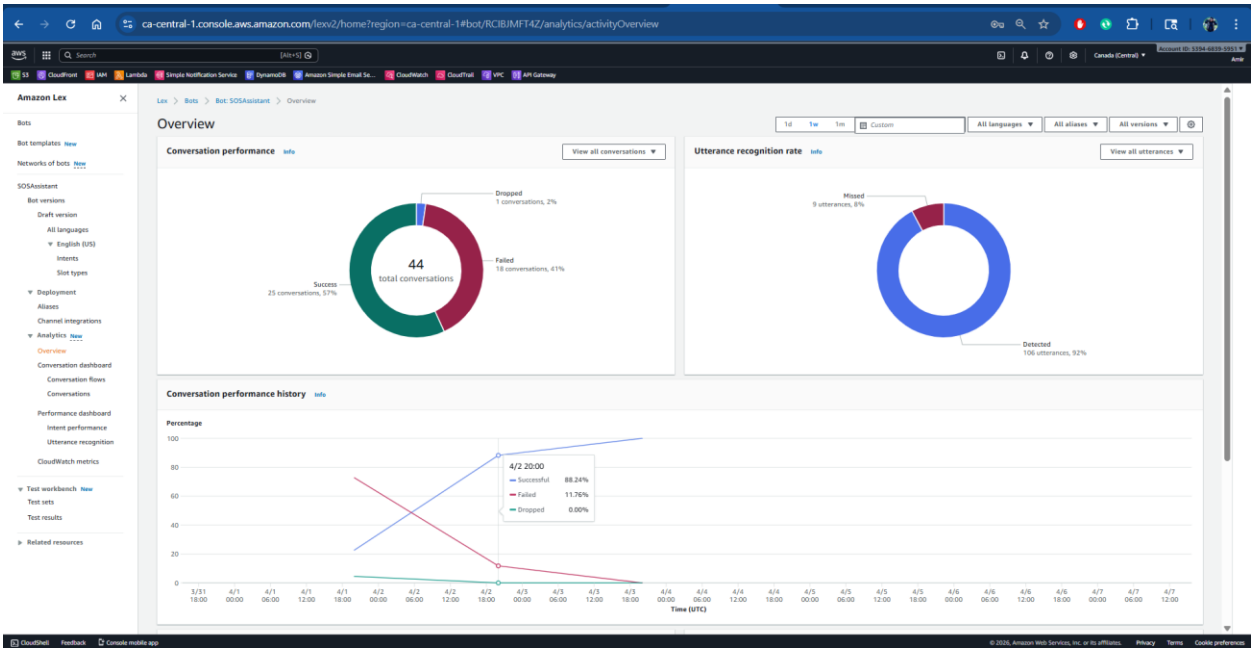


Figure 70: Amazon Lex - Overview

Amazon Lex – Intents

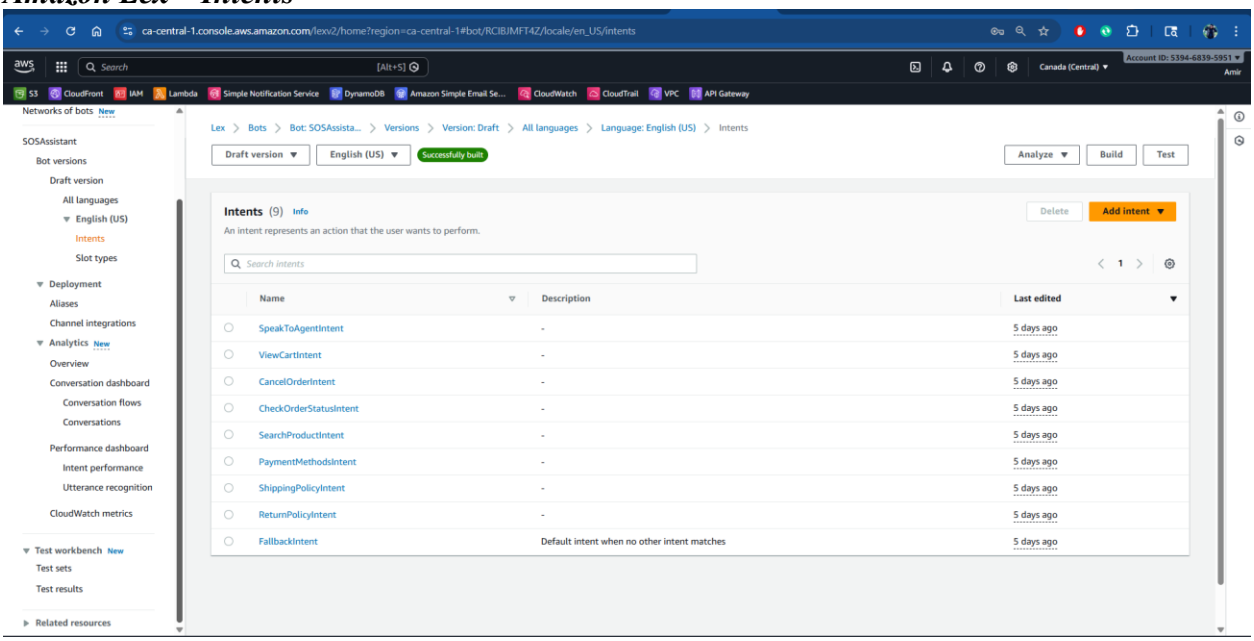


Figure 71: Amazon Lex - Intents

Amazon Lex – Conversation Flows

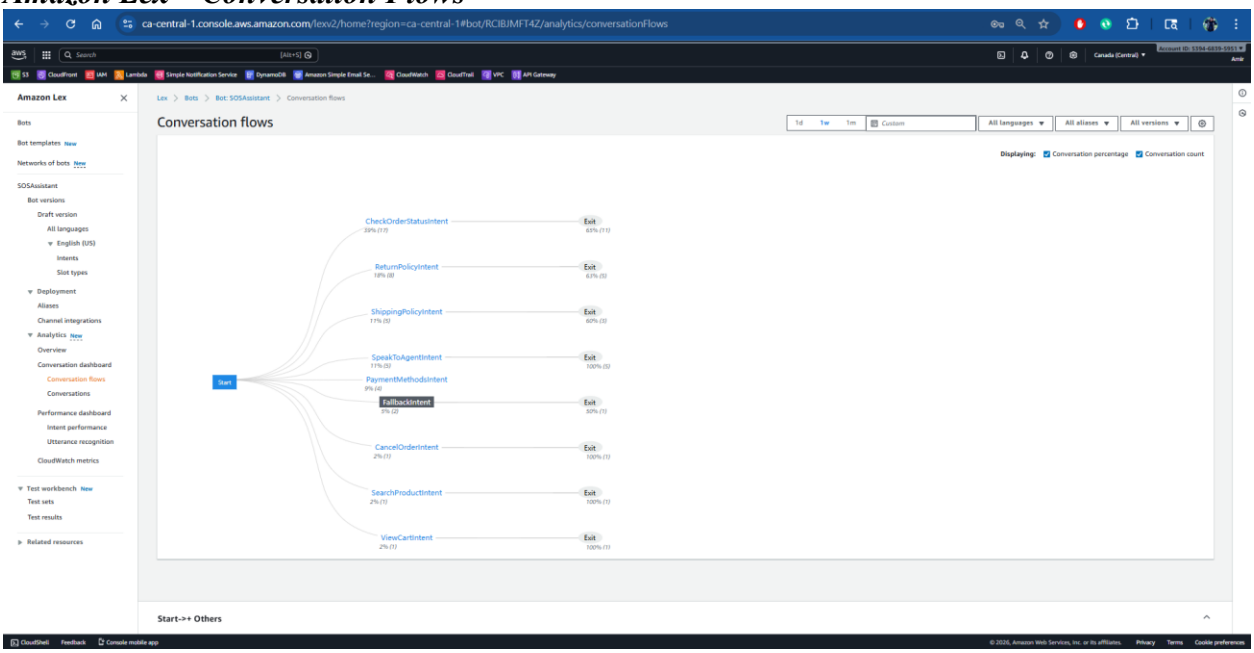


Figure 72: Amazon Lex – Conversation Flows

Amazon Lex – Conversations

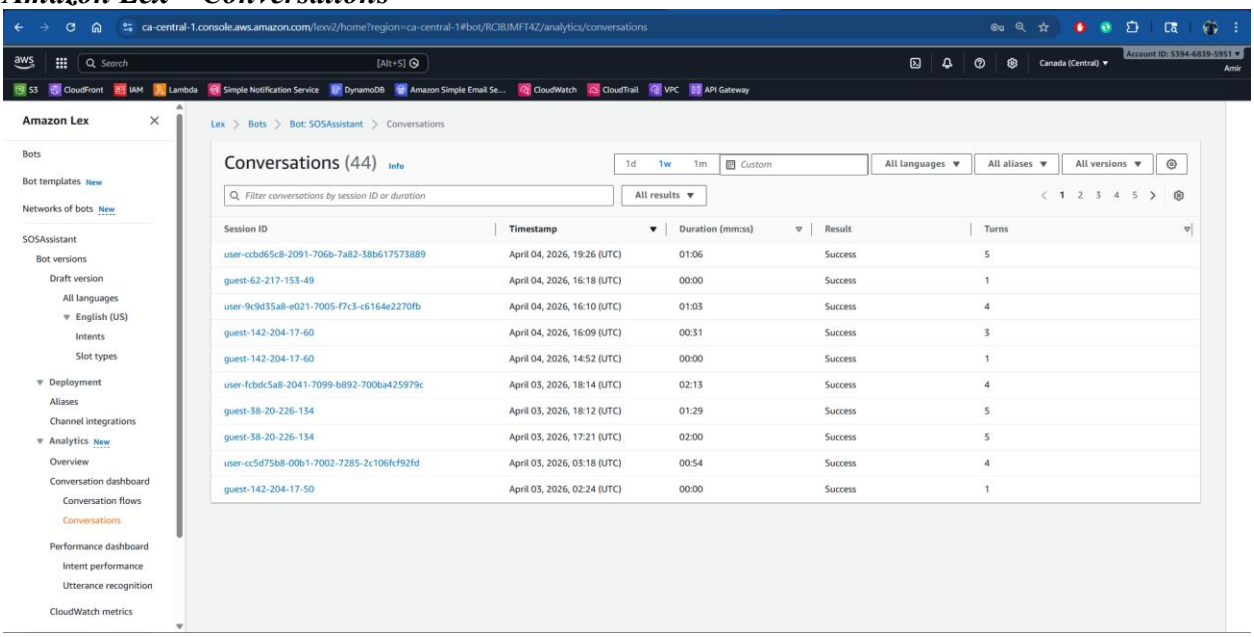


Figure 73: Amazon Lex – Conversations

Monitoring, Logging and Alerting Services

These services ensure system observability and tracking. In addition, real-time alerting is implemented using CloudWatch and SNS, which sends error notifications to a Telegram group using bot integration.

Amazon CloudWatch

CloudWatch collects logs and monitors performance across services.

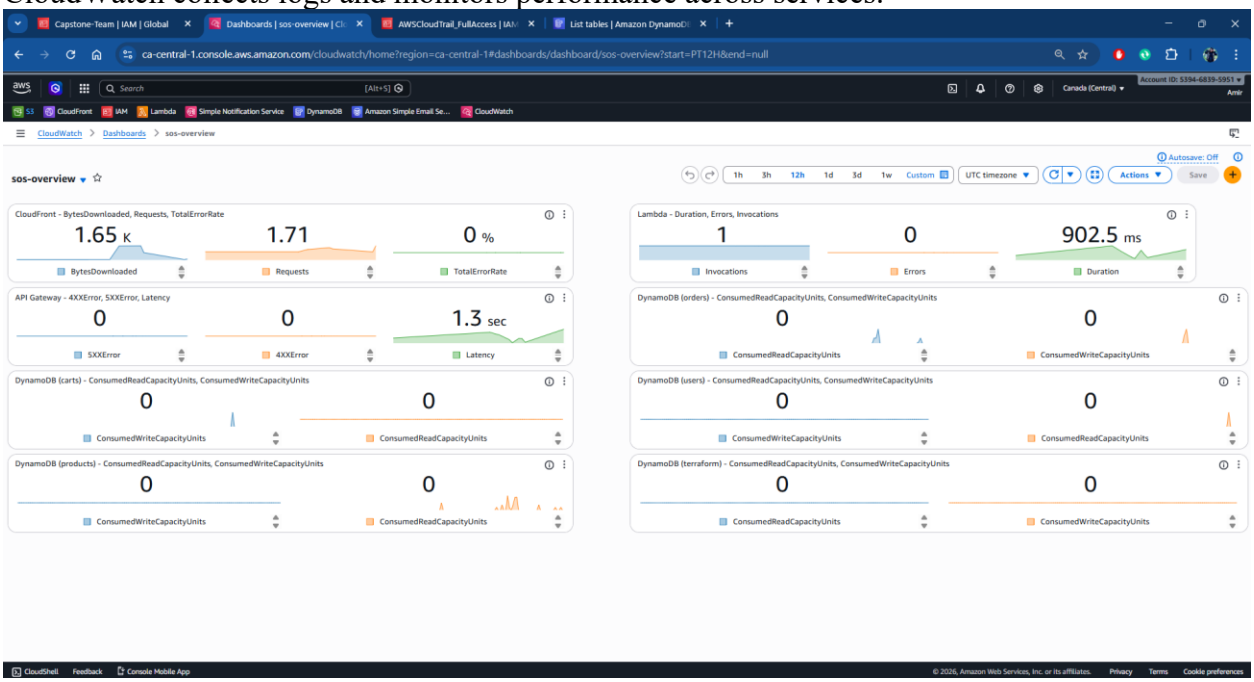


Figure 74: CloudWatch Overview

Cloudwatch – Log Management

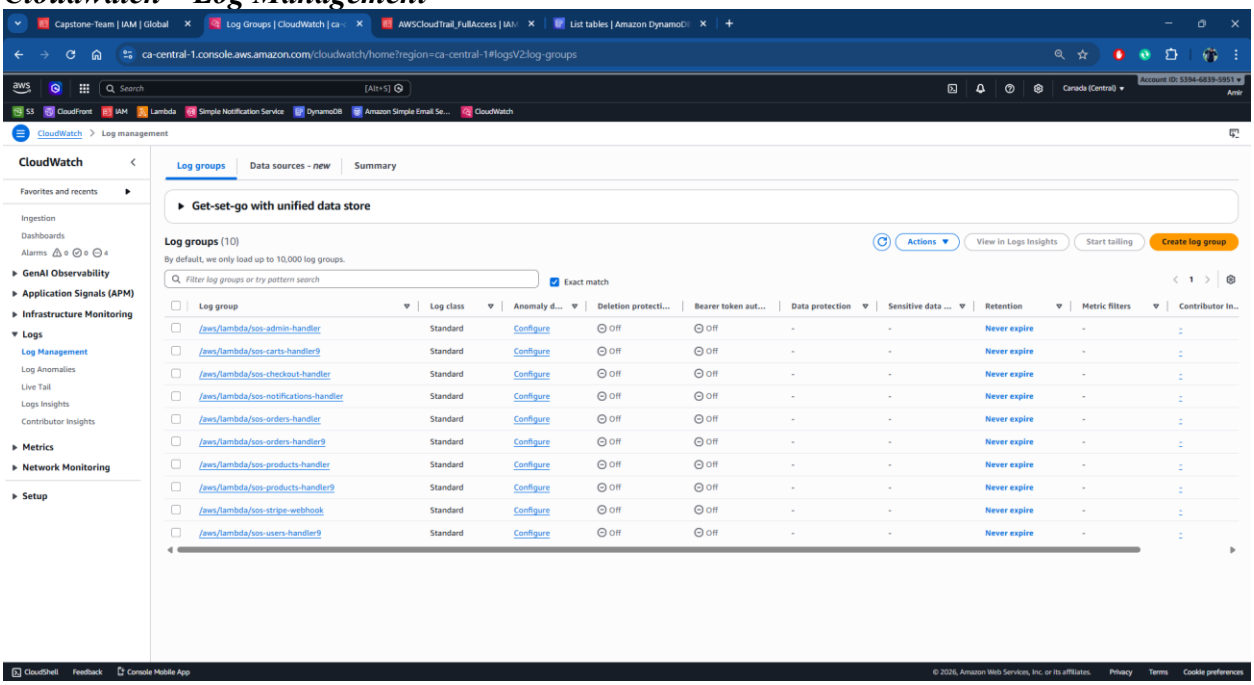


Figure 75: CloudWatch - Log Management

AWS CloudTrail

CloudTrail records all account activities for auditing and compliance.

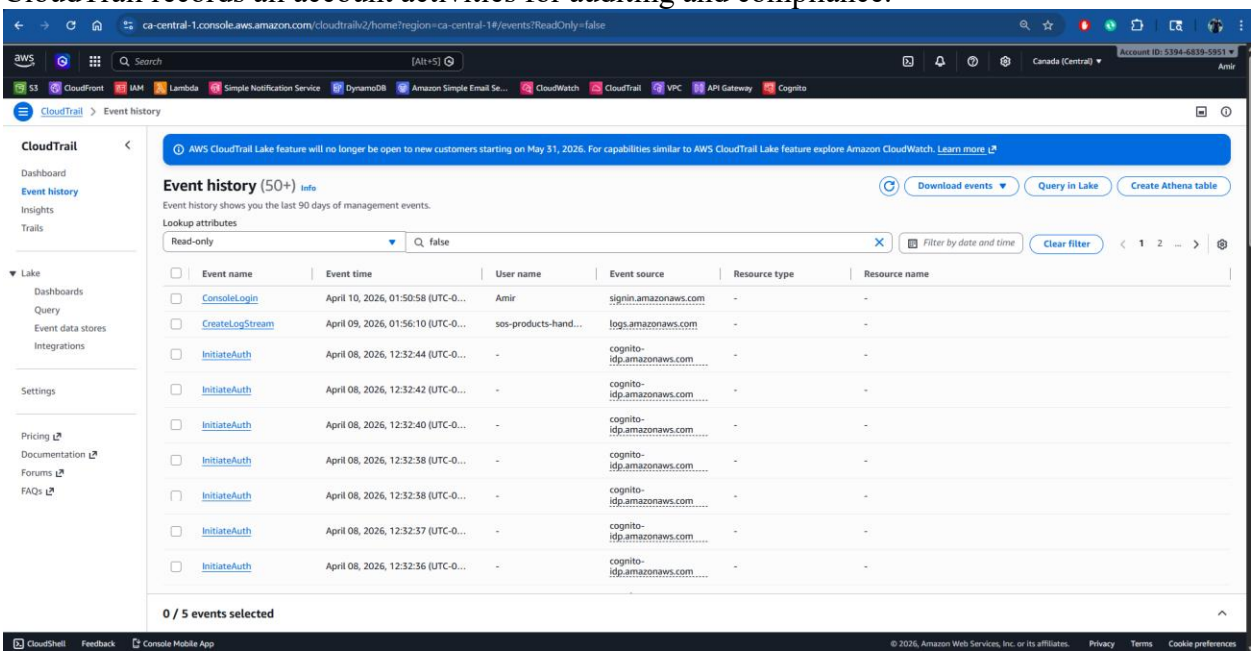


Figure 76: CloudTrail Event History

Telegram Notification Output

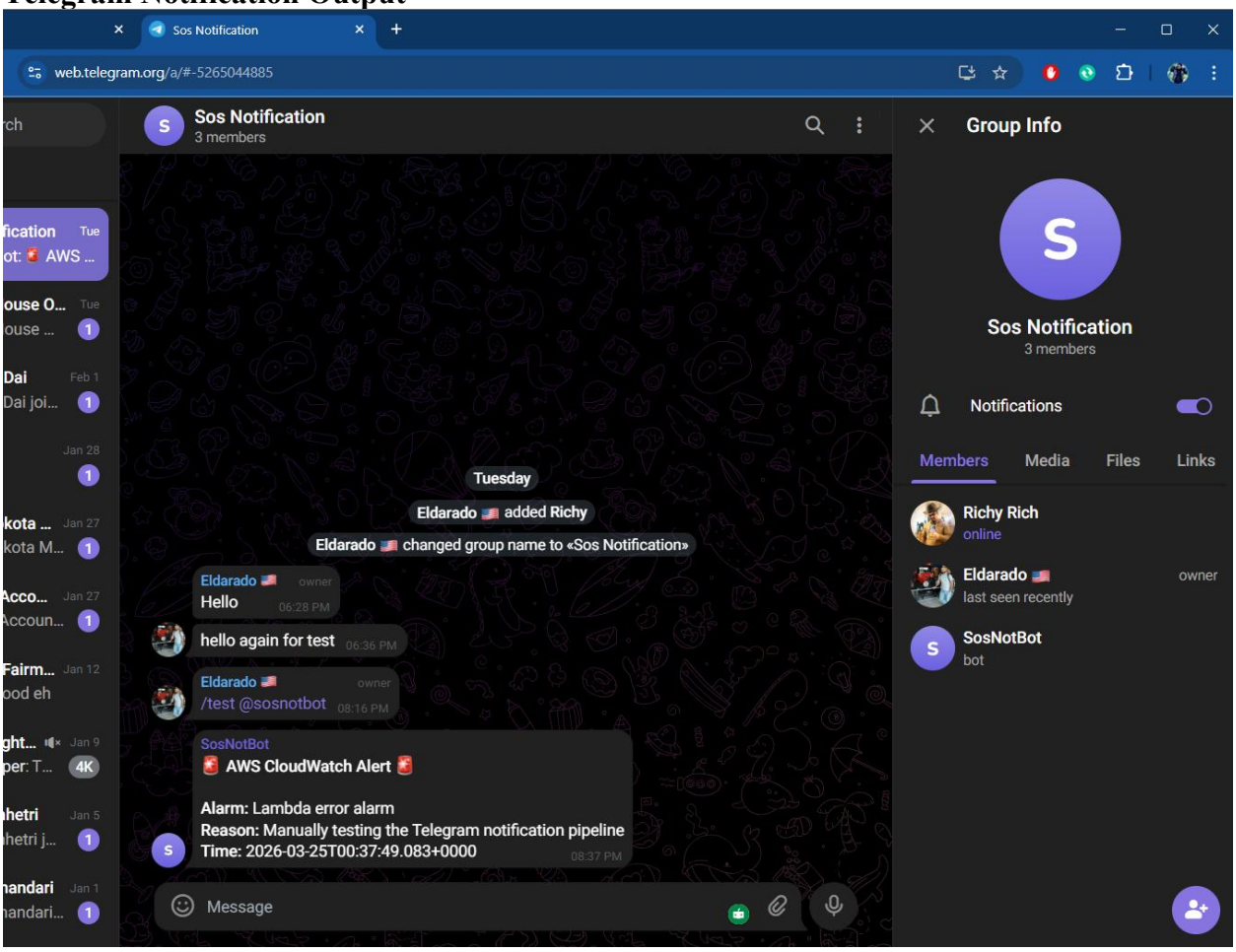


Figure 77: Telegram Notification Output

Security Services

These services ensure the system remains secure and follows best practices.

AWS IAM

IAM controls access permissions between AWS services.

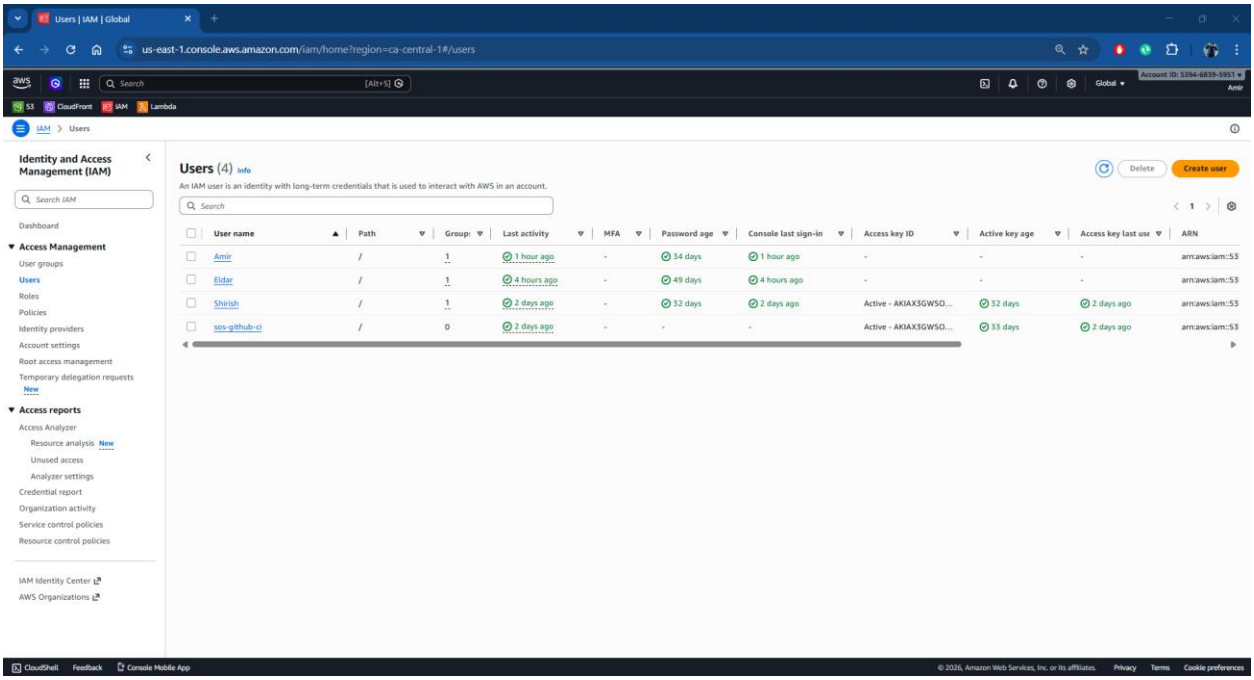


Figure 78: AWS IAM - Users

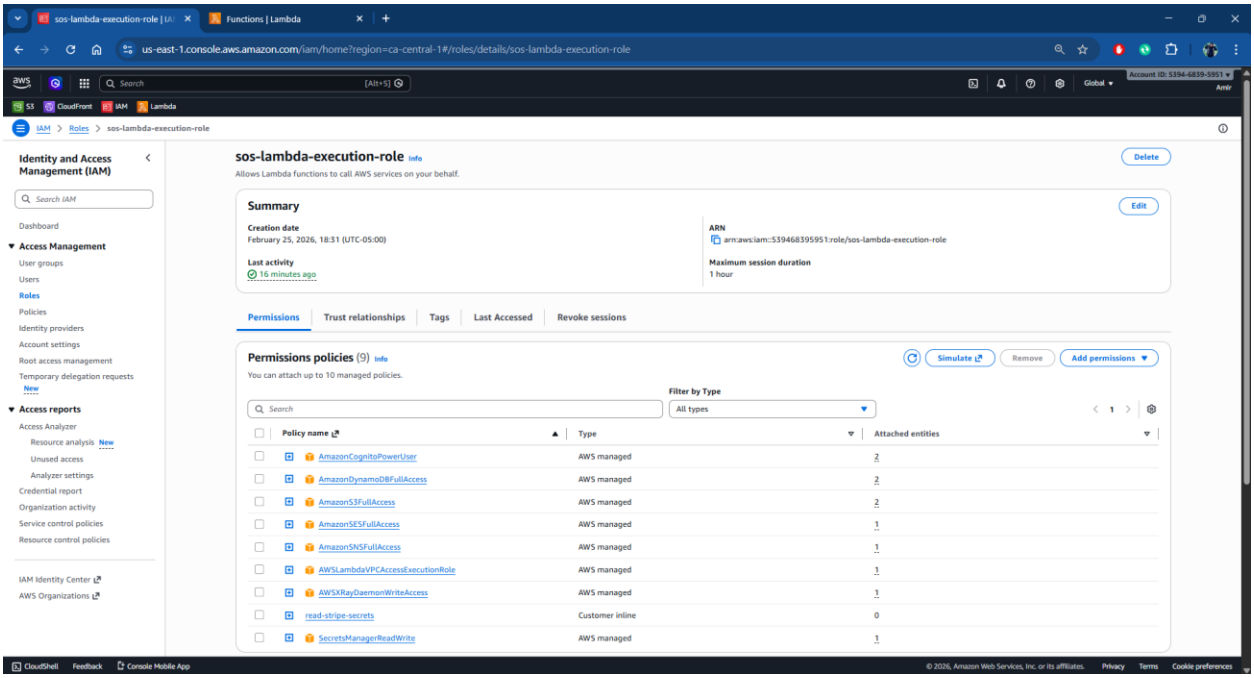


Figure 79: AWS IAM Roles (sos-lambda-execution-role)

AWS KMS

KMS is used to encrypt sensitive data stored in AWS services.

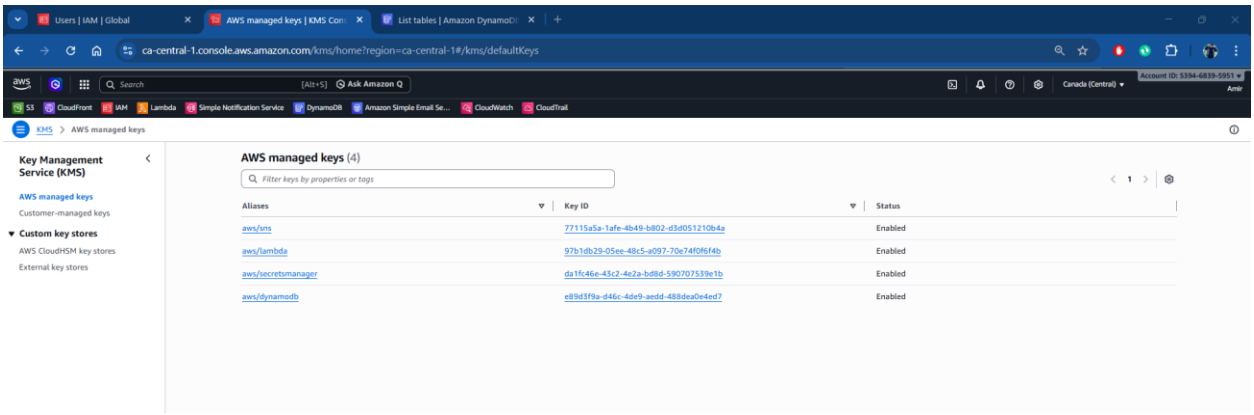


Figure 80: Amazon KMS – AWS Managed Keys

KMS – Customer Managed Keys

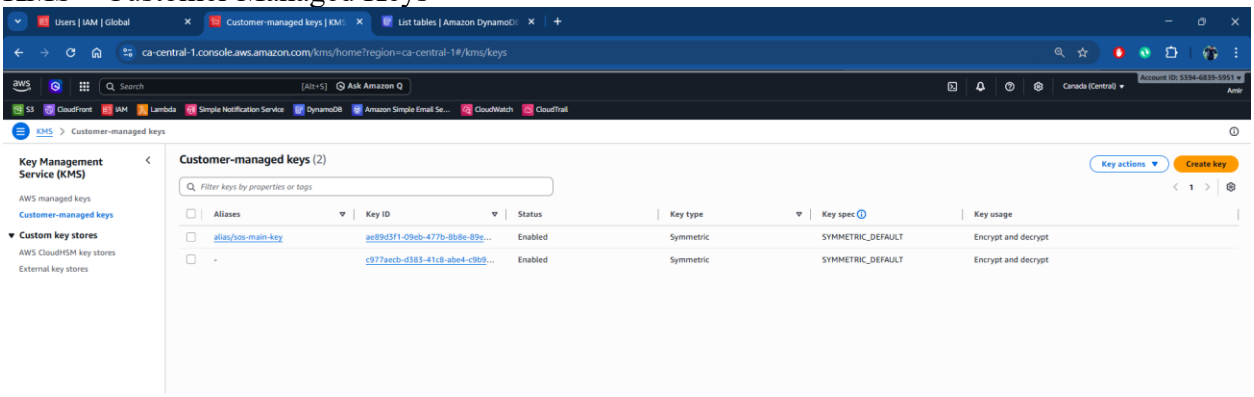


Figure 81: Amazon KMS – Customer Managed Keys

AWS Secrets Manager

Secrets Manager securely stores sensitive credentials such as API keys.

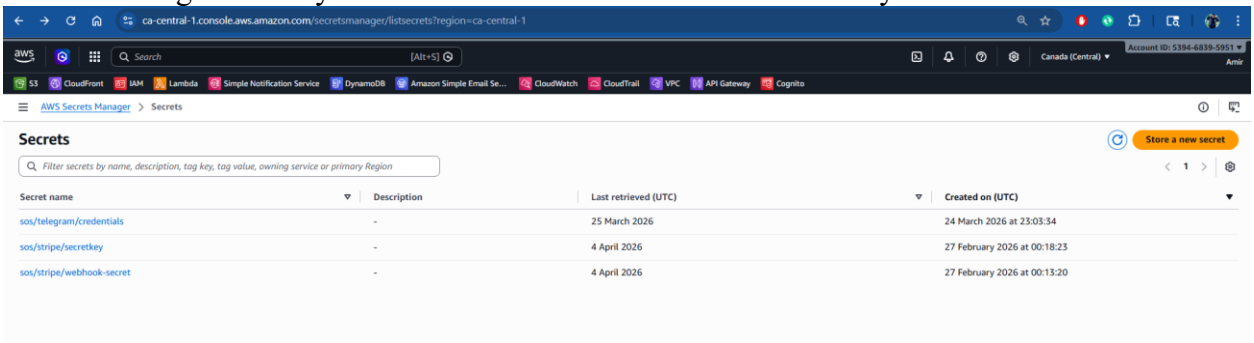


Figure 82: AWS Secrets Manager - Stored Secrets

DevOps and Infrastructure Management

Terraform is used as an Infrastructure as Code (IaC) tool to define and provision AWS resources in a repeatable and consistent manner. It enables automated infrastructure deployment and reduces manual configuration errors.

Terraform Implementation

Terraform is used to provision and manage AWS infrastructure in a consistent and automated manner. Instead of manually configuring resources through the AWS Console, infrastructure components such as IAM roles, backend storage, and supporting services are defined in code. The Terraform setup is integrated with a CI/CD pipeline using GitHub Actions, which securely authenticates to AWS using OpenID Connect (OIDC). This eliminates the need for storing long-term access keys. Terraform state is stored in Amazon S3, while DynamoDB is used for state locking to prevent conflicts during concurrent deployments. This ensures that infrastructure changes are tracked, version-controlled, and applied reliably.

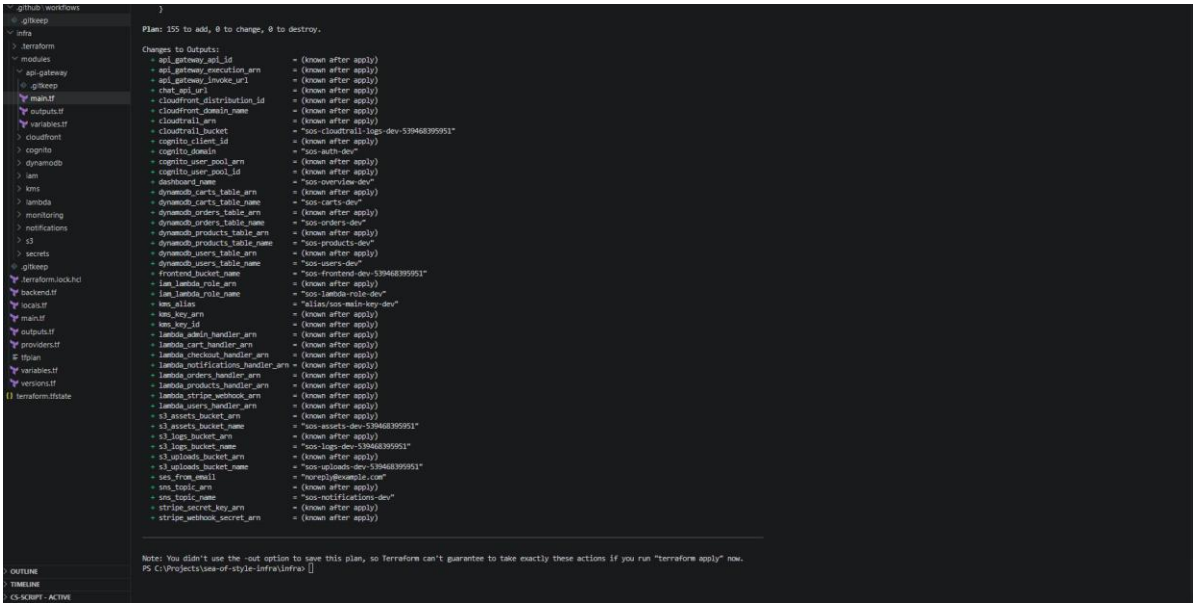


Figure 83: Terraform plan output

API Gateway Module

The api-gateway module defines the REST API that serves as the entry point for all client requests. The API is configured with a REGIONAL endpoint type, meaning it is deployed to a specific AWS region rather than distributed globally through CloudFront edge locations. A Cognito authorizer is wired directly to the Cognito user pool ARN, so every protected route validates the incoming JWT token against the user pool before the request reaches a Lambda function. API routes (such as /products) are defined as resources with method integrations pointing to their respective Lambda handlers.

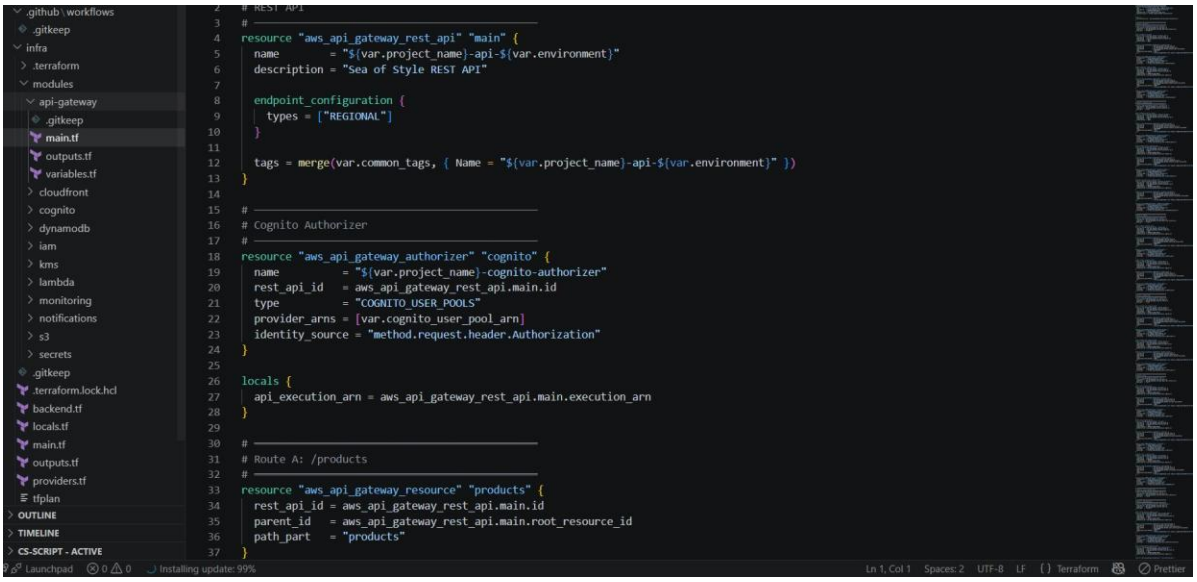


Figure 84: api-gateway - main.tf file

CloudFront Module: Frontend Hosting

The CloudFront module provisions the S3 bucket that hosts the SOS frontend, alongside all security and CDN configuration. The bucket name is built dynamically from the project name, environment, and AWS account ID to ensure global uniqueness. Versioning is enabled so every deployed build is preserved and can be rolled back if needed. All four public access block settings are set to true, meaning no object can ever be made publicly accessible directly through S3. Server-side encryption is applied with AES-256 so all stored assets are encrypted at rest. Public traffic reaches the frontend exclusively through the CloudFront distribution, which enforces HTTPS and provides edge caching.

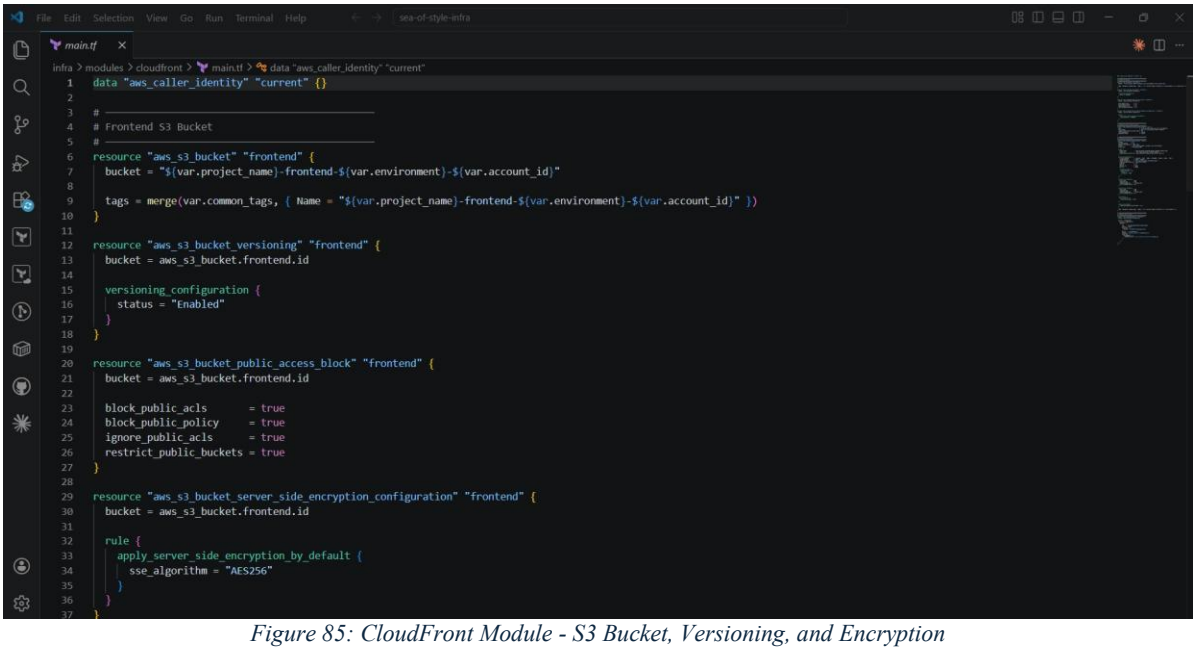


Figure 85: CloudFront Module - S3 Bucket, Versioning, and Encryption

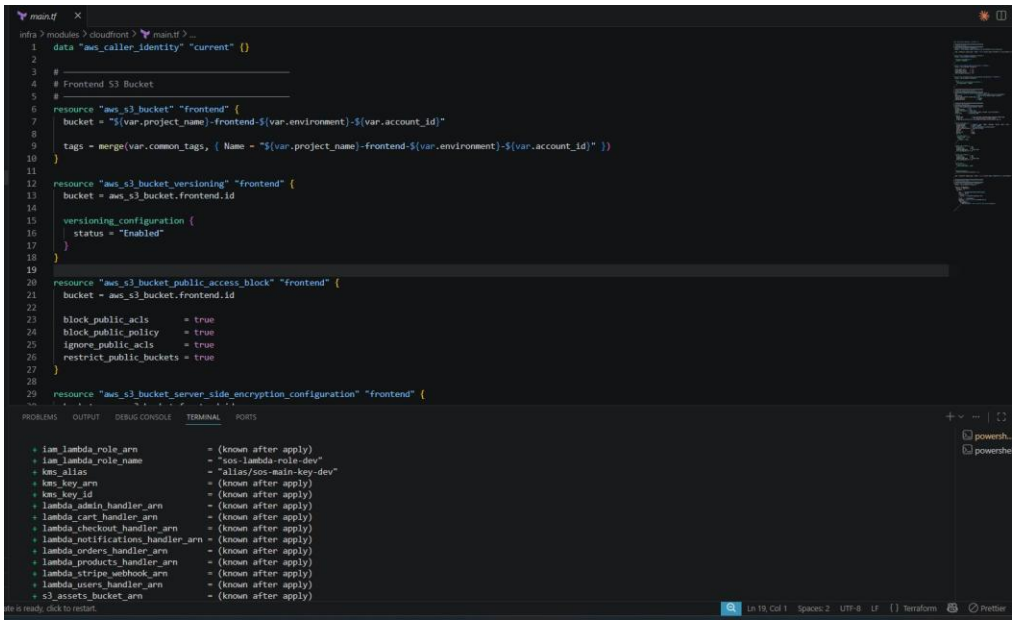


Figure 86: CloudFront Module with Plan Output

Cognito Module: Authentication

User authentication for the SOS platform is managed entirely through Amazon Cognito. The user pool uses email as both the username attribute and the auto-verified attribute, meaning users sign up and log in with their email address and receive a verification email automatically. A password policy enforces a minimum length of 8 characters with uppercase, lowercase, and numeric characters required. Symbols are optional to balance security with usability. Account recovery is configured to use verified email as the sole recovery mechanism. Self-registration is enabled by setting allow_admin_create_user_only to false. The schema blocks define email and name as required, mutable String attributes, ensuring the user pool schema matches the application data model from day one without any manual console configuration.

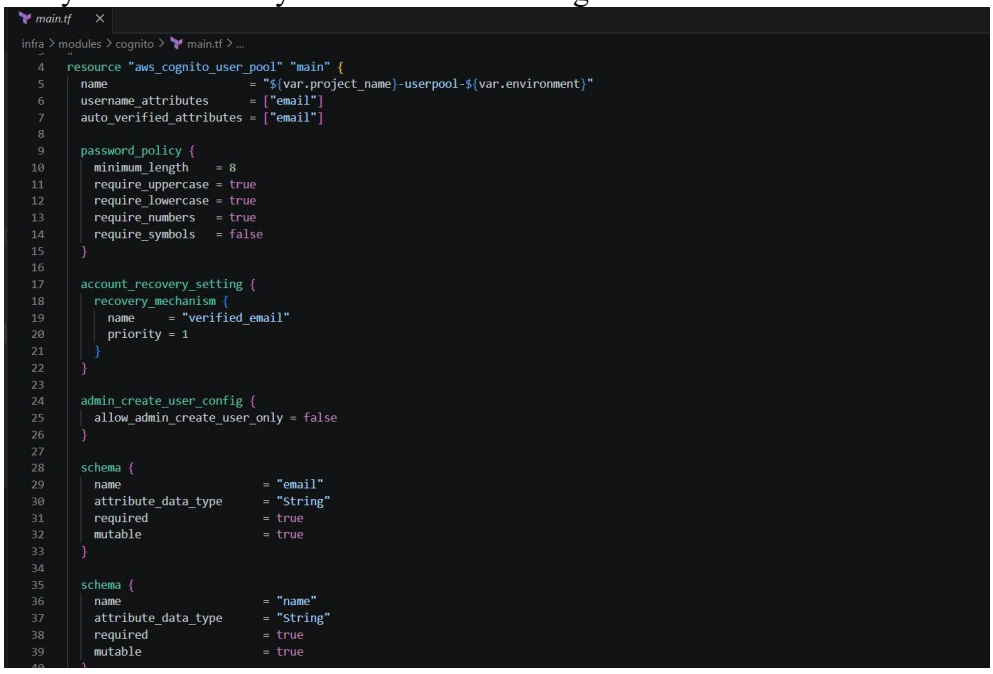
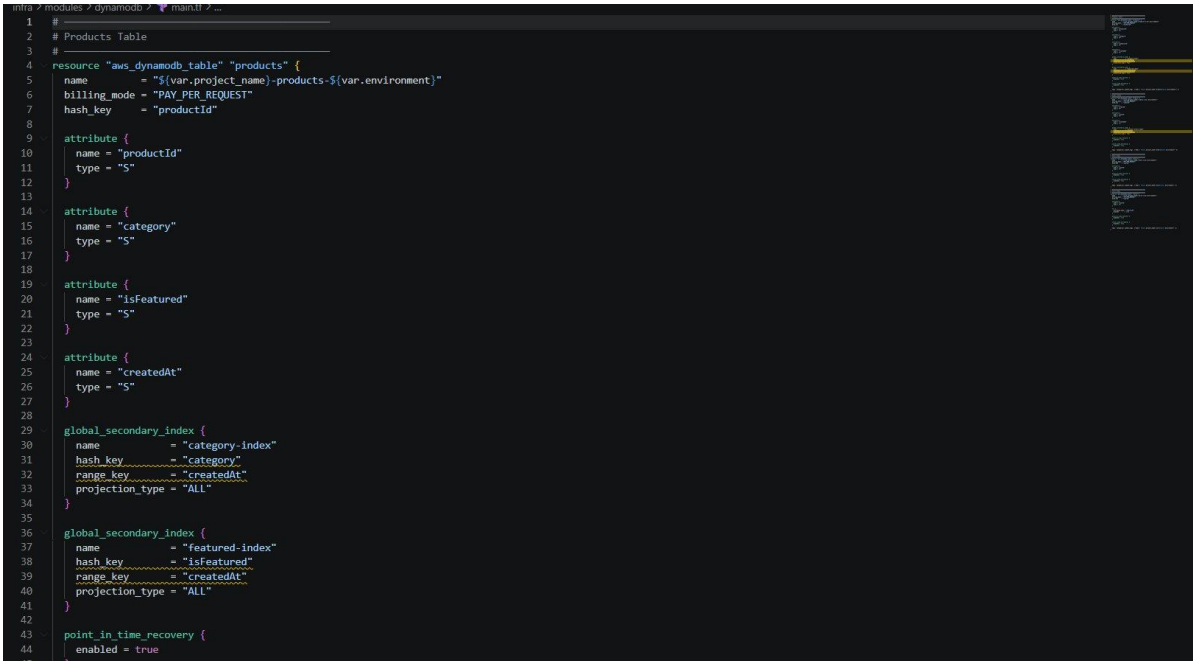


Figure 87: Cognito Module - User Pool Configuration

DynamoDB Module: Application Tables

All application data tables are defined in the DynamoDB module. Each table uses PAY_PER_REQUEST billing so capacity scales automatically with traffic and there is no need to manage provisioned throughput. Point-in-time recovery is enabled on every table, providing a continuous backup window without manual snapshots. The products table illustrates the design pattern used across all tables. The primary key is productId. Two Global Secondary Indexes are defined: category-index (partitioned by category, sorted by createdAt) and featured-index (partitioned by isFeatured, sorted by createdAt). These GSIs enable efficient filtered queries for product listings and the homepage without performing full table scans.



```
infra > modules > dynamodb > main.tf > ...
1   # ──────────────────────────────────────────────────────────
2   # Products Table
3   # ──────────────────────────────────────────────────────────
4   resource "aws_dynamodb_table" "products" {
5     name         = "${var.project_name}-products-${var.environment}"
6     billing_mode = "PAY_PER_REQUEST"
7     hash_key     = "productId"
8
9     attribute {
10      name = "productId"
11      type = "S"
12    }
13
14    attribute {
15      name = "category"
16      type = "S"
17    }
18
19    attribute {
20      name = "isFeatured"
21      type = "S"
22    }
23
24    attribute {
25      name = "createdAt"
26      type = "S"
27    }
28
29    global_secondary_index {
30      name            = "category-index"
31      hash_key        = "category"
32      range_key       = "createdAt"
33      projection_type = "ALL"
34    }
35
36    global_secondary_index {
37      name            = "featured-index"
38      hash_key        = "isFeatured"
39      range_key       = "createdAt"
40      projection_type = "ALL"
41    }
42
43    point_in_time_recovery {
44      enabled = true
```

Figure 88: DynamoDB Module - Products Table with GSIs

IAM Module: Execution Role and Policies

All Lambda functions share a single execution role provisioned by the IAM module. The trust policy allows only the lambda.amazonaws.com service principal to assume the role via sts:AssumeRole, ensuring no other service can use these permissions. A least privilege inline policy is attached to the role and broken into named permission statements. The CloudWatchLogs statement grants only the three log actions needed for Lambda to write execution logs, scoped to the project log group prefix. The S3BucketAccess statement grants only s3:GetObject and s3:PutObject, limited to the relevant buckets. Additional statements cover DynamoDB, Cognito, KMS, Secrets Manager, SNS, and SES, each scoped to the minimum actions required. Every permission change goes through code review before reaching AWS.

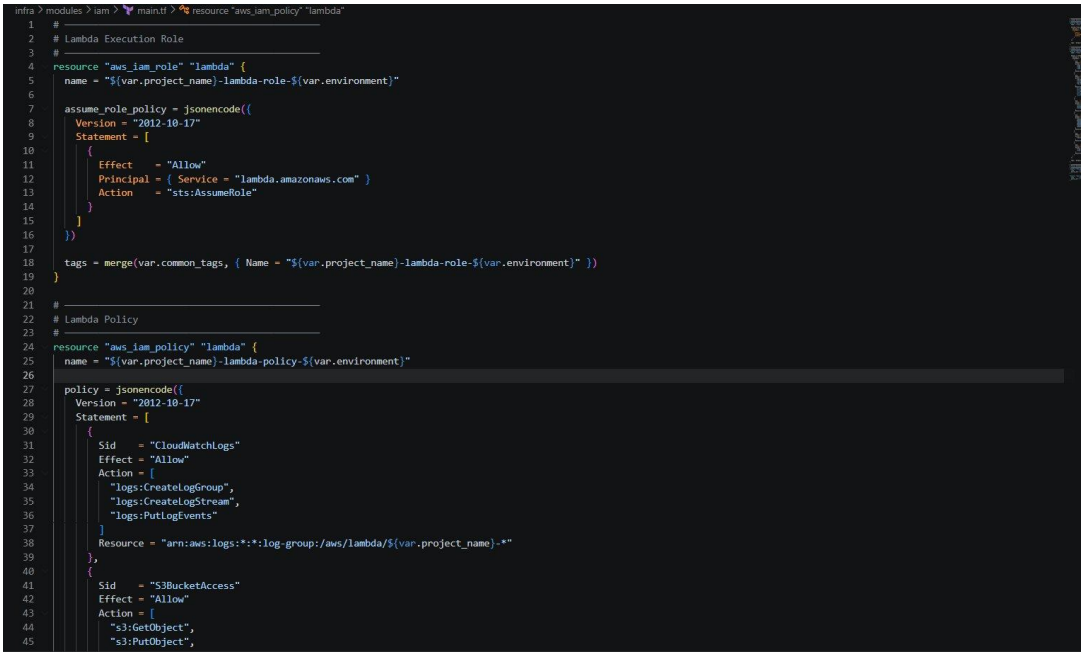


Figure 89: IAM Module - Lambda Role and Policy

Lambda Module: Serverless Functions

The Lambda module provisions all 8 serverless functions. A shared locals block defines the common configuration applied to every function: Node.js 20.x runtime, ARM64 (Graviton) architecture for cost efficiency, 512 MB memory, a 30-second timeout, the shared IAM execution role, and X-Ray active tracing. This avoids repeating the same settings across every function resource. A placeholder ZIP archive is generated at plan time using the archive_file data source. This allows Terraform to create the Lambda functions on the first apply before any real deployment artifact exists. The CI/CD pipeline for application code then updates the function code separately via a deploy workflow. Each function resource inherits all settings from the locals block and adds only its unique function name, description, and handler path.

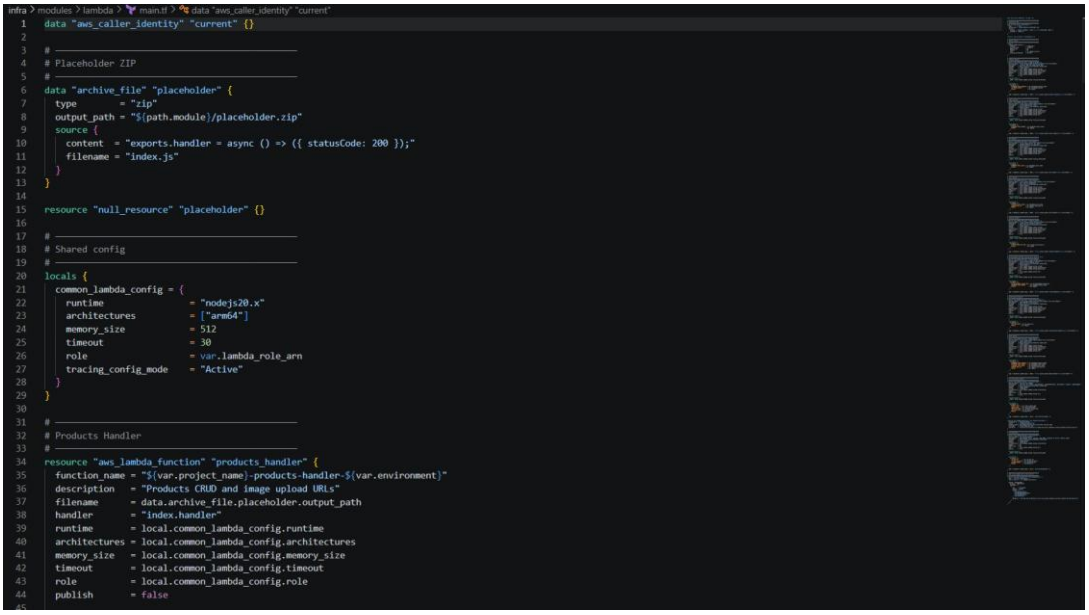


Figure 90: Lambda Module - Shared Config and Function Definition

Monitoring Module: CloudTrail and Audit Logging

The monitoring module configures AWS CloudTrail to capture a complete, tamper-resistant audit log of all API activity across the account. CloudTrail writes logs to a dedicated S3 bucket with its own specific bucket policy. The public access block on this bucket is set to fully private. The bucket policy defines two statements: AWSCloudTrailAclCheck allows CloudTrail to call s3:GetBucketAcl, and AWSCloudTrailWrite allows s3:PutObject for the AWSLogs path scoped to the account ID. A StringEquals condition on the PutObject statement restricts writes to this account only, preventing cross-account log injection. CloudWatch log groups for each Lambda function are also provisioned here, ensuring log retention policies are set at the infrastructure level.

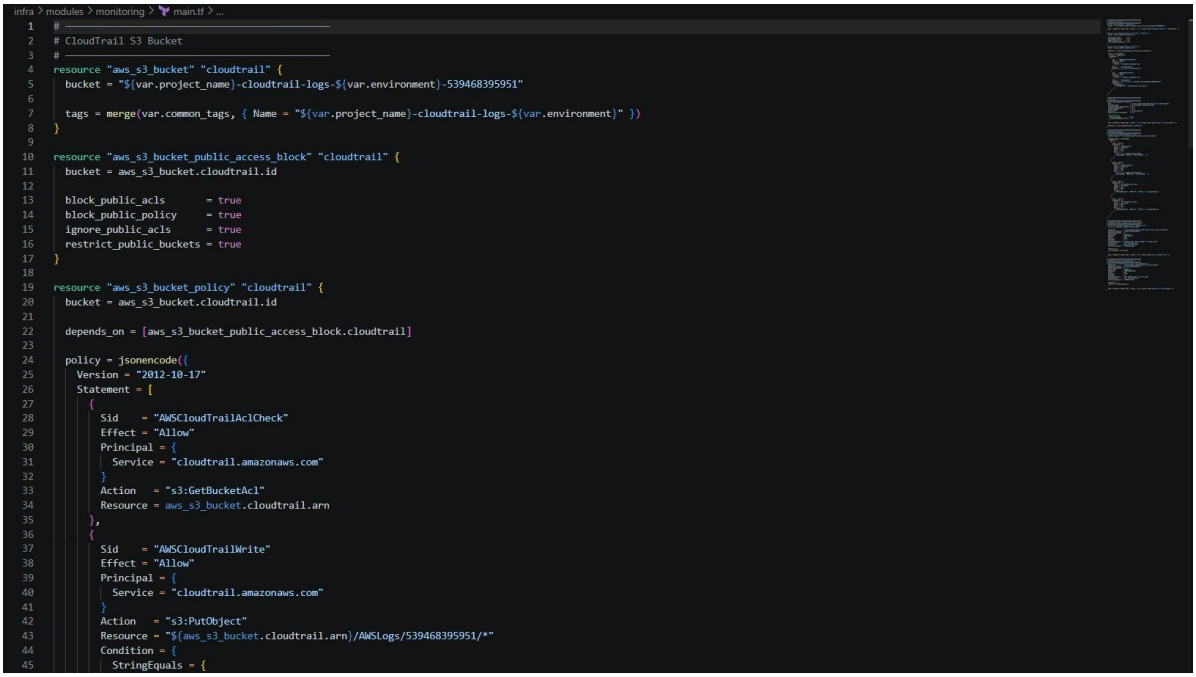


Figure 91: Monitoring Module - CloudTrail S3 Bucket and Policy

Notifications Module

The SOS platform uses Amazon SNS to enable event-driven communication within the system. When a user completes a purchase, an event is generated and processed through SNS, which triggers a notification workflow. Instead of sending email confirmations, the system forwards these notifications to a Telegram bot using a webhook integration, allowing administrators to receive real-time purchase alerts. For user account-related actions such as password recovery, Amazon Cognito is used. Cognito automatically sends verification emails containing a reset code, ensuring a secure and managed password reset process without requiring custom email handling.

```
infra > modules > notifications > main.tf > ...
# ─────────────────────────────────────────
# SNS Topic
# ─────────────────────────────────────────
resource "aws_sns_topic" "notifications" {
  name         = "${var.project_name}-notifications-${var.environment}"
  display_name = "SOS Notifications"

  tags = merge(var.common_tags, { Name = "${var.project_name}-notifications-${var.environment}" })
}

resource "aws_sns_topic_subscription" "admin_email" {
  topic_arn = aws_sns_topic.notifications.arn
  protocol  = "email"
  endpoint  = var.admin_email
}

resource "aws_sns_topic_subscription" "lambda" {
  topic_arn = aws_sns_topic.notifications.arn
  protocol  = "lambda"
  endpoint  = var.lambda_notifications_arn
}

resource "aws_lambda_permission" "sns_invoke" {
  statement_id  = "AllowSNSInvoke"
  action        = "lambda:InvokeFunction"
  function_name = var.lambda_notifications_arn
  principal     = "sns.amazonaws.com"
  source_arn    = aws_sns_topic.notifications.arn
}

# ─────────────────────────────────────────
# SES Email Identities
# NOTE: AWS will send verification emails to both addresses.
# Both must be verified before SES can send in sandbox mode.
# ─────────────────────────────────────────
resource "aws_ses_email_identity" "from" {
  email = var.ses_from_email
}

resource "aws_ses_email_identity" "admin" {
  email = var.admin_email
}

# ─────────────────────────────────────────
```

Figure 92: Notifications Module - SNS Topic and SES Identities

S3 Module: Data Buckets

The S3 module provisions three runtime data buckets: assets (product images and media), uploads (user-uploaded content), and logs (application and access logs). Each bucket follows the same security baseline as the frontend bucket: versioning enabled, full public access block applied, and AES-256 server-side encryption by default. Bucket names are constructed from the project name, environment, and AWS account ID for global uniqueness. Separating these buckets from the CloudFront module keeps concerns isolated. The frontend hosting lifecycle is independent of the data storage lifecycle, and each bucket can have different retention policies, lifecycle rules, or access controls applied without affecting the others.

```
infra > modules > s3 > main.tf > data "aws_caller_identity" "current"
data "aws_caller_identity" "current" {}

# ─────────────────────────────────────────
# Assets Bucket
# ─────────────────────────────────────────
resource "aws_s3_bucket" "assets" {
  bucket = "${var.project_name}-assets-${var.environment}-${data.aws_caller_identity.current.account_id}"

  tags = merge(var.common_tags, { Name = "${var.project_name}-assets-${var.environment}-${data.aws_caller_identity.current.account_id}" })
}

resource "aws_s3_bucket_versioning" "assets" {
  bucket = aws_s3_bucket.assets.id

  versioning_configuration {
    status = "Enabled"
  }
}

resource "aws_s3_bucket_public_access_block" "assets" {
  bucket = aws_s3_bucket.assets.id

  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls      = true
  restrict_public_buckets = true
}

resource "aws_s3_bucket_server_side_encryption_configuration" "assets" {
  bucket = aws_s3_bucket.assets.id

  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm = "AES256"
    }
  }
}

# ─────────────────────────────────────────
# Uploads Bucket
# ─────────────────────────────────────────
resource "aws_s3_bucket" "uploads" {
  bucket = "${var.project_name}-uploads-${var.environment}-${data.aws_caller_identity.current.account_id}"
```

Figure 93: S3 Module - Assets and Uploads Buckets

Secrets Module: KMS and Secrets Manager

The secrets module provisions the customer-managed KMS key used to encrypt sensitive data across the platform. Automatic key rotation is enabled, meaning AWS generates a new backing key every year without changing the key ARN or alias. The key policy defines three explicit permission statements: the root account receives full kms:* administrative access; the CloudTrail service receives kms:GenerateDataKey* and kms:Decrypt with a StringLike condition scoped to this account's trails; and the Lambda execution role receives kms:Decrypt so functions can read secrets from Secrets Manager at runtime. Stripe API keys are stored as secrets in AWS Secrets Manager and encrypted with this KMS key. By defining the key policy in code, every change to who can encrypt or decrypt data goes through the same pull request review process as all other infrastructure changes.

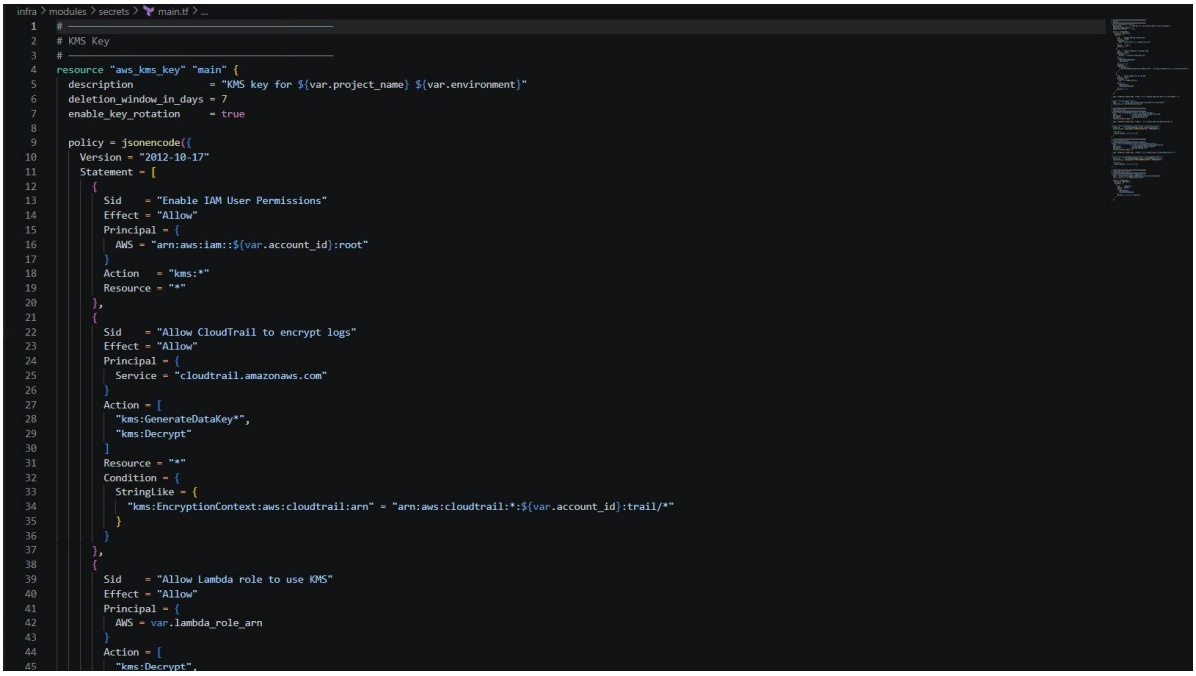


Figure 94: Secrets Module - KMS Key and Policy

CI/CD Pipeline and Automation

All Terraform operations run automatically through GitHub Actions. When a developer opens a pull request, Terraform generates a plan showing exactly what will change. On merge to main, it applies those changes to AWS automatically. No one touches the AWS console to make infrastructure changes.

AWS credentials are never stored as static secrets in GitHub. The pipeline uses an IAM OIDC identity provider, which grants the GitHub Actions runner a short-lived role with the minimum required permissions — only for the duration of the deploy run. Terraform state is stored in an S3 bucket in ca-central-1 with versioning enabled, preserving the full history of every infrastructure change. A DynamoDB table prevents two deployments from running at the same time and conflict with each other.

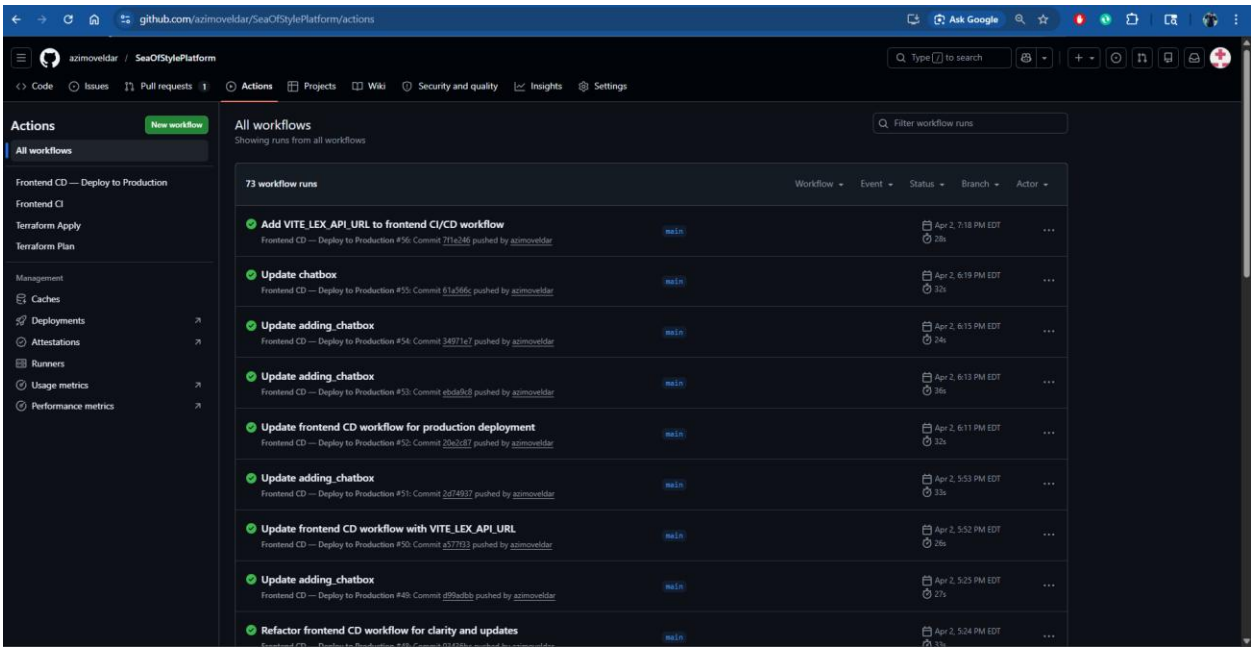


Figure 95: GitHub Actions Workflow

Why have we used these Service?

The selected AWS services were chosen to simplify development while ensuring the system remains scalable and efficient. By using serverless components such as AWS Lambda, API Gateway, and DynamoDB, there is no need to manage servers or infrastructure manually. This allows the application to automatically scale based on user demand and reduces operational complexity.

Amazon CloudFront is used to improve performance by delivering content from edge locations closer to users, resulting in faster load times. Amazon Cognito handles user authentication, removing the need to build and maintain a custom authentication system.

Overall, this approach focuses on reducing maintenance effort while improving performance, scalability, and reliability, making the system easier to manage and more suitable for real-world cloud applications.

AWS Well-Architected Framework Compliance

The SOS platform is designed in alignment with the AWS Well-Architected Framework, ensuring that the system follows industry best practices across all six pillars. The architecture leverages managed and serverless services to achieve a balance between performance, security, reliability, and cost efficiency.

Operational Excellence

The system is designed to be easy to monitor, manage, and continuously improve. Amazon CloudWatch is used to collect logs and track performance metrics, providing visibility into application behavior. In addition to basic monitoring, a real-time alerting mechanism is implemented. When a Lambda function encounters an error, a CloudWatch alarm is triggered,

which sends a notification through Amazon SNS and delivers it to a Telegram group using bot integration. This allows the team to respond quickly to issues as they occur. Furthermore, Terraform is used to manage infrastructure as code, and GitHub Actions is used for automated deployments. This ensures consistent deployment and reduces manual configuration errors.

Security
Security is implemented using a layered approach across the system. Amazon Cognito handles user authentication, including login, registration, and password recovery. The "Forgot Password" feature allows users to securely reset their credentials using email verification. AWS IAM is used to enforce role-based access control, ensuring that services only have the permissions they require. Sensitive data is encrypted at rest using AWS KMS.

Additionally, payment processing is handled through Stripe, which ensures that sensitive financial data is not stored within the system. AWS CloudTrail records all API activity, providing full visibility for auditing and security monitoring.

Reliability
The system is designed to remain available even during failures. AWS Lambda runs across multiple Availability Zones, ensuring that backend services continue to function without interruption. Amazon DynamoDB provides built-in replication and high availability for application data. CloudFront improves reliability by caching content and serving it even if backend services experience temporary issues. This combination ensures that the application remains accessible and stable under different conditions.

Performance Efficiency
Performance is optimized by using scalable and distributed services. CloudFront reduces latency by delivering content from edge locations closer to users. AWS Lambda automatically scales based on incoming requests, allowing the system to handle traffic spikes without performance degradation. DynamoDB provides fast and consistent data access, which is essential for real-time operations such as product browsing and order processing. The serverless design ensures that resources are allocated dynamically based on demand.

Cost Optimization
The architecture follows a pay-as-you-use model, ensuring that costs are aligned with actual usage. Services such as Lambda and API Gateway eliminate the need for continuously running infrastructure, reducing idle costs. DynamoDB on-demand pricing further ensures cost efficiency by scaling automatically based on workload. The current serverless architecture avoids fixed networking costs such as NAT Gateway. In more complex production designs, additional networking components may increase cost.

Sustainability
The serverless architecture contributes to sustainability by reducing unnecessary resource usage. AWS manages infrastructure efficiently, optimizing compute resources behind the scenes. By avoiding always-on servers and relying on event-driven execution, the system minimizes energy consumption and supports environmentally responsible cloud practices.

Cost Estimation

The SOS platform is designed using serverless architecture, which follows a pay-as-you-use pricing model. This ensures that costs are directly aligned with system usage, making the solution cost-efficient, especially for small to medium-sized workloads.

The following table provides an estimated monthly cost breakdown based on moderate usage, assuming approximately 1,000–2,000 users per month.

Estimated Monthly Cost Breakdown

Service	Usage Assumption	Estimated Monthly Cost (USD)
Amazon S3	Static website hosting (~5–10 GB storage + requests)	$1 – $3
Amazon CloudFront	Content delivery (~50–100 GB data transfer)	$5 – $10
Amazon Cognito	Up to 50,000 monthly active users (free tier applies)	$0
API Gateway	~1 million API requests	$3 – $5
AWS Lambda	~1 million requests (short execution time)	$1 – $3
DynamoDB	On demand (~low to moderate read/write traffic)	$3 – $6
Amazon SNS	Notification triggers (low usage)	<$1
Amazon SES	Email sending (within free tier limits)	$0 – $1
Amazon CloudWatch	Logs and monitoring	$2 – $5
AWS CloudTrail	Basic logging (mostly free tier)	$0
AWS KMS	Minimal key usage	$1
AWS Secrets Manager	Few secrets stored	$1 – $2
Amazon Lex	Low chatbot usage	$1 – $3
Terraform (S3 + DynamoDB backend)	State storage + locking	<$1
GitHub Actions	Free tier / minimal usage	$0

Total Estimated Monthly Cost: Approximately $20 – $45 USD per month
NOTE: (Cost may vary depending on traffic, usage patterns, and region.)

Cost Analysis

The overall cost of the system remains relatively low due to the use of serverless and managed services. Since services like AWS Lambda, API Gateway, and DynamoDB scale automatically, there is no need to pay for idle resources. This makes the architecture highly cost-efficient for applications with variable or unpredictable traffic. As traffic increases, services such as Lambda and API Gateway scale automatically, causing cost to grow proportionally with usage.

Additionally, several services such as Amazon Cognito and AWS CloudTrail operate within the AWS Free Tier under moderate usage, further reducing expenses. The use of on-demand pricing ensures that costs increase only when the system experiences higher traffic.

However, certain components, such as CloudFront data transfer and CloudWatch logs, can contribute to increased costs as usage grows. In a production environment, cost optimization strategies such as log retention policies, caching improvements, and efficient API design can help control expenses.

Conclusion

In conclusion, the SOS platform demonstrates a complete and practical implementation of a modern cloud-based application using AWS services. By adopting serverless architecture, the system can achieve high scalability, reduced operational overhead, and efficient resource utilization without the need to manage traditional infrastructure.

The integration of services such as Amazon S3, CloudFront, Cognito, API Gateway, Lambda, and DynamoDB enables the application to deliver a seamless user experience from browsing products to completing a purchase. Additional components like Amazon SNS, Telegram Integration and Lex enhance operational visibility and user interaction, while CloudWatch and CloudTrail provide visibility and control over system operations.

The architecture is designed with consideration for the AWS Well-Architected Framework, ensuring that best practices are followed in terms of security, reliability, performance, cost efficiency, and sustainability. Features such as automated deployments using Terraform, real-time monitoring with Telegram alerts, and secure authentication mechanisms further strengthen the overall solution.

Overall, this project reflects a well-structured and production-ready approach to building scalable cloud applications. This project not only demonstrates the ability to build a functional application but also reflects an understanding of how to design scalable, secure, and production-ready cloud solutions.

Questions and Answers:

a) How products are being updated on your website?

⇨ Products are updated through the admin side of the SOS platform. Authorized administrators can add, edit, or delete products using the admin dashboard. When an admin performs any of these actions, the request is sent through Amazon API Gateway to AWS Lambda functions, which process the request and updates the product data stored in Amazon DynamoDB. Once the database is updated, the changes are reflected on the website when users browse the product catalog.

b) Have you considered guest checking out? Or must all customers create their own account/profile first?

⇨ In the current implementation of the SOS platform, users must create an account and log in before placing an order. This approach was chosen to ensure secure access, maintain order history, and allow user-specific features such as profile management, password reset, and order tracking. Guest checkout was considered as a possible future enhancement, but it is not included in the current version of the project.

c) Are you saving customer financial information in your application?

⇨ No, the application does not store any customer financial information. Payment processing is handled securely through Stripe, which manages sensitive card and transaction details on its own platform. The SOS system only receives the payment result and transaction status required to process the order. This design reduces security risk and helps maintain a safer payment workflow.

d) How much is the monthly cost of your web application?

⇨ The estimated monthly cost of the SOS platform is approximately $20 to $45 USD under low to moderate usage. Since the application is built using serverless and managed AWS services, most of the cost depends on actual usage rather than fixed infrastructure. Services such as AWS Lambda, API Gateway, DynamoDB, S3, and CloudFront follow a pay-as-you-use model, which keeps the overall cost relatively low for small-scale deployments